MACHINE LEARNING FROM SCRATCH — MODULE 7

STATISTICS FOR MACHINE LEARNING

From Fundamentals to Advanced Statistical Learning

A Comprehensive Professional Reference Handbook

Kajola Gbenga Data Scientist & AI Engineer

How to Use This Module

This module assumes fluency in Python, NumPy, Pandas, Mathematics for ML (Module 5), and Probability for ML (Module 6). It does not re-teach those. Every major concept follows:

`CONCEPT → INTUITION → FORMULA → SYMBOLS → MANUAL EXAMPLE → PYTHON → INTERPRETATION → ML APPLICATION → ASSUMPTIONS → COMMON MISTAKES`

Code assumes `import numpy as np, pandas as pd; from scipy import stats; import statsmodels.api as sm`.

Table of Contents (Grouped)

Foundations (1–21): why statistics matters, population/sample, data types, measurement levels, data collection, descriptive stats, central tendency, dispersion, quantiles, boxplots, histograms, ECDF, skewness, kurtosis, z-scores, standard error, sampling methods, sampling bias, sampling distributions, CLT, LLN

Estimation & Inference (22–46): estimation, estimator properties, MLE, method of moments, confidence intervals (mean/proportion/difference/variance), hypothesis testing, hypotheses, p-values, error types, power, t-tests, z-test, chi-square, F-test, ANOVA, post-hoc, effect size, multiple testing

Correlation & Regression (47–66): nonparametric tests, correlation, covariance/correlation matrix, regression as statistical modeling, simple/multiple regression, assumptions, residuals, homoscedasticity, multicollinearity, outliers/influence, R^2/adjusted R^2, AIC/BIC, logistic regression, odds ratios, GLMs, Poisson/negative binomial regression

Resampling & Design (66–76): assumption-checking framework, bootstrap, jackknife, cross-validation, experimental design, A/B testing, causation vs correlation, Simpson's paradox, confounding

Bayesian & Advanced (77–98): Bayesian statistics, frequentist vs Bayesian, Bayesian estimation/regression, distribution fitting, Q-Q plots, goodness-of-fit, robust statistics, missing data, imputation, bootstrap uncertainty, statistical learning theory, bias-variance, ERM, MLE↔loss functions, regularization, feature selection, dimensionality reduction, multivariate statistics, PCA, factor analysis, clustering & statistics

Time Series & Production (99–123): time-series statistics, stationarity, autocorrelation, ARIMA intro, survival statistics, statistics for classification/regression/imbalanced data, model validation, uncertainty in metrics, model comparison, data drift, SPC, quality control, Python toolkit (Pandas/SciPy/statsmodels/scikit-learn), visualization, workflow, reporting, significance vs practical significance

Practice & Capstones (122–161): fallacies, troubleshooting, 7 end-to-end projects, simulation labs, exercises, coding challenges, interview prep, case studies, project structure, reproducibility, production testing, advanced theory (Fisher info, exponential families, asymptotics, decision theory), causal inference foundations, propensity scores, responsible AI statistics, ethics, formula sheet, distribution table, test-selection guide, ML method map, final capstone, final assessment, advanced challenges

Wrap-Up (157–165): career application, roadmap, memorize vs understand, cheat sheet, concept map, module connections, probability vs statistics, statistics vs ML, Module 8 preparation

1. Why Statistics Matters in Machine Learning

1.1 What Statistics Is

Statistics is the discipline of learning about a population from a sample: describing data, quantifying uncertainty, and drawing defensible conclusions that go beyond the specific numbers observed. Where Module 6's probability started from a known model and asked about likely data, statistics starts from **observed data** and asks what model or parameter plausibly produced it — the same inversion introduced in Module 5's likelihood section (Section 69), now developed into a full discipline.

1.2 Statistics vs Adjacent Fields

Field	Core Question
Probability	Given a known model, what data should we expect?
Statistics	Given observed data, what model/parameters are plausible?
Mathematics	What follows logically from axioms and definitions?
Data Analytics	What happened, described clearly? (mostly descriptive statistics)
Data Science	What will happen, and what should we do? (statistics + ML + engineering)
Machine Learning	How do we learn a predictive function from data, optimized for performance?

Descriptive statistics summarizes the data you have. **Inferential statistics** uses a sample to make justified claims about a population you don't fully observe. **Statistical modeling** prioritizes interpretability and uncertainty quantification (why did this happen, how confident are we); **predictive modeling** prioritizes forecasting accuracy on new data, often at the cost of interpretability — the two overlap heavily (a logistic regression is both a statistical model and a predictive model) but the emphasis differs.

1.3 The Chain

```
DATA → DESCRIPTION → SAMPLING → INFERENCE → MODELING → PREDICTION → DECISION
```

Every stage can silently corrupt everything downstream: a biased sample (Section 18) produces misleading description; ignoring sampling variability (Section 16) produces overconfident inference; violating a model's assumptions (Section 53, 66) produces an untrustworthy model; and even a well-validated model produces a bad *decision* if its uncertainty isn't communicated honestly (Section 120).

1.4 Why ML Without Statistical Thinking Misleads

Domain	Failure Without Statistical Thinking
Customer churn	Declaring a retention campaign "worked"

Domain	Failure Without Statistical Thinking
	from a small, non-random sample of responders, ignoring regression to the mean (Section 122)
Fraud detection	Reporting 99.9% accuracy on a 0.1% fraud rate, without recognizing this equals the majority-class baseline (Module 1, Section 4)
Healthcare	Treating a $p<0.05$ subgroup finding as clinically meaningful without correcting for the dozens of subgroups tested (Section 46)
Finance	Assuming historical volatility estimates are precise, ignoring the standard error of the estimate itself (Section 16)
Marketing	Ending an A/B test early because it "reached significance," inflating the false-positive rate through repeated peeking (Section 71)
Energy	Fitting a seasonal model without checking residual autocorrelation, producing systematically wrong prediction intervals (Section 101)
Supply chain	Treating a correlation between two operational metrics as causal and reallocating resources on that basis (Section 74)

Every one of these failures is a *statistical* failure hiding inside what looks like an ML or business problem — which is precisely why this module exists between Module 6 (Probability) and Module 8 (the ML Workflow): the workflow you're about to learn is only trustworthy if built on this foundation.

---

2. Population and Sample

2.1 Core Vocabulary

Term	Meaning
Population	The complete set of units you ultimately care about
Sample	A subset of the population actually observed
Census	Observing the *entire* population (rare, expensive, sometimes impossible)
Parameter	A numerical property of the population (fixed, usually unknown) — μ, σ, p
Statistic	A numerical property computed from the

Term	Meaning
	sample — $\bar{x}$, s, $\hat{p}$
Sampling frame	The literal list/mechanism from which the sample is drawn
Target population	Who you *want* to generalize to
Accessible population	Who you can *actually* sample from
Representative sample	A sample whose composition mirrors the population on characteristics that matter

2.2 Notation — Population vs Sample

Quantity	Population (parameter)	Sample (statistic)
Mean	μ	$\bar{x}$
Standard deviation	σ	s
Proportion	p	$\hat{p}$
Size	N	n

The hat and the bar both signal "estimated from data." A parameter is a fixed, usually-unknown truth; a statistic is a computed, known quantity that serves as our best guess at it. This distinction — introduced informally in Module 5's Section 68 — is the single most load-bearing piece of vocabulary in this entire module: nearly every later section (confidence intervals, hypothesis tests, standard errors) exists specifically to quantify the gap between a statistic and the parameter it estimates.

2.3 A Concrete Example

A telecom operator has **N = 2,000,000** active subscribers (the population). A data scientist pulls a random **n = 5,000** subscriber sample (the sample) to estimate the true churn rate p. The sample yields $\hat{p}$ = 4.3% — a statistic. The true population churn rate p is unknown and, realistically, unknowable exactly without a census. Everything from Section 16 onward is about how confidently we can say $\hat{p}$ = 4.3% approximates p.

Target vs accessible population, made concrete: if the operator wants to generalize to "all subscribers globally" (target) but can only pull data from subscribers in markets with a particular billing system (accessible), any inference is technically only valid for the accessible population — a distinction easy to overlook and a common source of overclaimed generalizability in ML projects.

3. Types of Data

Type	Definition	Example
Qualitative (categorical)	Describes a quality/category	`payment_method`, `region`
Quantitative (numeric)	Describes a measurable quantity	`income`, `tenure_months`
Discrete	Countable, often integer	`number_of_transactions`
Continuous	Any value in a range	`transaction_amount`
Cross-sectional	Many units, one point in time	A customer snapshot today
Time-series	One unit, many time points	Daily sales for one store
Panel (longitudinal)	Many units, many time points	Monthly churn status per customer over a year
Structured	Fits neatly into rows/columns	A SQL table
Semi-structured	Has some organization, not tabular	JSON logs
Unstructured	No inherent tabular structure	Free-text reviews, images

ML implication: panel data (a customer observed monthly for two years) requires a fundamentally different train/test splitting strategy than cross-sectional data — grouping by customer *and* respecting time order simultaneously (both Module 1's Section 18 group-splitting and Section 95's time-series splitting apply at once).

---

4. Levels of Measurement

Level	Definition	Example	Valid Operations	Appropriate Statistics	ML Implication
Nominal	Categories, no order	`country`, `payment_method`	Equality (=, ≠)	Mode, frequency	One-hot encode; never treat as ordinal integers
Ordinal	Categories, ordered, unequal gaps	`satisfaction: low<medium<high`	Equality, ordering (<, >)	Median, percentiles	Can integer-encode if gaps are treated as unknown; ordinal encoding preserves

Level	Definition	Example	Valid Operations	Appropriate Statistics	ML Implication
					order without implying equal spacing
Interval	Numeric, equal gaps, **no true zero**	Temperature in °C	+, −, ordering	Mean, SD	Ratios are meaningless (20°C is not "twice" 10°C)
Ratio	Numeric, equal gaps, **true zero exists**	Income, age, transaction amount	+, −, ×, ÷, ordering	Mean, SD, geometric mean, CV	Ratios and percentage changes are meaningful

The critical practical test: does "zero" mean "none of this exists"? Income of ₦0 means no income (ratio). 0°C does not mean "no temperature" (interval). Getting this wrong causes real errors — a common one is computing a *ratio* or *percentage change* from an interval-scaled variable (e.g. "temperature increased by 50%" from 10°C to 15°C is meaningless, since it depends on the arbitrary zero point of the Celsius scale).

Visualization by level: nominal → bar chart of counts; ordinal → ordered bar chart; interval/ratio → histogram, boxplot.

---

5. Data Collection

5.1 Collection Methods

Method	Description	Typical Use
Surveys	Directly asking respondents	Customer satisfaction
Experiments	Researcher controls treatment assignment	A/B tests (Section 71)
Observational studies	Data collected without controlled assignment	Most business/transactional data
Administrative data	Collected for operational, not research, purposes	Billing records
Transactional data	Generated by business transactions	Purchase logs
Sensor data	Automated physical measurement	IoT, equipment telemetry
Web/API data	Collected via web scraping or APIs	Pricing data, social data
Logs	System-generated event	Application logs

Method	Description	Typical Use
	records	
Streaming data	Continuously arriving	Real-time clickstreams

5.2 Data Quality Threats

Threat	Description	Example
Measurement error	The recorded value differs from the true value	A miscalibrated sensor
Missing data	Values absent (Section 85)	Unanswered survey questions
Selection bias	The sampling mechanism systematically favors some units	Surveying only app users, missing non-users
Response bias	Respondents systematically misreport	Social-desirability bias in surveys
Sampling bias	The sample doesn't represent the population (Section 18)	Convenience-sampling only engaged customers

The standing rule: the *analysis* can never fully fix a *collection* problem. Sophisticated statistics applied to badly collected data still produces badly justified conclusions — "garbage in, garbage out" holds for statistical inference exactly as it holds for ML.

---

6. Descriptive Statistics — Frequencies

```
df = pd.DataFrame({"plan":
["Basic","Premium","Basic","Standard","Premium","Basic","Standard"]})

freq = df["plan"].value_counts()
rel_freq = df["plan"].value_counts(normalize=True)
pct = rel_freq * 100
cum_freq = freq.cumsum()
cum_pct = pct.cumsum()

summary = pd.DataFrame({"freq": freq, "rel_freq": rel_freq.round(3),
                        "pct": pct.round(1), "cum_freq": cum_freq,
"cum_pct": cum_pct.round(1)})
print(summary)
#            freq  rel_freq   pct  cum_freq  cum_pct
# Basic         3     0.429  42.9         3     42.9
# Premium       2     0.286  28.6         5     71.4
# Standard      2     0.286  28.6         7    100.0
```

Term	Meaning
Frequency	Raw count per category
Relative frequency	Frequency ÷ total (a proportion)

Term	Meaning
Percentage	Relative frequency × 100
Proportion	Same as relative frequency, expressed as a fraction
Cumulative frequency	Running total of frequency, in a defined order
Cumulative percentage	Running total of percentage

Cumulative statistics are meaningful for **ordinal** data (a satisfaction scale — "what % rated 3 or below") but not generally meaningful for **nominal** data ordered alphabetically, where the ordering itself is arbitrary.

---

7. Measures of Central Tendency

7.1 The Four Means and When to Use Each

Measure	Formula	Use When
Arithmetic mean	$\bar{x} = (1/n)\Sigma x_i$	Symmetric data, no extreme outliers
Weighted mean	$\Sigma w_i x_i / \Sigma w_i$	Observations have different importance/reliability
Geometric mean	$(\Pi x_i)^{(1/n)}$	Growth rates, ratios, multiplicative processes
Harmonic mean	$n/\Sigma(1/x_i)$	Rates (e.g. average speed over equal distances)
Median	The middle value when sorted	Skewed data, outliers present
Mode	The most frequent value	Categorical data, or identifying a typical/common value

7.2 Worked Example — Salary Data With an Outlier

```
salaries = np.array([320_000, 340_000, 310_000, 330_000, 5_200_000])
# one executive outlier

print(f"Mean:   ₦{salaries.mean():,.0f}")      # Mean:   ₦1,300,000
print(f"Median: ₦{np.median(salaries):,.0f}")  # Median: ₦330,000
```

The mean (₦1,300,000) is pulled dramatically upward by one value and describes **nobody** in this dataset — no employee earns anywhere near that. The median (₦330,000) genuinely represents a typical employee. **This is the canonical justification for reporting median income/spend rather than mean whenever the distribution has a heavy right tail** (Section 13) — which most monetary variables (salary, transaction value, customer lifetime value) do.

7.3 Geometric Mean — Growth Rates

```
growth_rates = np.array([1.05, 1.12, 0.95, 1.08])   # year-over-year multipliers

arithmetic = growth_rates.mean()
geometric = growth_rates.prod() ** (1/len(growth_rates))
print(f"Arithmetic mean growth: {arithmetic:.4f}")   # 1.0500
print(f"Geometric mean growth:  {geometric:.4f}")    # 1.0470
```

Why geometric, not arithmetic, for growth rates: compounding 1.05 four times gives a different (correct) total return than raising the arithmetic mean to the 4th power. The geometric mean is the single constant growth rate that reproduces the same total compounded effect — the arithmetic mean systematically overstates typical multiplicative growth, a genuinely common error in financial and business reporting.

7.4 Skewness and the Mean-Median-Mode Relationship

Shape	Relationship
Symmetric	Mean ≈ Median ≈ Mode
Right (positive) skew	Mode < Median < Mean
Left (negative) skew	Mean < Median < Mode

Salary, transaction value, and customer spend are almost universally right-skewed — a few very large values pull the mean above the median, which is exactly what Section 7.2's example demonstrated.

---

8. Measures of Dispersion

8.1 The Catalogue

Measure	Formula	Interpretation
Range	max − min	Total spread; very outlier-sensitive
IQR	Q3 − Q1	Spread of the middle 50%; outlier-resistant
Variance	Section 8.2	Average squared deviation from the mean
Standard deviation	√variance	Spread in original units
Mean Absolute Deviation	$(1/n)\Sigma\lvert x_i - \bar{x}\rvert$	Average absolute deviation — more robust than variance
Coefficient of Variation	σ/μ (or $s/\bar{x}$)	Relative spread, unit-free — enables comparison across different scales

8.2 Population vs Sample Variance — Derived

```
Population:  σ² = (1/N) Σ(xᵢ − μ)²
Sample:      s² = (1/(n−1)) Σ(xᵢ − x̄)²      ← Bessel's correction
```

Why n−1: $\bar{x}$ is computed from the same data whose spread we're measuring, and $\bar{x}$ is, by construction, the value minimizing $\Sigma(x_i - c)^2$ over all constants c. Using the *sample* mean therefore systematically understates the true spread around the (unknown) population mean μ. Dividing by n−1 instead of n exactly corrects this bias — proven formally in Module 5's Section 60.2, revisited here in its full statistical context: s^2 is an **unbiased estimator** of σ^2 (Section 23), while the n-divided version is not.

```
x = np.array([2, 4, 4, 4, 5, 5, 7, 9])
print(x.var(ddof=0))   # 4.0    — population formula
print(x.var(ddof=1))   # 4.571  — sample formula (what you almost
always want)
print(np.std(x, ddof=1))  # 2.138
```

8.3 Coefficient of Variation — Comparing Across Scales

```
# Comparing variability of income (₦) vs tenure (months) is
meaningless in raw units
income = np.array([300_000, 450_000, 500_000, 550_000, 2_000_000])
tenure = np.array([2, 15, 24, 30, 48])

cv_income = income.std(ddof=1) / income.mean()
cv_tenure = tenure.std(ddof=1) / tenure.mean()
print(f"CV(income) = {cv_income:.3f}")   # 1.049
print(f"CV(tenure) = {cv_tenure:.3f}")   # 0.564
```

Income is *relatively* more variable than tenure in this sample (CV 1.05 vs 0.56), a comparison that raw standard deviations (in naira vs months) could never make directly.

---

9. Quartiles and Percentiles

9.1 Definitions

Term	Meaning
Percentile (P_k)	The value below which k% of the data falls
Quartile	The 25th, 50th, 75th percentiles (Q1, Q2, Q3)
Decile	The 10th, 20th, ..., 90th percentiles
Five-number summary	min, Q1, median, Q3, max
IQR	Q3 − Q1

```
transaction_amounts = np.array([50, 75, 100, 120, 150, 200, 250, 400,
900, 15000])  # fraud dataset

q1, median, q3 = np.percentile(transaction_amounts, [25, 50, 75])
```

```
iqr = q3 - q1
print(f"Q1={q1}, Median={median}, Q3={q3}, IQR={iqr}")
# Q1=106.25, Median=175.0, Q3=362.5, IQR=256.25

lower_fence = q1 - 1.5*iqr
upper_fence = q3 + 1.5*iqr
outliers = transaction_amounts[(transaction_amounts < lower_fence) |
(transaction_amounts > upper_fence)]
print("Flagged outliers:", outliers)     # [15000]
```

9.2 Applications

Salary analysis: reporting P10-P90 range gives a far more honest picture of a typical employee's pay than mean±SD, since salary is right-skewed (Section 7.2). **Customer spending:** identifying the top-decile ("P90+") customers for VIP treatment. **Fraud transaction amounts:** fraud often clusters at specific percentile ranges (either very small "testing" transactions or unusually large ones) — the five-number summary is a fast first diagnostic. **Model scores:** reporting the percentile distribution of predicted probabilities reveals whether a model's outputs are well-spread or collapsed toward one value.

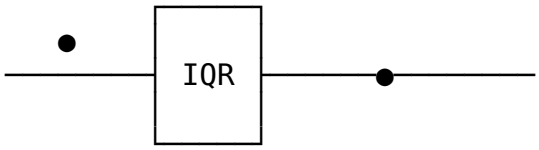
---

10. Boxplots

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.boxplot(transaction_amounts, vert=True)
ax.set_title("Transaction Amount Distribution")
# plt.show()

# Seaborn equivalent, grouped by category
import seaborn as sns
df = pd.DataFrame({"amount": transaction_amounts, "channel":
["online"]*5 + ["pos"]*5})
sns.boxplot(data=df, x="channel", y="amount")
```



outlier Q1 median Q3 whisker/outlier

Element	Meaning
Box	Q1 to Q3 (the IQR)
Line inside box	Median
Whiskers	Extend to the most extreme point within 1.5×IQR of the box

Element	Meaning
Points beyond whiskers	Flagged outliers

Interpretation habit: always compare box *widths* and *positions* across groups (e.g. churned vs retained customers) — a visibly higher median and narrower box for one group is often the fastest way to spot a genuinely discriminative feature before running any formal test.

---

11. Histograms

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, bins in zip(axes, [5, 20, 100]):
    ax.hist(transaction_amounts, bins=bins)
    ax.set_title(f"bins={bins}")
```

Term	Meaning
Bin	A contiguous value range grouped together
Bin width	The range each bin spans
Frequency histogram	Y-axis = counts
Density histogram	Y-axis scaled so total area = 1 (comparable to a PDF)

Bin width dramatically changes apparent shape: too few bins over-smooths and hides real structure (bimodality, gaps); too many bins shows noise as if it were signal. A common starting rule is the **Freedman-Diaconis rule**, `bin_width = 2·IQR·n^(-1/3)`, which scales bin width to both spread and sample size — better than a fixed bin count across differently-sized datasets.

```
bin_width = 2 * iqr * len(transaction_amounts) ** (-1/3)
n_bins = int((transaction_amounts.max() - transaction_amounts.min()) /
bin_width)
print(f"Suggested bins: {n_bins}")
```

---

12. Empirical Distributions

12.1 The Empirical CDF

```
F̂(x) = (number of observations ≤ x) / n
```

```
def ecdf(data):
    x = np.sort(data)
    y = np.arange(1, len(data)+1) / len(data)
    return x, y

x, y = ecdf(transaction_amounts)
plt.step(x, y, where="post")
plt.xlabel("Transaction amount"); plt.ylabel("Cumulative proportion")
```

```
# Probability from data, directly
prop_under_200 = (transaction_amounts <= 200).mean()
print(f"P(amount ≤ 200) ≈ {prop_under_200:.2f}")   # 0.60
```

Why ECDF beats a histogram for comparison: it requires no bin-width choice, and two ECDFs can be visually compared for a systematic shift (Section 82's Q-Q plots build directly on this idea) far more precisely than two histograms can. Comparing an empirical distribution to a **theoretical** one (does this data look normal? Poisson?) is the foundation of Section 81's distribution fitting and Section 83's goodness-of-fit tests.

13. Skewness

13.1 Definition and Formula

Moment-based (Fisher-Pearson) skewness:

$$\text{skew} = (1/n)\Sigma(x_i - \bar{x})^3 \;/\; s^3$$

Value	Shape
≈ 0	Symmetric
> 0	Right (positive) skew — long tail to the right
< 0	Left (negative) skew — long tail to the left

```
from scipy.stats import skew
print(skew(salaries))    # ~1.99 — strongly right-skewed, from the executive outlier

# Pearson's simpler approximation
pearson_skew = 3 * (salaries.mean() - np.median(salaries)) / salaries.std()
print(pearson_skew)
```

13.2 Why It Matters

Right-skewed variables (income, transaction amounts, wait times) violate the normality assumption many classical tests rely on (Section 36's t-test, Section 53's regression assumptions), motivating either a transformation (log transform is standard for right-skewed positive data) or a nonparametric alternative (Section 47).

```
log_salaries = np.log(salaries)
print(f"Raw skew: {skew(salaries):.2f}   Log-transformed skew: {skew(log_salaries):.2f}")
# Raw skew: 1.99   Log-transformed skew: 0.49 — much closer to symmetric
```

---

14. Kurtosis

14.1 Definition

$$\text{kurtosis} = (1/n)\Sigma(x_i - \bar{x})^4 \;/\; s^4 \qquad \text{excess kurtosis} = \text{kurtosis} - 3$$

Excess Kurtosis	Shape	Name
≈ 0	Normal-like tails	Mesokurtic
> 0	**Heavier tails, more extreme values**	Leptokurtic
< 0	Lighter tails, fewer	Platykurtic

Excess Kurtosis	Shape	Name
	extremes	

```
from scipy.stats import kurtosis
returns = np.random.default_rng(0).standard_t(df=3, size=10000)   # heavy-tailed by construction
print(f"Excess kurtosis: {kurtosis(returns):.2f}")   # large positive — heavy tails
```

14.2 Why Tail Behavior Matters in Finance and Risk

A normal distribution assigns vanishingly small probability to extreme moves; real financial returns exhibit **fat tails** — extreme moves happen far more often than a normal model predicts ("black swan" events). Using a normal-distribution-based Value-at-Risk (Section 127) on fat-tailed return data **systematically underestimates** the risk of large losses. This is not a theoretical nuance — it is a documented, material driver of real financial-model failures, and precisely why risk analytics teams check kurtosis before trusting any normal-based interval.

---

15. Z-Scores (Statistical Context)

Building directly on Module 5's Section 64:

```
z = (x − μ) / σ
```

Beyond the mechanics already covered: in inferential statistics, z-scores let us ask "how surprising is this specific value, if it truly came from a distribution with this mean and SD?" — a question directly answered by the empirical rule (68-95-99.7, Module 5 Section 63.3) and formalized into the entire hypothesis-testing framework beginning in Section 31.

```
customer_spend = np.array([200, 220, 210, 195, 205, 800])
z = (customer_spend - customer_spend.mean()) / customer_spend.std(ddof=1)
print(z.round(2))    # [-0.44 -0.40 -0.42 -0.45 -0.43  2.14]
```

The ₦800 spender is ~2.14 SDs above the mean — flagged, but not extreme enough under a strict 3σ rule (Module 5, Section 63.4) to be automatically called an outlier; a judgment call informed by, but not dictated by, the z-score alone.

---

16. Standard Error

16.1 Definition and Formula

```
SE(x̄) = σ/√n          (or s/√n when σ is unknown, the typical case)
```

16.2 Standard Deviation vs Standard Error — The Distinction Beginners Conflate

	Standard Deviation (s)	Standard Error (SE)
Measures	Spread of **individual observations**	Spread of the **sample mean** across repeated samples

	Standard Deviation (s)	Standard Error (SE)
Changes with n?	No (a property of the population)	**Yes — shrinks as 1/√n**
Answers	"How variable are individual customers' spend?"	"How precisely do I know the *average* spend?"

```
rng = np.random.default_rng(0)
population = rng.normal(500, 150, 1_000_000)   # a fixed population

for n in [10, 100, 1000, 10000]:
    sample = rng.choice(population, n)
    print(f"n={n:5d}: SD={sample.std(ddof=1):7.2f}
SE={sample.std(ddof=1)/np.sqrt(n):6.3f}")
# n=   10: SD= 145.32  SE=45.960
# n=  100: SD= 151.88  SE=15.188
# n= 1000: SD= 149.61  SE= 4.732
# n=10000: SD= 150.14  SE= 1.501
```

The standard deviation stays roughly constant (it estimates a fixed population property) **while the standard error shrinks steadily** — this is the direct, practical manifestation of Module 5's √n law (Section 65.3): quadrupling the sample halves the standard error. This single distinction underlies the correct interpretation of every confidence interval in Sections 26-30.

---

17. Sampling Methods

Method	Description	Pros	Cons	ML Implication
Simple random	Every unit equally likely	Unbiased, simple theory	Needs a full sampling frame	Default assumption behind most statistical formulas
Stratified	Sample within predefined subgroups, proportionally or by design	Guarantees subgroup representation; reduces variance	Requires known strata	Essential for imbalanced classification splits (Module 1, Section 18)
Cluster	Sample whole groups (e.g. all customers in randomly chosen branches)	Cheaper when a full frame is unavailable	Higher variance than SRS for the same n	Common for geographically distributed data collection
Systematic	Every kth unit from an ordered list	Simple to implement	Risk of hidden periodicity bias	Rare in ML pipelines
Multistage	Combine	Practical for	Complex	National

Method	Description	Pros	Cons	ML Implication
	methods hierarchically (cluster, then random within cluster)	very large/dispersed populations	variance calculations	surveys, large distributed systems
Convenience	Whoever is easiest to reach	Cheap, fast	High bias risk	Common (and risky) in early-stage data collection
Snowball	Existing participants recruit others	Useful for hard-to-reach populations	Severe selection bias	Rare in standard ML data
Quota	Fill predefined subgroup counts non-randomly	Ensures representation without a frame	Not probability-based; bias risk	Sometimes used in survey-based labeling

ML implication, generalized: whichever sampling method generated your *training data* determines what population your model can legitimately be said to represent. A model trained on convenience-sampled data (e.g. only customers who completed a voluntary survey) will silently underperform on the population it will actually be deployed against.

---

18. Sampling Bias

Bias Type	Description	Example
Selection bias	The sampling mechanism itself correlates with the outcome	Surveying only satisfied customers via a post-purchase email
Survivorship bias	Only "survivors" are observed; failures are invisible	Analyzing only currently-active accounts, missing churned ones entirely
Nonresponse bias	Respondents differ systematically from non-respondents	Busy, dissatisfied customers ignore surveys
Coverage bias	The sampling frame excludes part of the population	A phone survey misses customers without phones
Volunteer bias	Self-selected participants differ from the general population	Only highly engaged users opt into a beta programme
Sampling frame bias	The list used to sample is itself unrepresentative	Sampling from an outdated customer database

Why biased training data creates biased models — mechanically, not just morally: a model minimizes loss *on the training distribution it sees* (Module 1, Section 18.5's empirical risk minimization foundation, developed fully in Section 90). If that distribution systematically underrepresents a subgroup, the model will systematically underperform for that subgroup at deployment — the mathematics of "minimize average error" simply has no mechanism to protect a group it rarely saw.

```
# Demonstrating survivorship bias numerically
rng = np.random.default_rng(1)
n = 10_000
tenure = rng.exponential(scale=24, size=n)          # months, before
censoring
churned = tenure < 6                                  # many churn
early

# Biased analysis: only look at currently active customers
active_only_mean_tenure = tenure[~churned].mean()
true_mean_tenure = tenure.mean()
print(f"Mean tenure (active only, biased):
{active_only_mean_tenure:.1f} months")
print(f"Mean tenure (true, all customers): {true_mean_tenure:.1f}
months")
# The "active only" view systematically overstates typical tenure —
# exactly the mechanism Section 103's survival analysis exists to
correct.
```

---

19. Sampling Distributions

19.1 Definition

A **sampling distribution** is the distribution of a *statistic* (not raw data) computed across many hypothetical repeated samples — the theoretical object confidence intervals and hypothesis tests are built on.

19.2 Simulating the Sampling Distribution of the Mean

```
rng = np.random.default_rng(0)
population = rng.exponential(scale=50, size=1_000_000)   # skewed
population

sample_means = np.array([rng.choice(population, 30).mean() for _ in
range(5000)])
print(f"Population mean: {population.mean():.2f}")
print(f"Mean of sample means: {sample_means.mean():.2f}")     # ≈
population mean
print(f"SD of sample means (empirical SE): {sample_means.std():.2f}")
print(f"Theoretical SE = σ/√n: {population.std()/np.sqrt(30):.2f}")
# matches closely
```

19.3 Sampling Distribution of a Proportion

```
true_p = 0.042    # true churn rate
sample_props = np.array([rng.binomial(500, true_p) / 500 for _ in
range(5000)])
print(f"Mean of sample proportions: {sample_props.mean():.4f}")   # ≈
0.042
print(f"SD of sample proportions: {sample_props.std():.4f}")
print(f"Theoretical SE = √(p(1-p)/n): {np.sqrt(true_p*(1-
true_p)/500):.4f}")
```

Both simulations demonstrate the same universal fact: **the sampling distribution of a statistic centers on the true parameter and narrows as n grows** — the empirical basis for every confidence interval and test statistic that follows.

---

20. Central Limit Theorem in Statistics

Building directly on Module 5's Section 65 and Module 6's probability-first treatment — this section reframes it for statistical *practice*.

20.1 The Conditions, Stated Precisely

1. **Independence** — observations don't influence each other (violated by, e.g., panel data with repeated customers, or spatially/temporally correlated data)
2. **Finite variance** — the underlying distribution has a well-defined, finite σ^2 (violated by some heavy-tailed distributions)
3. **Sample size** — "large enough" depends on the population's skewness; symmetric distributions converge fast (n≈15-30 often suffices), heavily skewed ones need much larger n

20.2 Demonstration — Convergence Speed Depends on Shape

```
rng = np.random.default_rng(0)
symmetric_pop = rng.normal(0, 1, 1_000_000)
skewed_pop = rng.exponential(1, 1_000_000)

fig, axes = plt.subplots(2, 3, figsize=(15,6))
for row, pop, name in [(0, symmetric_pop, "Normal"), (1, skewed_pop,
"Exponential")]:
    for col, n in enumerate([2, 10, 30]):
        means = [rng.choice(pop, n).mean() for _ in range(2000)]
        axes[row, col].hist(means, bins=40)
        axes[row, col].set_title(f"{name}, n={n}")
```

The symmetric population's sample-mean distribution looks normal almost immediately (even n=2 is close); the exponential population's needs n≈30 before it visually approximates normal. This is the practical answer to "how big does n need to be" — it depends entirely on how far the population departs from symmetry.

20.3 Practical Limitations

The CLT is an **asymptotic** approximation — for genuinely small samples from heavily skewed or heavy-tailed data, it can be materially wrong, motivating either exact methods (Section 27's t-distribution, itself an exact result under normality, not a CLT approximation), nonparametric methods (Section 47), or bootstrap resampling (Section 67), which makes no distributional assumption at all.

---

21. Law of Large Numbers — Statistical Perspective

21.1 LLN vs CLT

	Law of Large Numbers	Central Limit Theorem
Claims	The sample mean **converges** to μ as $n \to \infty$	The sample mean's **distribution** approaches normal, at a known rate ($\sigma/\sqrt{n}$)
Answers	"Will my estimate eventually be right?"	"How precise is my estimate at a given n, and what shape is its uncertainty?"
Gives a confidence interval?	No	**Yes** — this is what makes CIs (Section 26) possible

```
rng = np.random.default_rng(0)
true_mean = 500
sample_means = [rng.normal(true_mean, 150, n).mean() for n in range(1,
5000, 10)]
plt.plot(range(1, 5000, 10), sample_means)
plt.axhline(true_mean, color='red', linestyle='--')
plt.xlabel("Sample size"); plt.ylabel("Running sample mean")
```

The running mean visibly settles toward the true value as n grows — LLN made visible — while the CLT is the complementary statement about *how far off* it's likely to be at any given n along the way. **Practical implication for ML:** LLN justifies that more training/validation data generally produces more stable metric estimates; the CLT (via Section 108's bootstrap) is what lets you actually quantify how stable a given metric estimate is at your current sample size.

22. Estimation

22.1 Point vs Interval Estimation

Point estimation produces a single best-guess number ($\bar{x}$ = 4.3% churn). **Interval estimation** produces a range with an attached confidence level (Section 26), honestly communicating the uncertainty a single point estimate hides entirely.

22.2 Estimator vs Estimate

Term	Meaning
Estimator	The *rule/formula/procedure* — e.g. "the sample mean"
Estimate	The *specific number* that rule produces from actual data — e.g. "4.3%"

This exactly mirrors Module 5's algorithm-vs-model distinction (Section 6.1): the estimator is the recipe; the estimate is the result of applying it once.

22.3 Bias, Variance, and MSE of an Estimator

```
Bias(θ̂) = E[θ̂] − θ
Var(θ̂)  = E[(θ̂ − E[θ̂])²]
MSE(θ̂)  = Bias(θ̂)² + Var(θ̂)
```

This is the **exact same decomposition** as Module 5's Section 73 bias-variance tradeoff for predictive models — here applied to a general parameter estimator rather than specifically to a model's predictions. A biased estimator is systematically wrong on average; a high-variance estimator is unreliable from sample to sample; MSE combines both into one comparable number.

```python
rng = np.random.default_rng(0)
true_mu = 100

# Two competing estimators of the mean of a sample of size 5
def estimator_A(sample): return sample.mean()                # the
usual sample mean
def estimator_B(sample): return sample.mean() * 0.9 + 5       # a
deliberately biased alternative

est_A = [estimator_A(rng.normal(true_mu, 15, 5)) for _ in range(5000)]
est_B = [estimator_B(rng.normal(true_mu, 15, 5)) for _ in range(5000)]

for name, ests in [("A (sample mean)", est_A), ("B (biased)", est_B)]:
    bias = np.mean(ests) - true_mu
    var = np.var(ests)
    mse = bias**2 + var
    print(f"{name}: bias={bias:.3f}  var={var:.3f}  MSE={mse:.3f}")
```

23. Properties of Estimators

Property	Definition	Why It Matters
Unbiasedness	$E[\hat{\theta}] = \theta$ exactly	No systematic error, on average
Consistency	$\hat{\theta} \to \theta$ as $n \to \infty$ (LLN-style convergence)	Guarantees more data eventually helps
Efficiency	Among unbiased estimators, has the smallest variance	The "best" unbiased choice, per the Cramér-Rao bound (Section 141)
Sufficiency	Uses *all* the information in the sample relevant to θ	No information is wasted
Robustness	Performance degrades gracefully under assumption violations	Real-world reliability, especially with outliers

A concrete tension: the sample mean is unbiased and efficient under normality but **not robust** — one extreme outlier can move it arbitrarily far (Section 7.2). The median is more robust but slightly less efficient under true normality. Choosing between them is precisely a bias/variance/robustness tradeoff — the same conceptual tradeoff appears throughout statistics, not just in ML models.

---

24. Maximum Likelihood Estimation — Applied Across Distributions

Building directly on Module 5's Section 70 and Module 6's likelihood treatment.

24.1 MLE for Common Distributions

Distribution	MLE Formula	Derivation Idea
Bernoulli	$\hat{p} = \bar{x}$ (sample proportion)	Maximize $\Sigma[y_i \log p + (1-y_i)\log(1-p)]$
Binomial	$\hat{p} = (\Sigma \text{ successes})/(n \times \text{trials})$	Same Bernoulli-family logic scaled up
Normal	$\hat{\mu} = \bar{x}$, $\hat{\sigma}^2 = (1/n)\Sigma(x_i - \bar{x})^2$	Maximize the Gaussian log-likelihood
Poisson	$\hat{\lambda} = \bar{x}$	Maximize $\Sigma[x_i \log \lambda - \lambda - \log(x_i!)]$
Linear regression	$\hat{\beta} = (X^T X)^{-1} X^T y$	MLE under Gaussian noise = OLS (Module 5, Section 74)
Logistic regression	No closed form; solved via gradient descent	MLE under Bernoulli likelihood (Module 5, Section 75)

```
data = np.array([4, 7, 5, 6, 8, 3, 5, 6])   # counts, assume Poisson
```

```
lambda_mle = data.mean()
print(f"λ^(MLE) = {lambda_mle}")   # 5.5 — for Poisson, MLE is simply
the sample mean

# Verify by brute-force likelihood maximization
from scipy.stats import poisson
lambdas = np.linspace(1, 10, 200)
log_liks = [poisson.logpmf(data, l).sum() for l in lambdas]
print(f"Grid-search maximum at λ={lambdas[np.argmax(log_liks)]:.2f}")
# confirms 5.5
```

24.2 The General Recipe, Reaffirmed

```
Likelihood L(θ) → log-likelihood ℓ(θ) → maximize over θ (analytically
or numerically) → θ^MLE
```

This is precisely Module 5's Sections 69-70, now shown across a wider range of distributions to reinforce that it is a *single, reusable recipe* — not a different technique per model.

---

25. Method of Moments

25.1 The Idea

Set **sample moments** equal to **theoretical moments** and solve for the parameters.

```
E[X] = x̄  (first moment: mean)
E[X²] = (1/n)ΣXᵢ²  (second moment)
```

Worked example — Gamma distribution (parameters shape k, scale θ, mean=kθ, var=kθ²):

```
data = rng.gamma(shape=3, scale=2, size=1000)   # true k=3, θ=2

mean_hat, var_hat = data.mean(), data.var(ddof=1)
# Solve: mean = kθ, var = kθ² → θ = var/mean, k = mean/θ = mean²/var
theta_hat = var_hat / mean_hat
k_hat = mean_hat**2 / var_hat
print(f"k̂={k_hat:.2f} (true 3), θ̂={theta_hat:.2f} (true 2)")
```

25.2 Method of Moments vs MLE vs MAP

Method	Approach	Typical Property
Method of Moments	Match sample moments to theoretical moments	Simple, sometimes less efficient than MLE
MLE	Maximize the likelihood of observed data	Usually asymptotically efficient (Section 141); the default choice
MAP	Maximize the *posterior* (likelihood × prior)	The Bayesian analogue of MLE (Section 79)

Method of moments is mainly valuable as a simple, fast starting point (or when MLE has no closed form and needs a good numerical initial guess) — MLE is the standard workhorse in modern practice.

---

26. Confidence Intervals

26.1 Definition and Formula

```
CI = point estimate ± (critical value) × (standard error)
```

26.2 The 95% Confidence Interval — Correctly Interpreted

Correct: *"If we repeated this sampling and interval-construction procedure many times, 95% of the resulting intervals would contain the true parameter."*

INCORRECT (a very common misconception): *"There is a 95% probability the true parameter lies within this specific, already-computed interval."*

Why the second statement is wrong: in the frequentist framework (Section 78), the parameter is a **fixed, unknown constant** — it either is or isn't in this particular interval, with no probability attached to that fact once the interval is computed. The 95% refers to the *procedure's long-run reliability*, not to this one realized interval. (Bayesian **credible intervals**, Section 79, *do* support the probability-of-containment interpretation people intuitively want — a genuine and important distinction between the two frameworks.)

```
rng = np.random.default_rng(0)
true_mu = 500
intervals_containing_truth = 0
n_sims = 200
fig, ax = plt.subplots(figsize=(6, 8))

for i in range(n_sims):
    sample = rng.normal(true_mu, 100, 30)
    se = sample.std(ddof=1) / np.sqrt(30)
    lo, hi = sample.mean() - 1.96*se, sample.mean() + 1.96*se
    contains = lo <= true_mu <= hi
    intervals_containing_truth += contains
    ax.plot([lo, hi], [i, i], color='green' if contains else 'red')

ax.axvline(true_mu, color='black', linestyle='--')
print(f"{intervals_containing_truth}/{n_sims} intervals contained the
true mean "
      f"({intervals_containing_truth/n_sims:.1%})")
# Approximately 95% — confirming the LONG-RUN interpretation,
visualized directly
```

This simulation is the single most convincing way to internalize the correct interpretation: about 5% of the intervals (shown in red) genuinely miss the true value, entirely by chance — which is exactly what "95% confidence" promises and nothing more.

---

27. Confidence Interval for a Mean

27.1 Known Variance — the Z Interval

```
CI = x̄ ± z_(α/2) × (σ/√n)
```

Rare in practice (σ is almost never truly known), but the conceptual starting point.

27.2 Unknown Variance — the t Interval (The Practical Default)

```
CI = x̄ ± t_(α/2, n−1) × (s/√n)
```

The **t-distribution** has heavier tails than the normal, widening the interval to honestly account for the *extra* uncertainty introduced by estimating σ with s rather than knowing it. As n grows, the t-distribution converges to the normal (fewer degrees of freedom penalty needed), which is why the z and t intervals nearly coincide for large samples.

```python
from scipy import stats

sample = np.array([320, 340, 310, 330, 355, 315, 325, 345, 300, 360])
n = len(sample)
mean, se = sample.mean(), sample.std(ddof=1)/np.sqrt(n)

t_crit = stats.t.ppf(0.975, df=n-1)
ci_t = (mean - t_crit*se, mean + t_crit*se)
print(f"t-based 95% CI: ({ci_t[0]:.2f}, {ci_t[1]:.2f})")

# scipy shortcut
ci_scipy = stats.t.interval(0.95, df=n-1, loc=mean, scale=se)
print(f"scipy confirms: {ci_scipy}")
```

When to use z vs t: use t whenever σ is unknown (nearly always) — the modern default is simply *always use t*, since it correctly reduces to approximately the z-interval at large n anyway, at no meaningful cost.

---

28. Confidence Interval for a Proportion

28.1 Normal Approximation (Wald Interval)

```
p̂ ± z_(α/2) × √(p̂(1−p̂)/n)
```

```python
successes, n = 43, 1000   # 43 churned out of 1000
p_hat = successes / n
se = np.sqrt(p_hat * (1-p_hat) / n)
ci_wald = (p_hat - 1.96*se, p_hat + 1.96*se)
print(f"Wald 95% CI: ({ci_wald[0]:.4f}, {ci_wald[1]:.4f})")
```

28.2 The Wilson Interval — Better for Small Samples or Extreme p

```python
from statsmodels.stats.proportion import proportion_confint
```

```
ci_wilson = proportion_confint(successes, n, alpha=0.05,
method='wilson')
print(f"Wilson 95% CI: {ci_wilson}")
```

Why Wilson is preferred, especially for small n or p near 0/1: the Wald interval can produce nonsensical bounds (below 0% or above 100%) when p̂ is near an extreme, and its coverage (actual long-run containment rate) degrades badly in exactly those cases. Wilson's interval is derived by inverting the score test rather than relying on the normal approximation directly, giving materially better real-world coverage — a genuine, well-documented, and easily-avoided pitfall in naive proportion CI code.

```
# Demonstrating the Wald interval's failure mode with a small, extreme
sample
successes_small, n_small = 1, 20
p_small = successes_small / n_small
se_small = np.sqrt(p_small*(1-p_small)/n_small)
print("Wald: ", (p_small - 1.96*se_small, p_small + 1.96*se_small))
# can go negative!
print("Wilson:", proportion_confint(successes_small, n_small,
method='wilson'))
```

---

29. Confidence Interval for Difference in Means

29.1 Independent Groups — Welch's Interval (Default)

$(\bar{x}_1 - \bar{x}_2) \pm t_{(\alpha/2,\ df)} \times \sqrt{s_1^2/n_1 + s_2^2/n_2}$

```
group_a = np.array([210, 220, 195, 230, 205, 215, 225])   # control
group_b = np.array([250, 260, 245, 270, 255, 265, 240])   # treatment

diff = group_b.mean() - group_a.mean()
se_diff = np.sqrt(group_a.var(ddof=1)/len(group_a) +
group_b.var(ddof=1)/len(group_b))

# Welch-Satterthwaite degrees of freedom
df_welch = (se_diff**4) / (
    (group_a.var(ddof=1)/len(group_a))**2/(len(group_a)-1) +
    (group_b.var(ddof=1)/len(group_b))**2/(len(group_b)-1)
)
t_crit = stats.t.ppf(0.975, df_welch)
print(f"Difference: {diff:.2f}, 95% CI: ({diff-t_crit*se_diff:.2f},
{diff+t_crit*se_diff:.2f})")
```

29.2 Pooled Variance (Assumes Equal Variances)

```
n1, n2 = len(group_a), len(group_b)
pooled_var = ((n1-1)*group_a.var(ddof=1) + (n2-1)*group_b.var(ddof=1))
/ (n1+n2-2)
se_pooled = np.sqrt(pooled_var*(1/n1 + 1/n2))
```

Welch vs pooled: Welch's version does **not** assume equal variances and is the modern statistical default even when variances look similar, since it costs almost nothing when variances genuinely are equal and protects you substantially when they aren't (mirroring Section 37's t-test guidance exactly).

29.3 Paired Samples

```
before = np.array([72, 68, 75, 70, 74])
after  = np.array([70, 65, 74, 66, 71])
diffs = after - before
se_paired = diffs.std(ddof=1) / np.sqrt(len(diffs))
t_crit = stats.t.ppf(0.975, len(diffs)-1)
print(f"Mean diff: {diffs.mean():.2f}, CI: ({diffs.mean()-
t_crit*se_paired:.2f}, "
      f"{diffs.mean()+t_crit*se_paired:.2f})")
```

Paired analysis reduces variance by removing between-subject variability entirely — appropriate whenever the same units are measured twice (before/after a campaign, Section 38).

---

30. Confidence Interval for Variance

30.1 The Chi-Square Distribution

```
(n−1)s²/χ²_(α/2, n−1)  <  σ²  <  (n−1)s²/χ²_(1−α/2, n−1)

sample = np.array([102, 98, 105, 110, 95, 108, 100, 103])
n = len(sample)
s2 = sample.var(ddof=1)

chi2_lower = stats.chi2.ppf(0.975, df=n-1)
chi2_upper = stats.chi2.ppf(0.025, df=n-1)
ci_var = ((n-1)*s2/chi2_lower, (n-1)*s2/chi2_upper)
print(f"95% CI for variance: ({ci_var[0]:.2f}, {ci_var[1]:.2f})")
```

Unlike the mean's CI, the variance CI is **asymmetric** around s^2 — a direct consequence of the chi-square distribution's skew — a detail worth knowing so an asymmetric result isn't mistaken for a computational error.

---

31. Hypothesis Testing — The Complete Workflow

31.1 The Full Process

```
1. State H₀ (null) and H₁ (alternative)
2. Choose significance level α (commonly 0.05)
3. Choose and compute a test statistic
4. Determine the p-value (or compare to a critical value)
5. Decide: reject H₀ if p < α, otherwise fail to reject
6. Interpret in context — NEVER "accept H₀"
```

31.2 The Vocabulary

Term	Meaning
Null hypothesis (H_0)	The default/no-effect claim being tested
Alternative hypothesis (H_1)	What you'd conclude if you reject H_0
Test statistic	A number summarizing how far the data deviates from H_0
Significance level (α)	The false-positive rate you're willing to tolerate
Critical region	Test-statistic values that lead to rejecting H_0
P-value	P(data this extreme or more | H_0 is true)
Type I error	Rejecting a true H_0 (false positive)
Type II error	Failing to reject a false H_0 (false negative)
Power	P(correctly rejecting a false H_0) = 1 − P(Type II error)

Why "fail to reject," never "accept" H_0: absence of evidence against H_0 is not evidence H_0 is true — it may simply mean the sample was too small to detect a real effect (Section 35's power). This asymmetry — you can disprove but never truly prove H_0 — is baked into the entire frequentist framework and is one of its most consequential, and most frequently misstated, properties.

---

32. Null and Alternative Hypotheses

Type	Form	Example
Two-sided (non-directional)	$H_1: \mu \neq \mu_0$	"The new checkout flow changes conversion" (either direction)
One-sided (directional)	$H_1: \mu > \mu_0$ or $\mu < \mu_0$	"The new checkout flow *increases* conversion"

```
# One-sided vs two-sided p-value differ by exactly a factor of 2 for a
symmetric test statistic
t_stat = 2.1
df = 20
p_two_sided = 2 * (1 - stats.t.cdf(abs(t_stat), df))
p_one_sided = 1 - stats.t.cdf(t_stat, df)
print(f"Two-sided p: {p_two_sided:.4f}   One-sided p:
{p_one_sided:.4f}")
```

Choosing before, not after, seeing the data is essential. Switching to a one-sided test *after* observing the data trends in a favorable direction (to halve the p-value and cross the 0.05 threshold) is a textbook form of p-hacking (Section 122) — the hypothesis type must be decided at the design stage, driven by the genuine business question, not by a post-hoc look at results.

---

33. P-Values — Correctly Understood

33.1 What a P-Value Actually Means

A p-value is the probability of observing data this extreme (or more), *assuming the null hypothesis is exactly true.* It is a statement about the data, conditional on H_0 — nothing more.

33.2 What a P-Value Does NOT Mean

Misconception	Why It's Wrong
"The probability H_0 is true"	The p-value conditions ON H_0 being true; it can't simultaneously be a statement about H_0's probability — that requires Bayesian machinery (Section 77)
"The probability the result is due to chance"	Closely related to the above misconception; still confuses $P(\text{data}\|H_0)$ with $P(H_0\|\text{data})$
"1 − p is the probability H_1 is true"	Same fundamental confusion of conditional direction
"p < 0.05 means the effect is real/large/important"	Says nothing about effect size (Section 45) — only about how surprising the data would be under H_0

33.3 Why p < 0.05 Doesn't Automatically Mean the Effect Is Large or Important

```
# A tiny, practically meaningless effect can be "significant" with
enough data
rng = np.random.default_rng(0)
n = 1_000_000
group_a = rng.normal(100.0, 15, n)
group_b = rng.normal(100.05, 15, n)   # a difference of 0.05 — utterly
trivial

t_stat, p_val = stats.ttest_ind(group_a, group_b)
print(f"t={t_stat:.2f}, p={p_val:.6f}")   # p < 0.05, despite a
meaningless effect size
```

A difference of 0.05 units, with a million observations per group, easily achieves statistical significance — and is very likely commercially irrelevant. This is precisely why Section 45 (effect size) and Section 121 (practical significance) must always accompany, never be replaced by, a p-value.

---

34. Type I and Type II Errors

	H_0 is actually True	H_0 is actually False
Reject H_0	**Type I error** (α, false positive)	Correct decision (power, $1-\beta$)
Fail to reject H_0	Correct decision ($1-\alpha$)	**Type II error** (β, false

	H_0 is actually True	H_0 is actually False
		negative)

34.1 Applied Examples

Domain	Type I Error (False Positive)	Type II Error (False Negative)
Fraud detection	Flagging a legitimate transaction as fraud (customer friction)	Missing actual fraud (financial loss)
Healthcare diagnosis	Diagnosing a healthy patient as sick (unnecessary treatment, anxiety)	Missing a genuine diagnosis (delayed treatment)
Security systems	False alarm (wasted response)	Missed intrusion (real breach)
Credit approval	Declining a genuinely creditworthy applicant (lost business)	Approving a defaulter (financial loss)

The central tradeoff: for a fixed sample size, reducing α (fewer false positives) mechanically increases β (more false negatives), and vice versa — you cannot minimize both simultaneously without more data or a more powerful test. Which error costs more, in a specific business context, should drive the choice of α (and often the decision threshold, exactly mirroring Module 1's precision/recall tradeoff, Section 28.7).

---

35. Statistical Power

35.1 Definition and Drivers

$$\text{Power} = 1 - \beta = P(\text{reject } H_0 \mid H_0 \text{ is false, true effect} = \delta)$$

Factor	Effect on Power
Larger effect size	Higher power (easier to detect a bigger difference)
Larger sample size	Higher power
Lower significance level (α)	Lower power (stricter threshold is harder to clear)
Lower variability	Higher power (less noise obscuring the signal)

35.2 Power Analysis in Python

```
from statsmodels.stats.power import TTestIndPower

analysis = TTestIndPower()
```

```
# How much power do I have, given n=50 per group, effect size d=0.5,
α=0.05?
power = analysis.power(effect_size=0.5, nobs1=50, alpha=0.05,
ratio=1.0)
print(f"Power: {power:.3f}")   # ≈ 0.70

# What sample size do I need for 80% power?
n_required = analysis.solve_power(effect_size=0.5, alpha=0.05,
power=0.8, ratio=1.0)
print(f"Required n per group: {n_required:.0f}")   # ≈ 64
```

35.3 Why Power Analysis Should Happen BEFORE Data Collection

Running an underpowered study risks a genuine effect going undetected (a costly Type II error) while running a wildly overpowered one wastes resources detecting effects too small to matter practically (revisiting Section 33.3's point). Power analysis, done at the **design** stage, is what determines a defensible sample size — this is the foundation for Section 71's A/B test sample-size planning, and its absence is one of the most common, and most preventable, weaknesses in real-world experiments.

36. One-Sample T-Test

36.1 Purpose, Assumptions, Formula

Purpose: test whether a sample mean differs from a hypothesized value μ_0. **Assumptions:** the sample is (approximately) normally distributed, or n is large enough for the CLT to apply; observations are independent.

```
t = (x̄ − μ₀) / (s/√n)          df = n − 1
```

```
call_times = np.array([4.2, 5.1, 3.8, 4.9, 5.5, 4.4, 4.7, 5.0, 4.1,
4.6])   # minutes
mu_0 = 5.0   # target/benchmark

# Manual
t_manual = (call_times.mean() - mu_0) /
(call_times.std(ddof=1)/np.sqrt(len(call_times)))
df = len(call_times) - 1
p_manual = 2 * (1 - stats.t.cdf(abs(t_manual), df))
print(f"Manual: t={t_manual:.3f}, p={p_manual:.4f}")

# SciPy
t_scipy, p_scipy = stats.ttest_1samp(call_times, mu_0)
print(f"SciPy:  t={t_scipy:.3f}, p={p_scipy:.4f}")

# Effect size (Cohen's d)
cohens_d = (call_times.mean() - mu_0) / call_times.std(ddof=1)
print(f"Cohen's d: {cohens_d:.3f}")
```

Interpretation: if $p < 0.05$, average call time is statistically distinguishable from the 5-minute benchmark; Cohen's d quantifies *how much* (here, a moderate effect), addressing Section 33.3's warning directly.

---

37. Two-Sample T-Test

37.1 Independent Samples, Equal vs Unequal Variance

```
control    = np.array([210, 220, 195, 230, 205, 215, 225, 200])
treatment  = np.array([250, 260, 245, 270, 255, 265, 240, 258])

# Student's t (assumes equal variances)
t_student, p_student = stats.ttest_ind(control, treatment,
equal_var=True)

# Welch's t (does NOT assume equal variances) — the modern default
t_welch, p_welch = stats.ttest_ind(control, treatment,
equal_var=False)

print(f"Student's t: t={t_student:.3f}, p={p_student:.5f}")
```

```
print(f"Welch's t:   t={t_welch:.3f}, p={p_welch:.5f}")

# Check the equal-variance assumption directly
levene_stat, levene_p = stats.levene(control, treatment)
print(f"Levene's test for equal variances: p={levene_p:.4f}")
```

37.2 Why Welch's Test Is Often Preferable

Student's t-test's equal-variance assumption is frequently violated in real business data (a "treatment" group with a new feature often has genuinely different variance, not just a different mean, from a "control" group). Welch's test adjusts the degrees of freedom (the Welch-Satterthwaite formula from Section 29.1) to remain valid under unequal variances, at essentially zero cost when variances *are* equal. **The professional default: use Welch's test unless you have a specific reason not to** — a recommendation increasingly standard across statistics teaching, reflecting decades of simulation evidence that it rarely underperforms and often meaningfully outperforms Student's version.

---

38. Paired T-Test

38.1 When Pairing Applies

Same units measured twice: before/after a training programme, before/after a marketing campaign, model performance before/after an optimization — anywhere natural pairing exists, exploiting it removes between-subject noise entirely.

```
# Model accuracy on 8 datasets, before and after a feature-engineering improvement
before = np.array([0.81, 0.76, 0.85, 0.79, 0.83, 0.77, 0.80, 0.82])
after  = np.array([0.85, 0.80, 0.87, 0.84, 0.86, 0.81, 0.83, 0.85])

t_paired, p_paired = stats.ttest_rel(after, before)
print(f"Paired t: t={t_paired:.3f}, p={p_paired:.5f}")

diffs = after - before
cohens_d_paired = diffs.mean() / diffs.std(ddof=1)
print(f"Cohen's d (paired): {cohens_d_paired:.3f}")
```

Why paired beats independent-samples here: the same 8 datasets are compared to themselves, removing dataset-to-dataset variability that would otherwise dilute the signal in an independent-samples test — a direct, practical demonstration of why Section 29.3's paired confidence interval exists.

---

39. Z-Test

39.1 When a Z-Test Applies

Requires a **known** population standard deviation, or (as an approximation) a sufficiently large sample where s reliably estimates σ.

```
# One-sample z-test: known population σ from historical data
sample_mean, sigma, n = 5.3, 1.2, 50
mu_0 = 5.0
z = (sample_mean - mu_0) / (sigma / np.sqrt(n))
p_val = 2 * (1 - stats.norm.cdf(abs(z)))
print(f"z={z:.3f}, p={p_val:.4f}")

# Two-sample z-test (large samples, known/well-estimated variances)
from statsmodels.stats.weightstats import ztest
z_stat, p_val2 = ztest(treatment, control)
print(f"Two-sample z: z={z_stat:.3f}, p={p_val2:.5f}")
```

In practice, t-tests dominate — σ is essentially never truly known, and modern software makes the t-distribution's slightly heavier computation free. Z-tests survive mainly for very large samples (where t≈z anyway) and for proportions (Section 28), where the normal approximation to the binomial is standard.

---

40. Chi-Square Tests

40.1 Goodness-of-Fit

Tests whether observed category frequencies match a hypothesized distribution.

```
observed = np.array([50, 30, 20])          # observed category counts
expected_props = np.array([0.5, 0.3, 0.2]) # hypothesized proportions
expected = expected_props * observed.sum()

chi2_stat, p_val = stats.chisquare(observed, expected)
print(f"χ²={chi2_stat:.3f}, p={p_val:.4f}")
```

40.2 Test of Independence (Two Categorical Variables)

```
# Contingency table: payment method x churn status
contingency = pd.DataFrame({
    "Churned":    [45, 30, 15],
    "Retained":   [255, 320, 180],
}, index=["Card", "Bank Transfer", "Cash"])

chi2, p, dof, expected = stats.chi2_contingency(contingency)
print(f"χ²={chi2:.3f}, df={dof}, p={p:.5f}")
print("Expected counts under independence:\n", pd.DataFrame(
    expected, index=contingency.index,
columns=contingency.columns).round(1))
```

Interpretation: a small p-value suggests payment method and churn status are **not independent** — but note this establishes *association*, not causation (Section 74). **Test of homogeneity** uses the identical chi-square machinery to compare whether several populations share the same categorical distribution — mathematically the same test, different sampling framing.

40.3 Assumption — Expected Cell Counts

The chi-square approximation is unreliable when expected cell counts are small (a common rule of thumb: expected count ≥ 5 in at least 80% of cells). Section 123's troubleshooting addresses the fix (Fisher's exact test for small tables) directly.

---

41. F-Test

41.1 Comparing Two Variances

```
F = s₁²/s₂²          (larger variance in the numerator by convention)
```

```
group1 = np.array([100, 105, 98, 102, 99, 101])
group2 = np.array([95, 115, 80, 120, 90, 110])

f_stat = group2.var(ddof=1) / group1.var(ddof=1)
df1, df2 = len(group2)-1, len(group1)-1
p_val = 2 * min(stats.f.cdf(f_stat, df1, df2), 1 - stats.f.cdf(f_stat,
df1, df2))
print(f"F={f_stat:.3f}, p={p_val:.4f}")
```

41.2 Relationship to ANOVA

ANOVA's F-statistic (Section 42) is exactly this same F-test idea, generalized: comparing **between-group variance** to **within-group variance** rather than comparing two individual sample variances directly. Understanding the two-variance F-test first makes ANOVA's F-statistic far more intuitive than treating it as a fundamentally new object.

---

42. ANOVA (One-Way)

42.1 The Variance Decomposition

```
Total variability = Between-group variability + Within-group
variability
        SST         =          SSB               +         SSW

F = (SSB / (k−1)) / (SSW / (N−k))
```

where k = number of groups, N = total sample size.

42.2 Worked Example — Marketing Channels

```
channel_a = np.array([12, 15, 14, 10, 13])
channel_b = np.array([18, 20, 17, 19, 21])
```

```
channel_c = np.array([9, 11, 8, 10, 12])

f_stat, p_val = stats.f_oneway(channel_a, channel_b, channel_c)
print(f"F={f_stat:.3f}, p={p_val:.5f}")

# Manual decomposition, to make the formula concrete
all_data = np.concatenate([channel_a, channel_b, channel_c])
grand_mean = all_data.mean()
groups = [channel_a, channel_b, channel_c]

ssb = sum(len(g) * (g.mean() - grand_mean)**2 for g in groups)
ssw = sum(((g - g.mean())**2).sum() for g in groups)
sst = ((all_data - grand_mean)**2).sum()
print(f"SST={sst:.2f}  SSB={ssb:.2f}  SSW={ssw:.2f}
(SSB+SSW={ssb+ssw:.2f})")

k, N = 3, len(all_data)
f_manual = (ssb/(k-1)) / (ssw/(N-k))
print(f"Manual F: {f_manual:.3f}")
```

Interpretation: the null hypothesis is that all group means are equal (μ_A=μ_B=μ_C). A significant F indicates **at least one** group differs — critically, ANOVA alone does not say *which* (Section 44's post-hoc testing exists precisely to answer that follow-up question).

---

43. Two-Way ANOVA

43.1 Main Effects and Interaction

```
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm

rng = np.random.default_rng(0)
n_per_cell = 20
data = pd.DataFrame({
    "channel": np.repeat(["Email", "Social"], n_per_cell*2),
    "segment": np.tile(np.repeat(["New", "Existing"], n_per_cell), 2),
})
# Simulate an interaction: Social works better for New, Email better
for Existing
base = 10
effect = np.where((data.channel=="Social")&(data.segment=="New"), 8,
          np.where((data.channel=="Email")&(data.segment=="Existing"),
6, 0))
data["response"] = base + effect + rng.normal(0, 2, len(data))

model = smf.ols("response ~ C(channel) * C(segment)", data=data).fit()
print(anova_lm(model, typ=2))
```

Term	Meaning
Main effect (channel)	Does channel matter *on average*, ignoring segment?
Main effect (segment)	Does segment matter *on average*, ignoring channel?
Interaction (channel×segment)	Does channel's effect *depend on* segment?

Business reading: a significant interaction (as constructed here) means "the best channel depends on which segment you're targeting" — a materially more actionable finding than either main effect alone, and a very common real pattern in marketing and product analytics.

---

44. Post-Hoc Testing

44.1 Why ANOVA Alone Isn't Enough

A significant ANOVA F only tells you *some* group differs — it doesn't identify which pair(s). Running many pairwise t-tests instead inflates the **family-wise error rate** (Section 46) — with k=5 groups, there are 10 pairwise comparisons, and even at α=0.05 each, the chance of at least one false positive across all 10 climbs well above 5%.

44.2 Tukey's HSD

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd

all_scores = np.concatenate([channel_a, channel_b, channel_c])
labels = np.repeat(["A", "B", "C"], 5)

tukey = pairwise_tukeyhsd(all_scores, labels, alpha=0.05)
print(tukey)
```

Tukey's HSD controls the family-wise error rate **across all pairwise comparisons simultaneously**, giving each individual comparison a stricter effective threshold than an uncorrected 0.05 — the correct way to follow up a significant ANOVA, directly connecting to Section 46's broader multiple-testing framework.

---

45. Effect Size

45.1 The Catalogue

Measure	Formula	Used With
Cohen's d	$(\bar{x}_1 - \bar{x}_2)/s_{pooled}$	t-tests
Eta squared (η^2)	SSB/SST	ANOVA — proportion of variance explained by group membership
Partial eta squared	SSB/(SSB+SSW)	ANOVA with multiple factors

Measure	Formula	Used With
Odds ratio	(a/b)/(c/d) from a 2×2 table	Logistic regression, chi-square 2×2
Risk ratio	P(event|exposed)/P(event|unexposed)	Epidemiology-style comparisons

```
# Cohen's d, with conventional interpretation bands
def cohens_d(g1, g2):
    n1, n2 = len(g1), len(g2)
    pooled_std = np.sqrt(((n1-1)*g1.var(ddof=1) + (n2-
1)*g2.var(ddof=1)) / (n1+n2-2))
    return (g1.mean() - g2.mean()) / pooled_std

d = cohens_d(treatment, control)
print(f"Cohen's d: {d:.3f}")
# |d|<0.2 negligible, 0.2-0.5 small, 0.5-0.8 medium, >0.8 large
(Cohen's conventional bands)

# Eta squared from the ANOVA above
eta_sq = ssb / sst
print(f"η² = {eta_sq:.3f}  ({eta_sq:.1%} of variance explained by
channel)")
```

45.2 Why Effect Size Must Accompany P-Values

A p-value answers "is there *any* detectable effect, given this sample size?" Effect size answers "**how big** is it, in units that matter?" Section 33.3 showed a trivial effect can be statistically significant with enough data; conversely, a substantial effect can fail to reach significance with too little data (Section 35's power). **Reporting both together is the professional standard** — neither number alone tells the complete story.

---

46. Multiple Testing

46.1 The Problem, Quantified

Running m independent tests at α=0.05 each, the probability of **at least one** false positive is:

```
P(at least one false positive) = 1 − (1−α)^m
```

```
for m in [1, 5, 10, 20, 50]:
    p_at_least_one = 1 - (1 - 0.05)**m
    print(f"m={m:3d} tests: P(≥1 false positive) =
{p_at_least_one:.1%}")
# m=  1: 5.0%
# m=  5: 22.6%
# m= 10: 40.1%
# m= 20: 64.2%
# m= 50: 92.3%
```

With 20 independent tests, there's a **64% chance** at least one shows "significance" purely by chance — the mathematical basis of the common warning about testing many subgroups or many features and reporting only the "significant" ones.

46.2 Corrections

Method	Approach	Conservatism
Bonferroni	Use α/m per test	Very conservative, simple
Holm	Sequentially adjusted, less conservative than Bonferroni	Moderate
Benjamini-Hochberg (FDR)	Controls the *expected proportion* of false discoveries among rejections, not the family-wise rate	Less conservative, standard for many simultaneous tests (e.g. genomics, large-scale feature screening)

```
from statsmodels.stats.multitest import multipletests

p_values = np.array([0.001, 0.02, 0.03, 0.04, 0.045, 0.06, 0.15,
0.30])

reject_bonf, p_bonf, _, _ = multipletests(p_values, alpha=0.05,
method='bonferroni')
reject_holm, p_holm, _, _ = multipletests(p_values, alpha=0.05,
method='holm')
reject_fdr, p_fdr, _, _ = multipletests(p_values, alpha=0.05,
method='fdr_bh')

comparison = pd.DataFrame({"raw_p": p_values, "bonferroni":
p_bonf.round(4),
                           "holm": p_holm.round(4), "fdr_bh":
p_fdr.round(4),
                           "reject_bonf": reject_bonf, "reject_fdr":
reject_fdr})
print(comparison)
```

When multiple testing matters most in ML/analytics: screening hundreds of candidate features for correlation with a target (Section 93), running many A/B test metrics simultaneously (Section 71), testing many customer segments for a campaign effect, or monitoring many features for drift (Section 110) — anywhere "test many things, report the significant ones" occurs, correction is mandatory for the conclusions to be trustworthy.

47. Nonparametric Statistics

47.1 Why They Exist

Parametric tests (t-test, ANOVA, Pearson correlation) assume a specific distributional form (usually normality). **Nonparametric tests** make far weaker assumptions — often only that data is ordinal or continuous — trading some statistical power (when parametric assumptions genuinely hold) for robustness (when they don't).

47.2 The Catalogue and Parametric Analogues

Nonparametric Test	Parametric Analogue	Use Case
Mann-Whitney U	Independent-samples t-test	Compare two independent groups, non-normal/ordinal data
Wilcoxon signed-rank	Paired t-test	Compare paired observations, non-normal data
Kruskal-Wallis	One-way ANOVA	Compare 3+ independent groups, non-normal data
Spearman correlation	Pearson correlation	Monotonic (not necessarily linear) association
Kendall's tau	Pearson correlation	Association via concordant/discordant pairs, robust to outliers

```
group_a = np.array([12, 15, 14, 95, 13])   # note the outlier
group_b = np.array([18, 20, 17, 19, 21])

# Parametric (sensitive to the outlier)
t_stat, p_t = stats.ttest_ind(group_a, group_b)

# Nonparametric (robust to the outlier — uses ranks, not raw values)
u_stat, p_u = stats.mannwhitneyu(group_a, group_b)

print(f"t-test:        p={p_t:.4f}")
print(f"Mann-Whitney:  p={p_u:.4f}")
```

The outlier distorts the t-test's conclusion far more than Mann-Whitney's, because Mann-Whitney operates on **ranks** rather than raw values — one extreme value can only ever contribute one extreme rank, capping its influence, unlike in a mean-based test where an extreme value can dominate arbitrarily.

```
# Wilcoxon signed-rank (paired, nonparametric)
w_stat, p_w = stats.wilcoxon(after, before)   # reusing Section 38's
paired data
print(f"Wilcoxon: p={p_w:.4f}")

# Kruskal-Wallis (3+ groups, nonparametric ANOVA analogue)
```

```
h_stat, p_h = stats.kruskal(channel_a, channel_b, channel_c)
print(f"Kruskal-Wallis: H={h_stat:.3f}, p={p_h:.5f}")
```

47.3 When to Choose Nonparametric

Small sample with visibly non-normal shape; ordinal (not interval/ratio) data; known outliers you don't want to (or can't legitimately) remove; or as a robustness check alongside a parametric result — if both agree, confidence in the conclusion increases; if they disagree, the discrepancy itself is diagnostic (often indicating outlier sensitivity or a skewed distribution driving the parametric result).

---

48. Correlation Analysis

48.1 The Three Correlation Coefficients

Coefficient	Measures	Robust to Outliers?	Captures Non-linear Monotonic?
Pearson (r)	Linear association	No	No
Spearman (ρ)	Monotonic association (via ranks)	Yes	Yes
Kendall's τ	Concordance of pairs (via ranks)	Yes	Yes

```
x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
y_nonlinear = x ** 3   # perfectly monotonic, strongly non-linear

r_pearson, _ = stats.pearsonr(x, y_nonlinear)
rho_spearman, _ = stats.spearmanr(x, y_nonlinear)
tau_kendall, _ = stats.kendalltau(x, y_nonlinear)

print(f"Pearson r:  {r_pearson:.4f}")    # 0.9155 — understates the relationship
print(f"Spearman ρ: {rho_spearman:.4f}") # 1.0000 — perfectly captures the monotonic truth
print(f"Kendall τ:  {tau_kendall:.4f}")  # 1.0000
```

Pearson correctly identifies a strong relationship but **understates** it (0.92, not 1.0) precisely because the relationship is monotonic but curved, not straight — this directly extends Module 5's Section 54.5 observation about Pearson's blindness to non-linearity into the fully general statistical toolkit.

48.2 Correlation ≠ Causation — Restated With Teeth

A high correlation between ice-cream sales and drowning deaths reflects a **common cause** (hot weather), not a causal link between the two. Section 74 develops this into a full framework; for now, the operating discipline is: **correlation is always a legitimate**

description of the observed data; it is never, by itself, license to claim one variable causes the other.

---

49. Covariance and Correlation Matrix

```
df = pd.DataFrame({
    "age": rng.integers(20, 70, 200),
    "income": rng.normal(500_000, 150_000, 200),
    "tenure": rng.integers(1, 60, 200),
})
df["spend"] = 0.02*df["income"] + 500*df["tenure"] + rng.normal(0,
5000, 200)

cov_matrix = df.cov()
corr_matrix = df.corr()

sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", center=0, vmin=-
1, vmax=1)
plt.title("Correlation Matrix")
```

Reading a correlation heatmap: the diagonal is always 1.0 (a variable perfectly correlates with itself); look for off-diagonal cells with |r|>0.7-0.8 as multicollinearity candidates (Section 56); look for cells with |r| near 0 to identify features carrying independent information.

Direct connections: - **PCA** (Section 96) eigendecomposes exactly this covariance/correlation matrix. - **Regression** (Sections 51-52) uses it implicitly through X^TX. - **Multivariate analysis** (Section 95) treats it as the fundamental object describing joint variable behavior. - **Feature selection** (Section 93) uses pairwise correlation with the target (and among features, to catch redundancy) as a first-pass screen.

---

50. Regression as Statistical Modeling

50.1 The General Linear Model

$$Y = \beta_0 + \beta_1X_1 + \beta_2X_2 + \ldots + \beta_pX_p + \varepsilon$$

Symbol	Meaning
$\mathbf{Y}$	The response/dependent variable
$\mathbf{X_1 \ldots X_p}$	Predictors/independent variables
$\boldsymbol{\beta_0}$	Intercept
$\boldsymbol{\beta_1 \ldots \beta_p}$	Coefficients (the effect of each predictor, holding others constant)
$\boldsymbol{\varepsilon}$	The error term — everything Y depends on that isn't in the model, assumed random noise

50.2 Statistical vs Predictive Regression — The Emphasis Shift

Module 5's Section 74 treated linear regression as a *prediction machine*, optimized via least squares/gradient descent, evaluated by held-out error. **Statistical regression** asks additionally: is each β_j *significantly different from zero* (a hypothesis test on every coefficient), how *confident* are we in each coefficient's value (a confidence interval per coefficient), and do the model's **assumptions** (Section 53) hold well enough to trust those inferences at all? The underlying mathematics (least squares) is identical — the *questions asked of the fitted model* differ, and this is exactly why `statsmodels` (which reports coefficient p-values and confidence intervals by default) is the standard tool for statistical regression, while `scikit-learn` (which reports only predictions) is the standard tool for predictive regression.

---

51. Simple Linear Regression

51.1 Least Squares, Derived (Recap From Module 5, Statistical Framing Added)

$\hat{y} = \hat{\beta_0} + \hat{\beta_1}x$
$\hat{\beta_1} = \Sigma(x_i-\bar{x})(y_i-\bar{y}) \;/\; \Sigma(x_i-\bar{x})^2 \;=\; Cov(X,Y)/Var(X)$
$\hat{\beta_0} = \bar{y} - \hat{\beta_1}\bar{x}$

```
x = np.array([5, 10, 15, 20, 25, 30, 35, 40])
y = np.array([12, 25, 33, 48, 55, 61, 75, 82])

n = len(x)
beta1 = np.sum((x-x.mean())*(y-y.mean())) / np.sum((x-x.mean())**2)
beta0 = y.mean() - beta1*x.mean()
y_hat = beta0 + beta1*x
residuals = y - y_hat

ss_res = (residuals**2).sum()
ss_tot = ((y-y.mean())**2).sum()
r_squared = 1 - ss_res/ss_tot
print(f"β0={beta0:.3f}, β1={beta1:.3f}, R²={r_squared:.4f}")

# statsmodels — for the STATISTICAL view (p-values, CIs)
X_sm = sm.add_constant(x)
model = sm.OLS(y, X_sm).fit()
print(model.summary())
```

51.2 Reading the statsmodels Summary Table

Output	Meaning
coef	β_j, the estimated effect
std err	$SE(\beta_j)$ — precision of the estimate
t	β_j / $SE(\beta_j)$ — tests H_0: $\beta_j=0$
P>|t|	p-value for that coefficient test
[0.025, 0.975]	95% confidence interval for β_j

Output	Meaning
R-squared	Fraction of Y's variance explained
Adj. R-squared	Penalized for the number of predictors (Section 59)
F-statistic	Tests whether the *whole model* explains significantly more than an intercept-only model

52. Multiple Linear Regression

```
rng = np.random.default_rng(1)
n = 300
X_df = pd.DataFrame({
    "sqft": rng.normal(1500, 400, n),
    "bedrooms": rng.integers(1, 6, n),
    "age": rng.integers(0, 50, n),
})
y_price = 50_000 + 120*X_df.sqft + 15_000*X_df.bedrooms - 800*X_df.age
+ rng.normal(0, 20_000, n)

X_sm = sm.add_constant(X_df)
model = sm.OLS(y_price, X_sm).fit()
print(model.params.round(2))
print(model.pvalues.round(4))
print(model.conf_int().round(2))
```

Coefficient interpretation with multiple predictors: each β_j is the expected change in Y for a one-unit increase in X_j, **holding all other predictors constant** — a materially more careful statement than simple regression's single-predictor interpretation, and the reason multiple regression is the standard tool whenever confounding predictors are present (directly connecting to Section 76).

Matrix formulation, restated in the statistical frame: $\beta = (X^TX)^{-1}X^Ty$ (Module 5, Section 74.2) — identical mathematics; `statsmodels.OLS` and `sklearn.linear_model.LinearRegression` compute the *same point estimate*, differing in that statsmodels additionally reports the full inferential apparatus (standard errors, p-values, CIs) this section is built around.

---

53. Assumptions of Linear Regression

Assumption	What It Means	Why It Matters	If Violated
Linearity	The true relationship between X and Y is linear in the parameters	Otherwise the model is structurally wrong, regardless of sample size	Use polynomial/spline terms, or a non-linear model
Independence	Residuals are	Violated by time	Use time-series or

Assumption	What It Means	Why It Matters	If Violated
	uncorrelated with each other	series/panel data; inflates apparent precision	clustered-error methods
Homoscedasticity	Constant residual variance across all fitted values	Violated → standard errors (hence p-values, CIs) are wrong	Robust standard errors, transform Y, weighted least squares
Normality of residuals	Residuals are approximately normal	Mainly matters for small-sample inference; large-sample CLT rescues coefficient inference regardless	Bootstrap inference, nonparametric alternatives
No severe multicollinearity	Predictors aren't near-linearly-dependent	Coefficients become unstable/uninterpretable (Module 5, Section 47.3)	Drop/combine features, Ridge regression

The professional habit: check every one of these *after* fitting, via residual diagnostics (Section 54), before trusting any coefficient's p-value or confidence interval. A model that predicts well can still have invalid *inferential* statistics if these assumptions are badly violated — prediction and inference have different assumption requirements, a distinction worth holding onto throughout this module.

---

54. Residual Analysis

```
fitted = model.fittedvalues
resid = model.resid

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes[0,0].scatter(fitted, resid, alpha=0.5)
axes[0,0].axhline(0, color='red', linestyle='--')
axes[0,0].set_xlabel("Fitted values");
axes[0,0].set_ylabel("Residuals")
axes[0,0].set_title("Residuals vs Fitted (linearity &
homoscedasticity)")

stats.probplot(resid, dist="norm", plot=axes[0,1])
axes[0,1].set_title("Q-Q Plot (normality)")

axes[1,0].hist(resid, bins=30)
axes[1,0].set_title("Residual distribution")

influence = model.get_influence()
```

```
axes[1,1].scatter(range(len(resid)), influence.hat_matrix_diag)
axes[1,1].set_title("Leverage")
```

Plot	What a Problem Looks Like
Residuals vs fitted	A curved pattern → non-linearity; a funnel shape → heteroscedasticity (Section 55)
Q-Q plot	Points departing from the diagonal line → non-normal residuals (Section 82 develops this fully)
Histogram of residuals	Visibly skewed or multimodal → a model misspecification signal
Leverage/influence	A few points with outsized influence → potential influential observations (Section 57)

Residual = Observed − Predicted, and by design, OLS residuals always sum to (very nearly) zero and are uncorrelated with the fitted values — any *systematic* pattern remaining in a residual plot indicates the model is missing something structural, not just noisy.

---

55. Homoscedasticity

55.1 Detection

```
# Breusch-Pagan test
from statsmodels.stats.diagnostic import het_breuschpagan
bp_stat, bp_pval, _, _ = het_breuschpagan(resid, X_sm)
print(f"Breusch-Pagan: stat={bp_stat:.3f}, p={bp_pval:.4f}")
# p < 0.05 suggests heteroscedasticity is present
```

55.2 Consequences and Remedies

Consequence: OLS coefficients remain unbiased under heteroscedasticity, but the standard errors (and therefore every p-value and confidence interval) become **wrong** — usually understated, producing false confidence.

```
# Robust (heteroscedasticity-consistent) standard errors — the
standard fix
model_robust = sm.OLS(y_price, X_sm).fit(cov_type='HC3')
print(model_robust.summary().tables[1])   # compare std errors to the
non-robust version
```

Robust standard errors (HC0-HC3 variants) correct the standard-error formula to remain valid under heteroscedasticity without changing the coefficient estimates themselves — the standard, low-cost professional remedy, and a very common line to add to any regression run on real (rarely perfectly homoscedastic) business data.

---

56. Multicollinearity

Recapping and extending Module 5's Section 47.

56.1 Correlation vs Multicollinearity

Pairwise correlation (Section 48) examines two variables at a time; **multicollinearity** more generally means a predictor is (nearly) a *linear combination of several others combined*, which pairwise correlation alone can miss.

56.2 Variance Inflation Factor (VIF)

$VIF_j = 1 / (1 - R_j^2)$

where R_j^2 comes from regressing predictor X_j on all *other* predictors.

```
from statsmodels.stats.outliers_influence import
variance_inflation_factor

X_vif = X_df.copy()
X_vif["sqft_thousands"] = X_vif["sqft"] / 1000   # deliberately near-
redundant with sqft

vif_data = pd.DataFrame({
    "feature": X_vif.columns,
    "VIF": [variance_inflation_factor(X_vif.values, i) for i in
range(X_vif.shape[1])]
})
print(vif_data)
# sqft and sqft_thousands will show VERY high VIF (>>10) — flagging
the redundancy
```

Interpretation: VIF=1 means no correlation with other predictors; VIF>5-10 (conventions vary) signals problematic multicollinearity. **Remedies:** drop or combine redundant features, use Ridge regression (Module 5, Section 72, which numerically handles exactly this problem via the λI term), or apply PCA (Section 96) to create orthogonal composite features.

---

57. Outliers and Influential Observations

Concept	Meaning
Outlier	An observation with an unusual **Y** value given its X (large residual)
Leverage	An observation with unusual **X** values (far from the predictor mean) — potentially influential regardless of its residual
Influence	How much a specific observation, if removed, would **change the fitted coefficients** — combines outlier-ness and leverage

Concept	Meaning
Cook's distance	A standard combined measure of influence

```
influence = model.get_influence()
cooks_d = influence.cooks_distance[0]

threshold = 4 / len(y_price)   # a common rule-of-thumb cutoff
influential_points = np.where(cooks_d > threshold)[0]
print(f"Flagged {len(influential_points)} influential observations (of
{len(y_price)})")

plt.stem(cooks_d)
plt.axhline(threshold, color='red', linestyle='--')
plt.title("Cook's Distance")
```

Why outliers are not automatically errors: a genuinely valid, extreme data point (a real ₦50m transaction in mostly-₦50,000 data) carries real information and may be exactly the case a fraud model most needs to learn from — removing it can quietly discard the signal the entire analysis exists to find. **The professional discipline:** investigate every flagged influential point's origin before deciding to exclude, down-weight, or (more often, correctly) keep it, documenting the reasoning either way.

58. R-Squared

58.1 Definition and Interpretation

$R^2 = 1 - SS_{res}/SS_{tot} = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$

The fraction of Y's variance explained by the model, versus predicting the mean for everyone (Module 5, Section 74.5's exact definition, revisited here with statistical caveats).

58.2 Why High R^2 Does Not Guarantee a Good Model

```
# Anscombe-style: high R² with a badly-fitting model (non-linear true
relationship)
x_curve = np.linspace(1, 10, 50)
y_curve = x_curve**2 + rng.normal(0, 5, 50)

X_lin = sm.add_constant(x_curve)
model_lin = sm.OLS(y_curve, X_lin).fit()
print(f"R² of a LINEAR fit to a QUADRATIC relationship:
{model_lin.rsquared:.3f}")
# Often still surprisingly high (e.g. 0.85+), despite the linear model
being structurally wrong
```

A high R^2 can coexist with badly violated linearity, heteroscedasticity, or a handful of influential points driving the fit — **R^2 alone never substitutes for residual diagnostics** (Section 54). Conversely, a **low R^2 can still be a useful, honest model** — in domains with inherently high irreducible noise (predicting individual human behavior, financial returns), a modest R^2 of 0.1-0.2 may represent genuine, actionable signal, and demanding a higher R^2 may simply be demanding the impossible (Module 5, Section 73.2's irreducible-error argument, applied here).

---

59. Adjusted R-Squared

59.1 Why Plain R^2 Always Increases With More Predictors

$\text{Adjusted } R^2 = 1 - [(1-R^2)(n-1)/(n-p-1)]$

```
rng2 = np.random.default_rng(5)
n = 100
y_target = rng2.normal(0, 1, n)

r2_values, adj_r2_values = [], []
X_accum = pd.DataFrame({"const": np.ones(n)})
for p in range(1, 30):
    X_accum[f"noise_{p}"] = rng2.normal(0, 1, n)   # pure noise,
unrelated to y
    model_p = sm.OLS(y_target, X_accum).fit()
    r2_values.append(model_p.rsquared)
    adj_r2_values.append(model_p.rsquared_adj)

plt.plot(r2_values, label="R²")
```

```
plt.plot(adj_r2_values, label="Adjusted R²")
plt.legend(); plt.xlabel("Number of (noise) predictors added")
```

R^2 mechanically rises even when adding pure noise predictors — because OLS can always find *some* spurious fit to extra dimensions. Adjusted R^2 penalizes model complexity (via the p in the formula), and can genuinely **decrease** when an added predictor doesn't earn its keep — the correct metric for comparing models with different numbers of predictors, and a direct statistical cousin of Module 1's overfitting caution (Section 20).

---

60. AIC and BIC

60.1 Formulas and Interpretation

```
AIC = 2p − 2ln(L̂)
BIC = p·ln(n) − 2ln(L̂)
```

where p = number of parameters, $\hat{L}$ = the maximized likelihood.

```
models_to_compare = {}
X_simple = sm.add_constant(X_df[["sqft"]])
X_full = sm.add_constant(X_df)

models_to_compare["sqft only"] = sm.OLS(y_price, X_simple).fit()
models_to_compare["all predictors"] = sm.OLS(y_price, X_full).fit()

for name, m in models_to_compare.items():
    print(f"{name:16s}: AIC={m.aic:.1f}  BIC={m.bic:.1f}
R²={m.rsquared:.4f}")
```

60.2 The Tradeoff and Its Limits

Both penalize model complexity, but **BIC penalizes it more heavily** (via ln(n), which grows with sample size), tending to favor simpler models than AIC on large datasets. **Lower AIC/BIC indicates a preferred tradeoff between fit and complexity under that specific criterion** — it does **not** guarantee better out-of-sample predictive performance, which is what cross-validation (Section 69) directly measures instead. AIC/BIC are most useful for comparing **non-nested** models fit to the *same* data and the *same* target — comparing them across different datasets, or against models optimized for a different objective, is not meaningful.

---

61. Logistic Regression as a Statistical Model

Building directly on Module 5's Section 75, adding the statistical inference layer.

```
n = 1000
rng3 = np.random.default_rng(2)
tenure = rng3.integers(1, 60, n)
support_calls = rng3.poisson(1.5, n)
true_logit = -1.5 - 0.05*tenure + 0.6*support_calls
```

```
churn = (rng3.random(n) < 1/(1+np.exp(-true_logit))).astype(int)

X_logit = sm.add_constant(pd.DataFrame({"tenure": tenure,
"support_calls": support_calls}))
logit_model = sm.Logit(churn, X_logit).fit()
print(logit_model.summary())

# Odds ratios with confidence intervals
odds_ratios = np.exp(logit_model.params)
odds_ratio_ci = np.exp(logit_model.conf_int())
print(pd.DataFrame({"odds_ratio": odds_ratios, "ci_low":
odds_ratio_ci[0], "ci_high": odds_ratio_ci[1]}))
```

What `statsmodels.Logit` adds over the from-scratch Module 5 implementation: per-coefficient p-values (is *this specific* predictor's effect distinguishable from zero?), confidence intervals on both the coefficients and the derived odds ratios, and standard model-fit diagnostics — exactly the statistical apparatus that makes logistic regression usable for regulated, explainable decision-making (credit, healthcare) rather than only for raw prediction.

---

62. Odds and Odds Ratios

62.1 Definitions

```
Odds = p / (1−p)
Odds Ratio (OR) = Odds(exposed) / Odds(unexposed)

# A 2x2 table: smoking status vs disease
#              Disease   No Disease
# Smoker          90         210
# Non-smoker      30         470

odds_smoker = 90/210
odds_nonsmoker = 30/470
OR = odds_smoker / odds_nonsmoker
print(f"Odds (smoker): {odds_smoker:.3f}")
print(f"Odds (non-smoker): {odds_nonsmoker:.3f}")
print(f"Odds Ratio: {OR:.3f}")
# Smokers have ~6.7x the odds of disease compared to non-smokers
```

62.2 Interpreting Odds Ratios Carefully

OR value	Meaning
OR = 1	No association
OR > 1	Higher odds in the exposed/predictor-present group
OR < 1	Lower odds

A common and costly misinterpretation: an odds ratio of 6.7 does **not** mean smokers are "6.7 times more likely" to have the disease in the everyday probability sense — that colloquial

phrasing describes a *risk ratio*, a related but numerically different quantity, and the two diverge substantially when the outcome isn't rare. Odds ratios and risk ratios only converge closely when the outcome probability is small (the "rare disease assumption") — worth stating explicitly whenever reporting an OR to a non-technical audience, to avoid this exact confusion.

---

63. Generalized Linear Models

63.1 The GLM Framework

```
Random component:      Y ~ some distribution from the exponential
family (Section 143)
Systematic component:  η = Xβ           (the linear predictor)
Link function:          g(E[Y]) = η       (connects the mean of Y to
the linear predictor)
```

Model	Distribution	Link Function
Linear regression	Gaussian	Identity: g(μ)=μ
Logistic regression	Bernoulli	Logit: g(μ)=ln(μ/(1−μ))
Poisson regression	Poisson	Log: g(μ)=ln(μ)

```
# GLM interface unifies all three
glm_gaussian = sm.GLM(y_price, sm.add_constant(X_df),
family=sm.families.Gaussian()).fit()
glm_binomial = sm.GLM(churn, X_logit,
family=sm.families.Binomial()).fit()
# print(glm_gaussian.summary()); print(glm_binomial.summary())
```

Why the GLM abstraction matters: it reveals that linear and logistic regression aren't unrelated techniques you separately memorize — they are the *same estimation framework* with a different assumed noise distribution and link function, directly echoing Module 5's Section 70.4 observation that MSE and cross-entropy are both negative log-likelihoods differing only in the assumed distribution.

---

64. Poisson Regression

64.1 For Count Outcomes

```
rng4 = np.random.default_rng(3)
n = 500
marketing_spend = rng4.uniform(0, 100, n)
log_rate = 0.5 + 0.03*marketing_spend
website_visits = rng4.poisson(np.exp(log_rate))

X_poisson = sm.add_constant(marketing_spend)
poisson_model = sm.GLM(website_visits, X_poisson,
family=sm.families.Poisson()).fit()
print(poisson_model.summary())
```

```
# Interpreting the coefficient: exp(β) = multiplicative effect on the
RATE
print(f"Each unit of spend multiplies visit rate by:
{np.exp(poisson_model.params[1]):.4f}")
```

Applications: insurance claims counts, safety incidents, website visits, fraud events per period — any non-negative integer count. **Exposure/offset:** when counts accrue over varying observation windows (claims per policy-year, incidents per machine-hour), an **offset** term (log of the exposure) is included so the model correctly estimates a *rate*, not a raw count biased by exposure differences.

---

65. Negative Binomial Regression

65.1 When Poisson Fails — Overdispersion

Poisson assumes **mean = variance** (Module 5, Section 62.4). Real count data is frequently **overdispersed** (variance > mean) — customers vary in underlying propensity beyond what Poisson allows for.

```
print(f"Mean visits: {website_visits.mean():.2f}, Variance:
{website_visits.var():.2f}")
# If variance >> mean, Poisson's equidispersion assumption is violated

nb_model = sm.GLM(website_visits, X_poisson,
                  family=sm.families.NegativeBinomial()).fit()
print(nb_model.summary())
```

The Negative Binomial distribution adds a dispersion parameter, decoupling mean and variance — the standard remedy whenever Poisson's residual deviance is much larger than its degrees of freedom (a standard overdispersion diagnostic), which is common enough in real business count data that many practitioners default to Negative Binomial and only simplify to Poisson if variance and mean turn out to be genuinely close.

---

66. Statistical Assumptions — A Unified Framework

A reference table, applying the same checklist discipline across every method in this module.

Method	Key Assumptions	How to Check	If Violated
t-test	Approx. normal, independent obs.	Q-Q plot, Shapiro-Wilk (Section 83)	Mann-Whitney/ Wilcoxon (Section 47)
ANOVA	Normal residuals, equal variances, independence	Q-Q plot, Levene's test	Welch's ANOVA, Kruskal-Wallis
Pearson	Linear relationship,	Scatter plot	Spearman/Kendall

Method	Key Assumptions	How to Check	If Violated
correlation	no severe outliers		
Linear regression	Linearity, independence, homoscedasticity, normal residuals, low multicollinearity	Residual plots, Breusch-Pagan, VIF	Robust SEs, transformations, Ridge
Logistic regression	Independence, linearity in log-odds, low multicollinearity	Residual/link diagnostics, VIF	Add polynomial/interaction terms
Chi-square	Expected cell counts ≥5	Inspect expected-count table	Fisher's exact test
Poisson regression	Mean = variance (equidispersion)	Compare mean/variance, deviance test	Negative Binomial

The unifying discipline, stated once for the whole module: every statistical method makes assumptions that, when true, justify its specific mathematical guarantees (unbiasedness, correct standard errors, valid p-values). **Checking assumptions is not academic formality — it is the difference between a trustworthy and an actively misleading analysis.** This table is the reference point Section 123's troubleshooting section returns to repeatedly.

67. Bootstrap

67.1 The Idea

Resample the data itself (with replacement) many times, computing the statistic of interest on each resample, to build an empirical approximation of its sampling distribution — without assuming any particular theoretical distribution.

67.2 From-Scratch Implementation

```
def bootstrap_ci(data, statistic_fn, n_boot=10000, ci=0.95, seed=0):
    rng = np.random.default_rng(seed)
    n = len(data)
    boot_stats = np.empty(n_boot)
    for i in range(n_boot):
        resample = rng.choice(data, size=n, replace=True)
        boot_stats[i] = statistic_fn(resample)
    alpha = 1 - ci
    lo, hi = np.percentile(boot_stats, [100*alpha/2, 100*(1-alpha/2)])
    return statistic_fn(data), lo, hi, boot_stats

data = np.array([320, 340, 310, 330, 5200, 315, 325, 345, 300, 360])
# has an outlier

point_est, lo, hi, boot_dist = bootstrap_ci(data, np.median,
n_boot=10000)
print(f"Median: {point_est}, Bootstrap 95% CI: ({lo:.1f}, {hi:.1f})")

point_est_mean, lo_m, hi_m, _ = bootstrap_ci(data, np.mean,
n_boot=10000)
print(f"Mean: {point_est_mean:.1f}, Bootstrap 95% CI: ({lo_m:.1f},
{hi_m:.1f})")
```

Why bootstrap is valuable here specifically: the classical t-based CI (Section 27) assumes approximate normality, which this outlier-contaminated, skewed sample violates; and there is **no simple closed-form formula for the median's standard error at all**. Bootstrap sidesteps both problems entirely, working for essentially any statistic (mean, median, correlation, a model's R^2, a custom business metric) with the identical procedure.

67.3 Bias-Corrected Bootstrap (Conceptual)

```
bias = boot_dist.mean() - point_est_mean
print(f"Estimated bootstrap bias: {bias:.2f}")
bias_corrected_estimate = point_est_mean - bias
```

A simple bias correction subtracts the estimated bootstrap bias from the original point estimate; more refined variants (BCa — bias-corrected and accelerated) adjust the percentile cutoffs themselves rather than just shifting the point estimate, giving more accurate coverage in skewed cases — available via `scipy.stats.bootstrap(method='BCa')` in practice.

```
from scipy.stats import bootstrap
res = bootstrap((data,), np.median, n_resamples=10000, method='BCa',
```

```
random_state=0)
print(f"BCa 95% CI: {res.confidence_interval}")
```

---

68. Jackknife

68.1 Leave-One-Out Resampling

```
def jackknife(data, statistic_fn):
    n = len(data)
    jack_stats = np.array([statistic_fn(np.delete(data, i)) for i in
range(n)])
    theta_hat = statistic_fn(data)
    jack_mean = jack_stats.mean()

    bias_jack = (n-1) * (jack_mean - theta_hat)
    se_jack = np.sqrt((n-1)/n * np.sum((jack_stats - jack_mean)**2))
    return theta_hat, bias_jack, se_jack

theta, bias, se = jackknife(data, np.mean)
print(f"Estimate: {theta:.2f}, Jackknife bias: {bias:.2f}, Jackknife
SE: {se:.2f}")
```

68.2 Jackknife vs Bootstrap

	Jackknife	Bootstrap
Resamples	Exactly n (leave-one-out)	Arbitrary number, with replacement
Computational cost	O(n) refits	O(n_boot) refits, typically thousands
Smoothness	Can behave poorly for non-smooth statistics (e.g. median)	Generally more robust
Primary modern use	Historical/simple bias-variance estimation	The default resampling tool in modern practice

In modern practice, bootstrap has largely superseded jackknife for most applications due to its generality and superior behavior on non-smooth statistics; jackknife remains conceptually valuable and computationally cheaper for very large datasets where thousands of bootstrap resamples would be costly.

---

69. Cross-Validation as Statistical Resampling

69.1 The Statistical Framing

Cross-validation is, at heart, another **resampling procedure for estimating the sampling variability of a statistic** — here, a model's out-of-sample performance metric — using systematic held-out partitions rather than random-with-replacement resampling.

```
from sklearn.model_selection import KFold, StratifiedKFold,
RepeatedKFold, LeaveOneOut, TimeSeriesSplit
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

X_cv, y_cv = X_logit.drop(columns="const").values, churn

kf = KFold(n_splits=5, shuffle=True, random_state=0)
scores = []
for train_idx, test_idx in kf.split(X_cv):
    model = LogisticRegression().fit(X_cv[train_idx], y_cv[train_idx])
    p = model.predict_proba(X_cv[test_idx])[:, 1]
    scores.append(roc_auc_score(y_cv[test_idx], p))

print(f"5-fold CV AUC: {np.mean(scores):.4f} ± {np.std(scores):.4f}")
```

69.2 The Variant Catalogue

Variant	Use Case
K-Fold	General default
Stratified K-Fold	Classification with class imbalance — preserves class proportions per fold (Module 1, Section 18.4)
Repeated K-Fold	Reduces variance of the CV estimate itself by repeating with different random splits
Leave-One-Out (LOOCV)	Very small datasets; expensive (n model fits), can have surprisingly high variance despite the intuition it should be "most thorough"
Time-Series Split	Temporal data — respects chronological order, never trains on the future (Module 1, Section 18.4)

Both mean and standard deviation of the CV scores matter — the mean estimates expected performance; the spread across folds is itself an estimate of the *statistic's sampling variability*, directly informing Section 108's model-performance uncertainty and Section 109's model comparison.

---

70. Experimental Design

70.1 Core Vocabulary

Term	Meaning
Treatment	The intervention being tested
Control	The baseline/no-intervention comparison group
Randomization	Assigning units to treatment/control by chance, to balance both known and unknown confounders
Replication	Running the experiment across enough independent units for stable estimates
Blocking	Grouping similar units before randomizing within blocks, reducing noise from known nuisance factors
Confounding	A third variable that influences both treatment assignment and the outcome, distorting the apparent effect

70.2 Why Randomization Is the Single Most Important Design Choice

Randomization is what allows a **causal** claim ("the treatment caused the difference") rather than merely a correlational one (Section 74) — by construction, randomized assignment makes treatment and control groups statistically similar on **every** characteristic, measured and unmeasured, in expectation. This is the design-level solution to confounding that no amount of clever post-hoc statistical adjustment can fully replicate in observational data (Section 76).

---

71. A/B Testing — Complete Framework

71.1 The Full Workflow

```
1. Define the metric (e.g. conversion rate)
2. Define the minimum detectable effect (MDE) that matters practically
3. Power analysis → required sample size (Section 35)
4. Randomly assign users to control/treatment
5. Run for the pre-determined duration — DO NOT peek and stop early
6. Validate data quality (sample ratio mismatch, instrumentation)
7. Compute the difference in proportions/means
8. Confidence interval + hypothesis test
9. Effect size
10. Interpret: statistical AND practical significance together
```

71.2 Sample Size Planning

```
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize
```

```
baseline_rate, mde = 0.10, 0.02   # 10% baseline, want to detect a 2-
point lift
effect_size = proportion_effectsize(baseline_rate + mde,
baseline_rate)

analysis = NormalIndPower()
n_required = analysis.solve_power(effect_size=effect_size, alpha=0.05,
power=0.8)
print(f"Required sample size per group: {n_required:.0f}")
```

71.3 Analyzing the Result

```
n_control, conv_control = 5000, 480
n_treatment, conv_treatment = 5000, 560

p_control = conv_control/n_control
p_treatment = conv_treatment/n_treatment

# Two-proportion z-test
p_pool = (conv_control + conv_treatment) / (n_control + n_treatment)
se_pool = np.sqrt(p_pool*(1-p_pool)*(1/n_control + 1/n_treatment))
z_stat = (p_treatment - p_control) / se_pool
p_val = 2 * (1 - stats.norm.cdf(abs(z_stat)))

# CI for the difference (unpooled SE, the standard for CI
construction)
se_diff = np.sqrt(p_control*(1-p_control)/n_control + p_treatment*(1-
p_treatment)/n_treatment)
diff = p_treatment - p_control
ci_diff = (diff - 1.96*se_diff, diff + 1.96*se_diff)

relative_lift = diff / p_control
print(f"Control: {p_control:.3%}  Treatment: {p_treatment:.3%}")
print(f"Absolute lift: {diff:.3%}, 95% CI: ({ci_diff[0]:.3%},
{ci_diff[1]:.3%})")
print(f"Relative lift: {relative_lift:.1%}")
print(f"z={z_stat:.3f}, p={p_val:.5f}")
```

Both absolute and relative lift matter to different stakeholders — a 1-point absolute lift on a 2% baseline (a 50% relative lift) tells a very different story than the same absolute lift on a 50% baseline (a 2% relative lift), and reporting only one can mislead.

71.4 Why You Must Not Peek and Stop Early

Checking results daily and stopping "as soon as it's significant" inflates the false-positive rate dramatically beyond the nominal α=0.05 — this is a repeated-testing problem structurally identical to Section 46's multiple-comparisons inflation, applied across *time* instead of across *hypotheses*. The disciplined fix is either committing to the pre-calculated sample size/duration, or using a formal **sequential testing** method explicitly designed to allow valid early stopping — never informal peeking.

72. A/B Testing in Practice

72.1 A Complete Worked Example — Website Conversion

```
rng5 = np.random.default_rng(42)

# Simulate a realistic experiment with a sample-ratio-mismatch check
built in
n_users = 20000
assignment = rng5.choice(["control", "treatment"], n_users, p=[0.5,
0.5])
n_control_actual = (assignment == "control").sum()
n_treatment_actual = (assignment == "treatment").sum()

# Sample Ratio Mismatch (SRM) check — verify the split is genuinely
~50/50
srm_chi2, srm_p = stats.chisquare([n_control_actual,
n_treatment_actual],
                                  [n_users/2, n_users/2])
print(f"SRM check: p={srm_p:.4f}  ({'OK' if srm_p > 0.01 else
'INVESTIGATE — possible bug'})")

true_control_rate, true_lift = 0.12, 0.015
conversion_prob = np.where(assignment=="control", true_control_rate,
                           true_control_rate + true_lift)
converted = rng5.random(n_users) < conversion_prob

results_df = pd.DataFrame({"assignment": assignment, "converted":
converted})
summary = results_df.groupby("assignment")["converted"].agg(["mean",
"sum", "count"])
print(summary)

ctrl = results_df[results_df.assignment=="control"]["converted"]
trt = results_df[results_df.assignment=="treatment"]["converted"]
z_stat, p_val = proportion_effectsize(trt.mean(), ctrl.mean()), None
from statsmodels.stats.proportion import proportions_ztest
count = np.array([trt.sum(), ctrl.sum()])
nobs = np.array([len(trt), len(ctrl)])
z_stat, p_val = proportions_ztest(count, nobs)
print(f"z={z_stat:.3f}, p={p_val:.5f}")
```

72.2 Sample Ratio Mismatch — A Critical, Often-Skipped Check

SRM occurs when the actual split between control and treatment deviates from the intended ratio (49.5/50.5 instead of a designed 50/50) — often signaling a bug in the randomization or logging pipeline. **An experiment with SRM should not be trusted at all**, regardless of how the primary metric looks, since the mismatch itself is evidence something in the assignment mechanism is broken and likely correlated with the outcome. This single chi-square check

(Section 40.1) is a mandatory first gate before interpreting any A/B test result in professional practice.

---

73. Experimental Design for ML

73.1 ML-Specific Experimental Concerns

Concern	Description	Example
Model version testing	Comparing two model versions in production, not just offline metrics	Champion/challenger deployment
Recommendation/ranking systems	The treatment effect can differ by position, by user segment, or interact with other users (network effects)	A ranking change tested per-user, ignoring that a user's engagement is influenced by everyone else's feed
Randomization unit	User-level vs session-level vs request-level randomization changes what's measured and what contamination risks exist	Randomizing by session can let one user see both variants
Treatment contamination	Control-group users indirectly exposed to the treatment	A treatment email referral programme spreading effects to control-group friends
Sample ratio mismatch	Covered in Section 72.2	Assignment pipeline bugs
Experiment duration	Too short misses weekly/seasonal cycles; too long delays decisions and risks novelty effects fading or building	A feature that looks good in week 1 (novelty) but fades by week 4

73.2 Practical Guidance

Feature engineering and model-version experiments should follow the exact same rigor as marketing A/B tests: pre-registered success metrics, adequate power, SRM checks, and resistance to early stopping — the domain (an ML feature vs a marketing campaign) doesn't change the underlying statistical requirements at all. This is precisely where Module 8's ML Workflow will connect directly back to this module's experimental design foundation.

---

74. Causation vs Correlation

74.1 The Core Distinction, With Mechanisms

Failure Mode	Description	Example
Confounding	A third variable causes both	Ice cream sales & drownings,

Failure Mode	Description	Example
	X and Y	both caused by hot weather
Reverse causality	Y actually causes X, not the reverse	Wealthier customers buy premium support (not premium support causing wealth)
Selection bias	The observed sample's composition itself creates a spurious association	Only surviving companies are analyzed, hiding the failures

74.2 DAG Intuition

```
Confounding:          X ← Z → Y        (Z causes both; X,Y correlate
but neither causes the other)
Mediation:             X → M → Y        (M is the MECHANISM by which X
affects Y — not a confounder)
Collider:              X → C ← Y        (conditioning on C can CREATE
a spurious X-Y association)
```

Why this matters for ML feature engineering specifically: conditioning on (or filtering by) a collider variable can *manufacture* a correlation between two otherwise-unrelated features — a subtle, common, and genuinely dangerous source of misleading model behavior, developed further in Section 76.

74.3 What Establishes Causation

Randomized controlled experiments (Section 70) remain the gold standard. Where randomization is infeasible, quasi-experimental designs, careful confounder adjustment (Section 147's propensity scores), and — described only at a foundational level here — the broader causal inference toolkit (Section 146) exist specifically to approximate the guarantees randomization provides.

---

75. Simpson's Paradox

75.1 A Complete Numerical Example

```
# Two treatments, aggregated data suggests Treatment B is better...
treatment_a = pd.DataFrame({"group": ["Mild", "Severe"], "treated":
[81, 192], "total": [87, 263]})
treatment_b = pd.DataFrame({"group": ["Mild", "Severe"], "treated":
[234, 55],  "total": [270, 80]})

treatment_a["rate"] = treatment_a.treated / treatment_a.total
treatment_b["rate"] = treatment_b.treated / treatment_b.total
print("Treatment A by subgroup:\n", treatment_a)
print("Treatment B by subgroup:\n", treatment_b)

# Subgroup rates: A wins in BOTH subgroups
```

```
# Mild:   A=93.1%  B=86.7%   → A wins
# Severe: A=73.0%  B=68.8%   → A wins

overall_a = treatment_a.treated.sum() / treatment_a.total.sum()
overall_b = treatment_b.treated.sum() / treatment_b.total.sum()
print(f"\nOverall success rate — A: {overall_a:.1%}  B:
{overall_b:.1%}")
# But B wins OVERALL — because B was given disproportionately to the
easier "Mild" cases
```

The mechanism: Treatment A was used predominantly on **severe** cases (harder to treat) while Treatment B was used predominantly on **mild** cases (easier to treat). The subgroup case-mix — not the treatments' true effectiveness — drives the reversed aggregate conclusion. **Case mix (a confounder, per Section 76) is the variable doing the real work here.**

75.2 Implications

Domain	Simpson's Paradox Risk
Healthcare	A treatment appears best overall but is actually inferior in every patient subgroup
Hiring	Aggregate hiring rates by gender can reverse when broken down by department, if departments differ in both applicant pool and hiring rate
Marketing	An aggregated channel comparison can favor the channel disproportionately used on easier-to-convert segments
ML fairness	An aggregate fairness metric can mask, or even reverse, subgroup-level disparities — a direct link to Section 148
Business analytics	Year-over-year aggregate KPI trends can mislead when the underlying customer/product mix shifted

The standing discipline: whenever a decision hinges on a comparison, check whether it holds within *relevant subgroups*, not only in aggregate — especially when the groups being compared plausibly differ in composition.

---

76. Confounding

76.1 Confounder vs Mediator vs Collider

Term	DAG	Effect of "Controlling For" It
Confounder	Z → X, Z → Y	**Must** control for it to get an unbiased causal estimate
Mediator	X → M → Y	Controlling for it **removes**

Term	DAG	Effect of “Controlling For” It
		the genuine indirect effect you may want to measure
Collider	X → C ← Y	Controlling for it **introduces** bias that wasn’t there before

76.2 A Worked Mediator Mistake

```
# X = training programme, M = employee skill score, Y = performance
# If skill score is a MEDIATOR of the training's effect, controlling
for it
# in a regression predicting Y from X would (wrongly) suggest training
"doesn't matter" —
# because its effect operates THROUGH skill, and that channel has just
been blocked statistically.
```

The professional discipline: before adding a variable as a “control” in a regression, ask explicitly whether it’s a confounder (include it), a mediator of the effect you’re studying (exclude it, or model it separately), or a collider (never condition on it). This is not a full causal-inference course (Section 146 provides the fuller foundation) — but this three-way distinction alone prevents a large share of real-world analytical mistakes.

77. Bayesian Statistics

Building on Module 6's Bayes' theorem treatment (also revisited in Module 5, Section 57).

77.1 The Full Bayesian Workflow

```
Prior P(θ) → Likelihood P(data|θ) → Posterior P(θ|data) ∝ Likelihood × Prior → Posterior Predictive
```

```
# Bayesian estimation of a conversion rate, starting from a Beta prior
from scipy.stats import beta

prior_alpha, prior_beta = 2, 18   # a prior belief centered around 10% conversion
successes, trials = 45, 400        # observed data

posterior_alpha = prior_alpha + successes
posterior_beta = prior_beta + (trials - successes)

x = np.linspace(0, 0.3, 500)
prior_pdf = beta.pdf(x, prior_alpha, prior_beta)
posterior_pdf = beta.pdf(x, posterior_alpha, posterior_beta)

plt.plot(x, prior_pdf, label="Prior")
plt.plot(x, posterior_pdf, label="Posterior")
plt.legend(); plt.xlabel("Conversion rate")

posterior_mean = posterior_alpha / (posterior_alpha + posterior_beta)
credible_interval = beta.interval(0.95, posterior_alpha, posterior_beta)
print(f"Posterior mean: {posterior_mean:.4f}")
print(f"95% credible interval: {credible_interval}")
```

The Beta-Binomial conjugacy used here (a Beta prior combined with Binomial/Bernoulli data yields a Beta posterior in closed form) is not a coincidence — it's a direct instance of the exponential-family structure developed formally in Section 143, and is why Beta priors are the standard choice for conversion-rate/proportion problems specifically.

77.2 Credible Interval — The Interpretation People Actually Want

Unlike the frequentist confidence interval (Section 26.2), the Bayesian credible interval genuinely supports the intuitive interpretation: *"there is a 95% probability the true conversion rate lies in this interval, given the observed data and the stated prior."* This is possible because the Bayesian framework treats θ itself as a random variable with a probability distribution, not as a fixed unknown constant — a fundamentally different starting philosophy from the frequentist framework, developed fully in Section 78.

---

78. Frequentist vs Bayesian Inference

Aspect	Frequentist	Bayesian
Parameter interpretation	A fixed, unknown constant	A random variable with a probability distribution
Probability interpretation	Long-run frequency across repeated sampling	Degree of belief, updated by evidence
Confidence/Credible interval	"95% of such intervals would contain θ" (procedure property)	"95% probability θ is in this interval" (direct statement)
P-value	P(data this extreme | H_0 true)	Not typically used; Bayesian model comparison uses Bayes factors instead
Prior	Not used (or treated as illegitimate)	Central — an explicit, defensible starting belief
Posterior	Not applicable	The updated belief after seeing data
Likelihood	Central (MLE, Section 24)	Central (combined with the prior)
Decision-making	Reject/fail-to-reject a null hypothesis	Full posterior distribution informs a decision directly, often via expected loss (Section 145)

Neither is universally superior — they answer different questions and carry different costs. Frequentist methods dominate in regulated, standardized reporting contexts (clinical trials, much of classical business analytics) partly due to established convention and simpler computation; Bayesian methods excel when genuine prior information exists, when small-sample honest uncertainty quantification matters, or when sequential updating as data streams in is natural (e.g. online A/B testing, or an ML system continuously retrained with growing data).

---

79. Bayesian Estimation

79.1 Point Estimates From a Posterior

Estimate	Definition	When Preferred
Posterior mean	$E[\theta \mid \text{data}]$	Standard default; minimizes expected squared-error loss (Section 145)
Posterior median	The 50th percentile of the posterior	More robust to a skewed posterior
MAP (Maximum A Posteriori)	$\text{argmax}_\theta\ P(\theta \mid \text{data})$	The mode of the posterior; equivalent to MLE with a flat

Estimate	Definition	When Preferred
		(uninformative) prior

```
posterior_median = beta.median(posterior_alpha, posterior_beta)
posterior_mode = (posterior_alpha - 1) / (posterior_alpha +
posterior_beta - 2)   # Beta's MAP formula
print(f"Posterior mean: {posterior_mean:.4f}")
print(f"Posterior median: {posterior_median:.4f}")
print(f"Posterior mode (MAP): {posterior_mode:.4f}")
```

MAP's connection to regularization: Module 5's Section 72 showed Ridge regression's penalty term. **MAP estimation with a Gaussian prior on the coefficients produces exactly Ridge regression**, and MAP with a Laplace prior produces exactly Lasso — regularization is not a separate, ad hoc technique from Bayesian estimation; it *is* Bayesian MAP estimation under a specific prior choice, one of the most elegant unifications in this entire module.

---

80. Bayesian Regression

80.1 The Setup

```
Prior:      β ~ Normal(0, τ²I)
Likelihood: y | X, β ~ Normal(Xβ, σ²I)
Posterior:  β | X, y ~ Normal(...)     (closed form under conjugate
priors)

# A simple Bayesian linear regression via conjugate Normal-Normal
updating
X_bayes = sm.add_constant(x)   # reusing Section 51's data
tau2 = 100.0    # prior variance (weakly informative)
sigma2 = 25.0   # assumed noise variance

XtX = X_bayes.T @ X_bayes
posterior_cov = np.linalg.inv(XtX/sigma2 +
np.eye(X_bayes.shape[1])/tau2)
posterior_mean = posterior_cov @ (X_bayes.T @ y / sigma2)

print("Posterior mean (β):", posterior_mean.round(3))
print("Posterior SD (β):", np.sqrt(np.diag(posterior_cov)).round(3))

# Compare to OLS point estimate
ols_beta = np.linalg.lstsq(X_bayes, y, rcond=None)[0]
print("OLS point estimate:", ols_beta.round(3))
```

As the prior becomes uninformative ($\tau^2 \to \infty$), the Bayesian posterior mean converges to the OLS estimate — another direct illustration that classical and Bayesian regression aren't opposed methods but sit on a continuum, with the choice of prior determining how much the classical answer gets "pulled" toward prior belief.

80.2 Predictive Uncertainty

A genuine advantage of Bayesian regression: predictions carry a full **posterior predictive distribution**, naturally quantifying uncertainty for a specific new point — including both parameter uncertainty *and* residual noise — rather than a single point prediction requiring a separately bolted-on prediction interval (Section 105).

---

81. Statistical Distribution Fitting

81.1 The Workflow

```
1. Visualize the data (histogram, ECDF)
2. Propose candidate distributions based on domain knowledge and shape
3. Estimate parameters (MLE, Section 24)
4. Assess fit (Q-Q plot, goodness-of-fit test, AIC/BIC)
5. Choose the best-fitting, most defensible candidate
```

```
data_to_fit = rng.gamma(shape=2, scale=3, size=1000) + rng.normal(0,
0.5, 1000)

candidates = {"norm": stats.norm, "gamma": stats.gamma, "lognorm":
stats.lognorm, "expon": stats.expon}
results = {}
for name, dist in candidates.items():
    params = dist.fit(data_to_fit)
    log_lik = dist.logpdf(data_to_fit, *params).sum()
    k = len(params)
    aic = 2*k - 2*log_lik
    results[name] = {"params": params, "AIC": aic}

for name, r in sorted(results.items(), key=lambda kv: kv[1]["AIC"]):
    print(f"{name:10s}: AIC={r['AIC']:.1f}")
```

AIC comparison across candidate distributions directly extends Section 60's model-selection logic to distribution fitting: the candidate with the lowest AIC offers the best fit-complexity tradeoff among those tried — never a certificate that the "true" distribution has been found, only the best among the candidates actually tested.

---

82. Q-Q Plots

82.1 Construction and Reading

A **Q-Q (quantile-quantile) plot** plots the data's empirical quantiles against a theoretical distribution's quantiles — if the data genuinely follows that distribution, points fall approximately on the 45° diagonal line.

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
```

```
normal_data = rng.normal(0, 1, 500)
skewed_data = rng.exponential(1, 500)
heavy_tailed = rng.standard_t(df=3, size=500)

for ax, d, title in zip(axes, [normal_data, skewed_data,
heavy_tailed],
                        ["Normal data", "Skewed data", "Heavy-tailed
data"]):
    stats.probplot(d, dist="norm", plot=ax)
    ax.set_title(title)
```

Q-Q Pattern	Interpretation
Points on the diagonal	Data matches the theoretical distribution well
S-curve (bowed)	Skewness present
Points diverge at both ends	Heavy or light tails relative to the reference distribution

Direct use in regression diagnostics (Section 54): applying a normal Q-Q plot to the model's *residuals* (not the raw data) is the standard visual check for the normality-of-residuals assumption — often more informative than a formal test (Section 83) alone, since it shows *where* and *how* departure occurs, not merely whether it's statistically detectable.

---

83. Goodness-of-Fit

Test	Tests	Notes
Chi-square goodness-of-fit	Categorical/binned data against expected frequencies	Sensitive to bin choice (Section 40.1)
Kolmogorov-Smirnov (K-S)	Continuous data against a specified distribution	Compares the empirical CDF (Section 12) to the theoretical CDF directly
Shapiro-Wilk	Specifically tests normality	Generally the most powerful normality test for small-to-moderate n
Anderson-Darling	Distributional fit, with extra sensitivity in the tails	More powerful than K-S for detecting tail departures

```
sample_to_test = rng.normal(0, 1, 200)

ks_stat, ks_p = stats.kstest(sample_to_test, 'norm',
args=(sample_to_test.mean(), sample_to_test.std()))
shapiro_stat, shapiro_p = stats.shapiro(sample_to_test)
print(f"K-S test:       stat={ks_stat:.4f}, p={ks_p:.4f}")
print(f"Shapiro-Wilk:   stat={shapiro_stat:.4f}, p={shapiro_p:.4f}")
```

83.1 Limitations — Especially at Scale

With **very large n**, goodness-of-fit tests become so powerful that they reject the null (perfect fit) for even trivial, practically irrelevant departures from the theoretical distribution — mirroring Section 33.3's statistical-vs-practical-significance warning exactly. **Visual inspection (histograms, Q-Q plots) alongside formal tests, not formal tests alone, is the professional standard** — a formal test answers "is this detectably different," a visual answers "does it matter."

---

84. Robust Statistics

84.1 The Toolkit

Method	Formula/Idea	Robust Alternative To
Trimmed mean	Mean after removing the top/bottom k%	Ordinary mean
Winsorization	Cap (not remove) extreme values at a percentile	Ordinary mean/variance
Median Absolute Deviation (MAD)	median($\|x_i - \text{median}(x)\|$)	Standard deviation
Robust regression (Huber, RANSAC)	Down-weight or exclude outlier influence in fitting	OLS

```
data_outlier = np.array([100, 102, 98, 105, 99, 101, 103, 5000])

print(f"Mean: {data_outlier.mean():.1f}")                    # wildly
distorted
print(f"Trimmed mean (10%): {stats.trim_mean(data_outlier, 0.1):.1f}")
from scipy.stats.mstats import winsorize
print(f"Winsorized mean: {winsorize(data_outlier, limits=[0.1,
0.1]).mean():.1f}")

mad = stats.median_abs_deviation(data_outlier)
print(f"MAD: {mad:.1f}  (vs SD: {data_outlier.std():.1f})")
```

When robust methods are worth using: whenever outliers are expected to be genuine but non-representative of the "typical" pattern you want to summarize (income, transaction amounts, sensor glitches) — robust statistics deliberately trade some efficiency under perfectly clean data for dramatically better behavior when the data isn't clean, which describes most real business data.

---

85. Missing Data

85.1 The Three Mechanisms

Mechanism	Definition	Example
MCAR (Missing Completely At Random)	Missingness is unrelated to any observed or unobserved values	A random data-transmission glitch
MAR (Missing At Random)	Missingness depends on *observed* variables, not the missing value itself	Younger customers skip an income question more often, but *given age*, missingness is random
MNAR (Missing Not At Random)	Missingness depends on the *unobserved value itself*	High earners specifically decline to report income

Why the mechanism matters enormously: under MCAR, simply dropping missing rows (complete-case analysis) is unbiased, just less efficient. Under MAR, dropping rows can introduce bias unless the missingness-predicting variables are properly accounted for (motivating model-based imputation). **Under MNAR, no amount of clever imputation using only the observed data can fully fix the bias** — the missingness itself carries information that isn't recoverable from what's present, a genuinely hard problem requiring domain-specific handling or external data.

85.2 Diagnosing the Mechanism (Practically)

```
df_missing = pd.DataFrame({
    "age": rng.integers(20, 70, 500).astype(float),
    "income": rng.normal(500_000, 150_000, 500),
})
# Simulate MAR: younger customers more likely to have missing income
missing_prob = np.where(df_missing.age < 30, 0.4, 0.05)
df_missing.loc[rng.random(500) < missing_prob, "income"] = np.nan

print(f"Missing rate: {df_missing.income.isna().mean():.1%}")
# Check whether missingness relates to an OBSERVED variable — evidence
for MAR
missing_indicator = df_missing.income.isna()
t_stat, p_val = stats.ttest_ind(df_missing.age[missing_indicator],
df_missing.age[~missing_indicator])
print(f"Age differs by missingness status: t={t_stat:.2f},
p={p_val:.5f}")
# A significant result here supports MAR (not MCAR) — missingness
relates to age
```

---

86. Imputation From a Statistical Perspective

Method	Description	Risk
Mean/median/mode	Fill with a central tendency	**Shrinks variance**

Method	Description	Risk
imputation	measure	**artificially**, distorts correlations
Regression imputation	Predict the missing value from other features	Understates uncertainty; can leak target information if done carelessly
KNN imputation	Fill using similar observations' values	Sensitive to feature scaling (Module 5, Section 16.3)
Multiple imputation	Generate several plausible imputed datasets, analyze each, pool results	The statistically principled gold standard — properly propagates imputation uncertainty into final inference

```
from sklearn.impute import SimpleImputer, KNNImputer

income_before_var = df_missing.income.var()
imputer_mean = SimpleImputer(strategy="mean")
income_mean_imputed =
imputer_mean.fit_transform(df_missing[["income"]])
print(f"Original variance: {income_before_var:,.0f}")
print(f"Post mean-imputation variance:
{pd.Series(income_mean_imputed.ravel()).var():,.0f}")
# Variance shrinks — every imputed value is IDENTICAL (the mean),
artificially reducing spread
```

The critical, easily-missed danger: data leakage in imputation. Fitting the imputer (computing the mean/regression/KNN neighbors) on the **full dataset including the test set** leaks test information into training — the exact same discipline as Module 1's Section 18.5 and Module 5's Section 64.4: **fit the imputer on training data only**, then apply it to validation/test.

---

87. Bootstrap and Uncertainty

87.1 Connecting Resampling to Every Downstream Use

This section is deliberately a **connective bridge**, tying Section 67's bootstrap mechanics to where they recur throughout the rest of the module:

Application	Section
Model performance metric uncertainty	Section 108
Confidence intervals for non-standard statistics (median, custom business metrics)	Section 67
Prediction intervals for individual forecasts	Section 105
Comparing two models' performance	Section 109

Application	Section
distributions	
Quantifying uncertainty around a fitted regression coefficient without distributional assumptions	An alternative to Section 51's t-based CI

```
# Bootstrap a regression coefficient's CI, no normality assumption
required
def fit_slope(indices, x_data, y_data):
    xs, ys = x_data[indices], y_data[indices]
    return np.sum((xs-xs.mean())*(ys-ys.mean())) / np.sum((xs-
xs.mean())**2)

n_boot = 5000
boot_slopes = np.empty(n_boot)
idx_all = np.arange(len(x))
for i in range(n_boot):
    boot_idx = rng.choice(idx_all, len(x), replace=True)
    boot_slopes[i] = fit_slope(boot_idx, x, y)

print(f"Bootstrap slope 95% CI: ({np.percentile(boot_slopes,2.5):.3f},
"
      f"{np.percentile(boot_slopes,97.5):.3f})")
```

The unifying idea worth carrying forward: whenever a closed-form formula for a statistic's uncertainty is unavailable, complicated, or relies on assumptions you don't trust, bootstrap resampling is the general-purpose fallback — a single technique replacing dozens of specialized formulas.

88. Statistical Learning Theory Introduction

88.1 Reframing Module 1's Concepts in Statistical Language

Module 1 Term	Statistical Learning Term
Training performance	Empirical risk on the training sample
Test performance	An estimate of the true expected risk on the population
Generalization	How closely empirical risk approximates true expected risk
Overfitting	Empirical risk far understates true expected risk (fitting sample-specific noise)
Underfitting	Empirical risk itself is poor — the model class can't represent the true relationship

Statistical learning theory formalizes exactly why held-out test performance is a valid, unbiased-in-expectation estimate of true future performance — the identical logic underlying every sampling-distribution argument from Section 19, now applied to a model's *error* as the statistic of interest rather than a simple mean or proportion.

88.2 Bias and Variance, Restated Once More

This is the **third** appearance of bias-variance in this curriculum (Module 1, Section 21; Module 5, Section 73) — appropriate, since it is genuinely one of the handful of ideas that recur at every level of statistical and ML reasoning. Here the framing is explicitly statistical: bias is a property of the **estimator** (the learning algorithm), variance is the **sampling variability** of that estimator's output across different training samples — precisely Section 22's estimator-property vocabulary, now applied to an entire learning algorithm rather than a simple parameter estimate.

---

89. Bias-Variance Tradeoff — Statistical Simulation

```
def true_function(x): return 3 + 2*x - 0.5*x**2

rng6 = np.random.default_rng(7)
x_grid = np.array([1.5])
n_train, n_trials, noise_sd = 25, 300, 3.0

degree_results = {}
for degree in [1, 2, 8]:
    predictions = []
    for _ in range(n_trials):
        x_sample = rng6.uniform(0, 5, n_train)
        y_sample = true_function(x_sample) + rng6.normal(0, noise_sd,
n_train)
        coeffs = np.polyfit(x_sample, y_sample, degree)
        predictions.append(np.polyval(coeffs, x_grid)[0])
```

```
    predictions = np.array(predictions)
    bias_sq = (predictions.mean() - true_function(x_grid[0]))**2
    variance = predictions.var()
    degree_results[degree] = (bias_sq, variance,
bias_sq+variance+noise_sd**2)

print(f"{'Degree':>7} {'Bias²':>10} {'Variance':>10} {'Total
(+σ²)':>12}")
for d, (b, v, t) in degree_results.items():
    print(f"{d:>7} {b:>10.3f} {v:>10.3f} {t:>12.3f}")
```

This exact simulation pattern first appeared in Module 5's Section 73.3 — repeated here specifically because it is the fulcrum on which statistical learning theory, model selection (Sections 58-60), and regularization (Section 92) all pivot. **Total expected error = Bias² + Variance + Irreducible error, always** — every technique in the rest of this section either trades bias for variance (regularization) or reduces variance without touching bias (more data, ensembling).

---

90. Empirical Risk Minimization

90.1 The Formal Objects

```
Loss:            L(y, ŷ)                              — error on one
example
True Risk:       R(f) = E_(X,Y)[L(Y, f(X))]           — the TRUE expected
error (unknown, over the full population)
Empirical Risk:  R̂(f) = (1/n)Σ L(yᵢ, f(xᵢ))           — the AVERAGE
error on your specific sample
ERM:             f̂ = argmin_f R̂(f)                    — what training
actually does
```

The entire justification for "training = minimizing average loss on your sample" is that R̂(f) is an unbiased, LLN-consistent estimator of the true risk R(f) (Sections 21, 24) — provided the sample is representative (Section 18) and independent. Every model you have ever trained, in every prior module, has been performing ERM; this section simply names the formal object precisely.

90.2 Why ERM Alone Isn't Sufficient — the Gap

```
# Minimizing empirical risk WITHOUT capacity control overfits —
illustrated directly
x_train = rng6.uniform(0, 5, 15)
y_train = true_function(x_train) + rng6.normal(0, noise_sd, 15)
x_test = rng6.uniform(0, 5, 200)
y_test = true_function(x_test) + rng6.normal(0, noise_sd, 200)

for degree in [1, 2, 14]:
    coeffs = np.polyfit(x_train, y_train, degree)
    train_risk = np.mean((y_train - np.polyval(coeffs, x_train))**2)
    test_risk = np.mean((y_test - np.polyval(coeffs, x_test))**2)
```

```
    print(f"degree={degree:2d}: empirical (train)
risk={train_risk:7.2f}  true (test) risk={test_risk:9.2f}")
```

At high degree, empirical risk keeps falling while true risk explodes — the empirical/true risk **gap grows with model capacity**, which is precisely why unconstrained ERM is dangerous and why regularization (constraining the search, Section 92) or cross-validation (Section 69, honestly estimating true risk) are both necessary companions to ERM in practice, never ERM alone.

---

91. Maximum Likelihood and Loss Functions

91.1 The Complete Unification Table

This section consolidates a thread woven throughout Modules 5-7:

Assumed Noise Distribution	MLE Objective	Equivalent Common ML Loss
Gaussian	Maximize Gaussian log-likelihood	**MSE / squared error**
Bernoulli	Maximize Bernoulli log-likelihood	**Binary cross-entropy / log loss**
Categorical	Maximize Categorical log-likelihood	**Categorical cross-entropy**
Laplace	Maximize Laplace log-likelihood	**MAE (absolute error)**
Poisson	Maximize Poisson log-likelihood	**Poisson deviance loss**

```
# Demonstrating MAE ⇔ Laplace-noise MLE numerically
from scipy.optimize import minimize_scalar

data_laplace = rng.laplace(loc=5, scale=2, size=1000)

# MLE via direct log-likelihood maximization
neg_log_lik = lambda mu: -stats.laplace.logpdf(data_laplace, mu,
2).sum()
mle_result = minimize_scalar(neg_log_lik, bounds=(0, 10),
method='bounded')

# MAE-minimizing value = the MEDIAN (a known, provable result)
print(f"MLE estimate: {mle_result.x:.3f}")
print(f"Sample median (MAE-minimizer): {np.median(data_laplace):.3f}")
# These converge — confirming MAE-minimization IS Laplace-noise MLE
```

Every loss function is a distributional assumption in disguise. Choosing MSE implicitly assumes symmetric, light-tailed (Gaussian) errors; choosing MAE implicitly assumes heavier-tailed (Laplace) errors and targets the conditional median rather than the mean — a choice

with real consequences (Module 5, Section 71.2) that most practitioners make without realizing they're making a distributional assumption at all.

---

92. Regularization as Statistical Modeling

92.1 The MAP Connection, Formalized

Building on Section 79.1's preview:

```
Ridge:    argmin [ MSE(β) + λΣβⱼ² ]  ⇔  MAP under a Gaussian prior on β
Lasso:    argmin [ MSE(β) + λΣ|βⱼ| ] ⇔  MAP under a Laplace (double-exponential) prior on β
```

```python
# Empirically confirming Ridge = MAP with a Gaussian prior
from sklearn.linear_model import Ridge

X_reg, y_reg = X_df[["sqft","bedrooms","age"]].values, y_price.values
ridge_model = Ridge(alpha=100).fit(X_reg, y_reg)
print("Ridge (frequentist) coefficients:", ridge_model.coef_.round(3))

# A Bayesian MAP estimate with an explicit Gaussian prior gives the SAME answer
tau2_prior = 1e6 / 100   # chosen to match Ridge's alpha via the known equivalence
X_design = sm.add_constant(X_reg)
XtX = X_design.T @ X_design
map_coef = np.linalg.solve(XtX + (1/tau2_prior)*np.eye(X_design.shape[1]), X_design.T @ y_reg)
print("Bayesian MAP coefficients:  ", map_coef[1:].round(3))
```

This is genuinely one of the most valuable unifications a practitioner can internalize: "regularization" and "Bayesian priors" are not two separate toolkits to learn — choosing a regularization strength λ **is** choosing how strong a prior belief you hold that coefficients should be small, made concrete and computable without ever explicitly invoking Bayesian machinery.

92.2 Bias-Variance, One More Time

Regularization deliberately introduces **bias** (shrinking coefficients away from their unconstrained OLS values) to reduce **variance** (less sensitivity to the specific training sample) — the canonical, textbook example of the Section 89 tradeoff being *exploited on purpose*, not merely observed as a diagnostic.

---

93. Feature Selection From a Statistical Perspective

Method	Approach	Caveat
Correlation-based	Rank features by |	Misses non-linear

Method	Approach	Caveat
	correlation| with the target	relationships (Section 48)
Mutual information	Measures general (including non-linear) statistical dependence	More computationally expensive; no sign/direction information
Chi-square selection	Tests categorical feature-target association (Section 40)	Requires categorical/binned features
ANOVA F-test selection	Tests whether a numeric feature's means differ across target classes	Assumes normality per class, similar to Section 42

```
from sklearn.feature_selection import SelectKBest, f_classif,
mutual_info_classif, chi2

X_features = pd.DataFrame({
    "tenure": tenure, "support_calls": support_calls,
    "random_noise": rng.normal(0, 1, len(churn))   # a deliberately
useless feature
})

f_scores, f_pvalues = f_classif(X_features, churn)
mi_scores = mutual_info_classif(X_features, churn, random_state=0)

selection_summary = pd.DataFrame({"feature": X_features.columns,
                                  "F_score": f_scores.round(2),
"F_pvalue": f_pvalues.round(4),
                                  "mutual_info": mi_scores.round(4)})
print(selection_summary.sort_values("F_score", ascending=False))
# random_noise should show a high p-value and low mutual information,
correctly flagged as unhelpful
```

93.1 Statistical Significance vs Predictive Usefulness

A feature can be **statistically significantly** associated with the target (given enough data, Section 33.3) while contributing negligible **predictive** value once other correlated features are already included — significance and usefulness are related but distinct questions, exactly mirroring the p-value/effect-size distinction from Section 45.

93.2 The Data Leakage Warning

Feature selection must be performed on training data only, refit within each cross-validation fold — selecting features using the full dataset (including what will become the test set) is a well-documented, common form of leakage that silently inflates reported performance, directly extending Module 1's Section 18.5 leakage discussion to this specific pipeline stage.

---

94. Dimensionality Reduction and Statistics

Building directly on Module 5's Sections 50-52 (eigenvalues, eigenvectors, PCA), now framed statistically.

```
X_multi = df[["age", "income", "tenure"]].values
X_std = (X_multi - X_multi.mean(axis=0)) / X_multi.std(axis=0)

cov_matrix = np.cov(X_std, rowvar=False)
eigvals, eigvecs = np.linalg.eigh(cov_matrix)
order = np.argsort(eigvals)[::-1]
eigvals, eigvecs = eigvals[order], eigvecs[:, order]

explained_variance_ratio = eigvals / eigvals.sum()
print("Explained variance ratio:", explained_variance_ratio.round(4))
print("Cumulative:", np.cumsum(explained_variance_ratio).round(4))
```

The statistical framing, made explicit: PCA finds the linear combinations of correlated features that capture the most **variance** — and variance, in the statistical sense developed throughout Sections 8, 60, and 89, is precisely "information about how much the data varies," making PCA a direct, principled application of everything this module has built toward variance as the central currency of statistical reasoning.

---

95. Multivariate Statistics

95.1 The Multivariate Normal Distribution

$$X \sim MVN(\mu, \Sigma)$$

where μ is a vector of means and Σ is the full covariance matrix (Section 49) — the natural multi-dimensional generalization of the univariate normal (Module 5, Section 63).

95.2 Mahalanobis Distance

$$D_M(x) = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)}$$

```
mean_vec = X_multi.mean(axis=0)
cov_matrix_raw = np.cov(X_multi, rowvar=False)
cov_inv = np.linalg.inv(cov_matrix_raw)

def mahalanobis(x, mean, cov_inv):
    diff = x - mean
    return np.sqrt(diff @ cov_inv @ diff)

new_point = np.array([45, 5_000_000, 30])   # an outlier on income
specifically
d_maha = mahalanobis(new_point, mean_vec, cov_inv)
d_euclid = np.linalg.norm((new_point - mean_vec) /
X_multi.std(axis=0))
```

```
print(f"Mahalanobis distance: {d_maha:.2f}")
print(f"(Scale-naive) Euclidean distance: {d_euclid:.2f}")
```

Why Mahalanobis distance beats Euclidean for multivariate outlier detection: it accounts for the **correlation structure** between features, not just their individual scales (which standardization alone handles, Module 5 Section 16.3) — a point that's unremarkable on each feature individually can still be a genuine multivariate outlier if it violates the *typical joint relationship* between features (e.g. very high income but very low tenure, when income and tenure are usually correlated). This is the statistically correct generalization of the univariate z-score (Section 15) to multiple dimensions simultaneously.

95.3 MANOVA — Brief Introduction

MANOVA (Multivariate ANOVA) extends Section 42's one-way ANOVA to test whether group means differ across **several outcome variables simultaneously**, accounting for correlations among those outcomes — used when, for example, comparing marketing channels not on one metric but jointly on conversion rate, order value, *and* retention at once. Covered here only at the conceptual level; full MANOVA mechanics belong to a dedicated multivariate-methods course.

---

96. Principal Component Analysis — Three Perspectives, Unified

Perspective	View	Where Covered
Statistical	Find linear combinations of features maximizing captured variance	Section 94
Geometric	Rotate coordinate axes to align with the data's directions of greatest spread	Module 5, Section 49.3
Matrix/linear algebra	Eigendecompose the covariance matrix; eigenvectors are components, eigenvalues are variances	Module 5, Section 52

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X_std)
print("Explained variance ratio:",
pca.explained_variance_ratio_.round(4))
print("Components (loadings):\n", pd.DataFrame(pca.components_,
columns=["age","income","tenure"]))
```

All three perspectives describe the identical mathematical object — a genuinely valuable realization for interview settings, where PCA is frequently asked about from whichever angle (statistics, linear algebra, or ML) the interviewer's own background favors, and being able to fluently switch between framings signals real, not memorized, understanding.

97. Factor Analysis

97.1 PCA vs Factor Analysis — The Key Conceptual Difference

	PCA	Factor Analysis
Goal	Maximize explained variance; a data-reduction/compression technique	Explain **correlations** among observed variables via a smaller number of **latent (unobserved) factors** assumed to causally underlie them
Model assumption	None — purely a mathematical decomposition	Assumes observed variables = factor loadings × latent factors + unique error
Components/Factors interpretable as "real" constructs?	Not by design — purely mathematical directions of variance	Often explicitly intended to represent a real underlying construct (e.g. "customer satisfaction" as a latent factor behind several survey items)

```
from sklearn.decomposition import FactorAnalysis

survey_items = pd.DataFrame({
    "q1_ease_of_use": rng.normal(4, 1, 300),
    "q2_speed": rng.normal(3.5, 1, 300),
    "q3_reliability": rng.normal(4.2, 1, 300),
})
# Construct correlated items reflecting one latent "satisfaction"
factor
latent_satisfaction = rng.normal(0, 1, 300)
for col in survey_items.columns:
    survey_items[col] += 0.7 * latent_satisfaction

fa = FactorAnalysis(n_components=1)
factor_scores = fa.fit_transform(survey_items)
print("Factor loadings:", fa.components_.round(3))
```

Practical guidance: use PCA for dimensionality reduction as a *preprocessing* step for a downstream model (Module 5, Section 94); use factor analysis when the actual research/business question is about identifying and measuring **latent constructs** (satisfaction, brand perception, financial risk appetite) from a battery of related observed indicators — a genuinely different goal that happens to use similar-looking linear-algebra machinery.

98. Clustering and Statistics

98.1 K-Means as Variance Minimization

Directly recapping Module 5's Section 82: K-Means minimizes **within-cluster sum of squares** — a variance-minimization objective, exactly analogous to how OLS minimizes squared residual variance around a fitted line.

98.2 Gaussian Mixture Models — Probabilistic Clustering

```
from sklearn.mixture import GaussianMixture

X_cluster = np.vstack([rng.normal([0,0], 0.5, (100,2)),
rng.normal([5,5], 0.8, (100,2))])

gmm = GaussianMixture(n_components=2, random_state=0).fit(X_cluster)
soft_assignments = gmm.predict_proba(X_cluster)
print("Sample soft cluster assignments (probabilities):\n",
soft_assignments[:5].round(3))
print("Cluster means:\n", gmm.means_.round(2))
print("Cluster covariances (shape):", gmm.covariances_.shape)
```

K-Means vs GMM, statistically: K-Means assigns each point to exactly **one** cluster (hard assignment) and implicitly assumes spherical, equal-sized clusters. **GMM is a genuinely probabilistic model** — it assumes data arises from a mixture of several multivariate normal distributions (Section 95.1), fitted via MLE (specifically, the EM algorithm), and returns a full probability of membership in each cluster per point (soft assignment) rather than a single hard label — a materially richer statistical object, at the cost of more parameters to estimate and a stronger distributional assumption.

98.3 Cluster Validation

```
from sklearn.metrics import silhouette_score
labels_kmeans = gmm.predict(X_cluster)
sil_score = silhouette_score(X_cluster, labels_kmeans)
print(f"Silhouette score: {sil_score:.3f}")   # closer to 1 = well-
separated clusters
```

Within-cluster variance (WCSS, Module 5's Section 82.4 elbow method) and the silhouette score both serve the same statistical purpose as any goodness-of-fit measure (Section 83): quantifying how well a proposed structure (here, k clusters) actually organizes the observed variance in the data.

99. Time-Series Statistics

99.1 The Core Vocabulary

Component	Meaning
Trend	A long-run upward/downward movement
Seasonality	A regular, calendar-linked repeating pattern (weekly, monthly, yearly)
Cycles	Longer, non-fixed-period fluctuations (e.g. economic cycles)
Noise	The irreducible random component remaining after trend/seasonality/cycles are removed
Autocorrelation	Correlation of the series with a lagged version of itself (Section 101)
Partial autocorrelation	Correlation at a given lag, with the effect of shorter lags removed
Stationarity	Statistical properties (mean, variance, autocovariance) are stable over time (Section 100)

```
t = np.arange(365*2)
trend = 0.05 * t
seasonality = 20 * np.sin(2*np.pi*t/365)
noise = rng.normal(0, 5, len(t))
series = 100 + trend + seasonality + noise

from statsmodels.tsa.seasonal import seasonal_decompose
ts = pd.Series(series, index=pd.date_range("2023-01-01",
periods=len(series)))
decomposition = seasonal_decompose(ts, model='additive', period=365)
decomposition.plot()
```

This module deliberately stops at the statistical *foundations* — a dedicated Time-Series ML module (referenced but not covered here) builds forecasting models (ARIMA, exponential smoothing, ML-based approaches) directly on top of everything in Sections 99-102.

---

100. Stationarity

100.1 Weak vs Strong Stationarity

Weak (covariance) stationarity: constant mean, constant variance, and autocovariance depending only on the *lag* (not the specific time point) — the practical, testable standard used almost universally. **Strong stationarity** additionally requires the *entire joint distribution* to be time-invariant — a much stronger, rarely directly tested condition.

100.2 Testing for Stationarity

```
from statsmodels.tsa.stattools import adfuller, kpss

non_stationary = pd.Series(t*0.1 + rng.normal(0,5,len(t)))   # has a
trend
stationary = pd.Series(rng.normal(0, 5, len(t)))              #
genuinely stationary

for name, s in [("Non-stationary (trend)", non_stationary),
("Stationary", stationary)]:
    adf_stat, adf_p, *_ = adfuller(s)
    kpss_stat, kpss_p, *_ = kpss(s, nlags="auto")
    print(f"{name}: ADF p={adf_p:.4f}  KPSS p={kpss_p:.4f}")
```

Test	Null Hypothesis	Interpretation of a small p-value
ADF (Augmented Dickey-Fuller)	The series has a unit root (is non-stationary)	Reject → evidence FOR stationarity
KPSS	The series IS stationary	Reject → evidence AGAINST stationarity

Note the opposite null hypotheses — a genuinely common source of confusion. Best practice runs **both** tests: agreement (ADF rejects, KPSS doesn't) gives confidence; disagreement flags an ambiguous case warranting closer visual inspection. **Limitation:** both tests have limited power in small samples and can be misled by structural breaks (a sudden regime change) masquerading as a unit root.

---

101. Autocorrelation

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
plot_acf(stationary, lags=30, ax=axes[0])
plot_pacf(stationary, lags=30, ax=axes[1])
```

Plot	Shows	Diagnostic Use
ACF	Correlation at each lag, including indirect effects through intermediate lags	Identifies the order of an MA (Moving Average) component
PACF	Correlation at each lag, with shorter-lag effects removed	Identifies the order of an AR (AutoRegressive) component

Direct connection to Module 1 and Module 5: significant autocorrelation is the exact mathematical reason random train/test splitting is invalid for time-series data (Module 1, Section 18.4; Module 5, Section 95.1) — if yesterday strongly predicts today, a random split

leaks near-duplicate information across the train/test boundary, producing inflated, unrealistic validation metrics.

---

102. ARIMA — Statistical Introduction

102.1 The Three Components

```
AR(p):  Xₜ = c + φ₁Xₜ₋₁ + ... + φₚXₜ₋ₚ + εₜ          (regress on its own past values)
I(d):   difference the series d times to achieve stationarity (Section 100)
MA(q):  Xₜ = μ + εₜ + θ₁εₜ₋₁ + ... + θ_qεₜ-_q          (regress on past forecast ERRORS)
```

```python
from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(ts.dropna(), order=(1,1,1))
fitted = model.fit()
print(fitted.summary().tables[1])
```

This section is deliberately introductory — order selection (choosing p, d, q via ACF/PACF patterns or automated search), diagnostic checking (residual autocorrelation should vanish, per Section 101), and forecasting mechanics belong to the dedicated Time-Series ML module this curriculum references. What matters here is recognizing ARIMA as a direct, natural extension of the AR/MA statistical vocabulary just introduced, not an unrelated new technique.

---

103. Survival Statistics

103.1 Time-to-Event Data and Censoring

Censoring occurs when the event of interest hasn't happened yet by the end of observation — a customer who hasn't churned by the data cutoff isn't "never going to churn," their true time-to-event is simply unknown (right-censored). Ignoring censoring (Section 18's survivorship-bias mechanism, revisited here formally) badly biases naive average-tenure calculations.

```python
from lifelines import KaplanMeierFitter

n_customers = 500
true_lifetimes = rng.exponential(scale=24, size=n_customers)
observation_cutoff = 18  # months
observed_time = np.minimum(true_lifetimes, observation_cutoff)
event_observed = (true_lifetimes <= observation_cutoff).astype(int)
# 1=churned, 0=censored

kmf = KaplanMeierFitter()
kmf.fit(observed_time, event_observed)
```

```
kmf.plot_survival_function()
print(f"Median survival time: {kmf.median_survival_time_:.1f} months")
```

103.2 Survival Function and Hazard

```
S(t) = P(T > t)                    "probability of surviving beyond
time t"
h(t) = lim_{Δt→0} P(t≤T<t+Δt | T≥t)/Δt      "instantaneous risk of the
event, given survival so far"
```

The Kaplan-Meier estimator correctly handles censored observations by only "using" each customer's information up to the point they either churn or are censored — a naive complete-case or mean-imputation approach would either drop censored customers (survivorship bias, Section 18) or incorrectly treat "still active" as "churned right now," both badly wrong. **Applications:** churn analysis, healthcare (time to disease progression/death), equipment reliability (time to failure) — anywhere the outcome is fundamentally "how long until X happens," with some units not yet having experienced X by the analysis date.

---

104. Statistics for Classification

104.1 Reframing Module 1's Metrics Statistically

Metric	Statistical Interpretation
Confusion matrix	A 2×2 (or k×k) contingency table (Section 40.2's exact structure)
Precision	P(actually positive | predicted positive) — a conditional probability (Section 56)
Recall	P(predicted positive | actually positive) — the reverse conditional
ROC curve	Traces (1−specificity, sensitivity) across every possible decision threshold
PR curve	Traces (recall, precision) across every threshold — preferred under class imbalance (Module 1, Section 28)

104.2 Calibration

```
from sklearn.calibration import calibration_curve

model_probs = np.clip(rng.beta(2, 5, 1000) + 0.1*churn[:1000] if
len(churn)>=1000 else rng.beta(2,5,len(churn)), 0, 1)
frac_pos, mean_pred = calibration_curve(churn[:len(model_probs)],
model_probs, n_bins=10)

plt.plot(mean_pred, frac_pos, marker='o', label="Model")
plt.plot([0,1],[0,1], linestyle='--', label="Perfectly calibrated")
plt.xlabel("Mean predicted probability"); plt.ylabel("Fraction of
```

```
actual positives")
plt.legend()
```

Calibration asks: of all cases where the model predicted 70% probability, do approximately 70% actually turn out positive? A model can have excellent ranking ability (high AUC) while being **badly calibrated** (systematically over- or under-confident) — a distinction with real consequences whenever predicted probabilities feed directly into an expected-value business calculation (Module 5, Section 59.2's fraud-loss example), where miscalibration directly corrupts the downstream monetary estimate even if the model ranks risk correctly.

---

105. Statistics for Regression

105.1 Error Metrics, Statistically Grounded

Metric	Statistical Basis
MAE	MLE under Laplace-distributed errors (Section 91.1); estimates the conditional median
MSE	MLE under Gaussian errors; estimates the conditional mean
RMSE	$\sqrt{\text{MSE}}$, in original units
R^2	Fraction of variance explained (Section 58)

105.2 Prediction Intervals

```
# A prediction interval for a NEW individual observation (wider than a
confidence interval
# for the MEAN prediction, because it must account for irreducible
individual-level noise too)
predictions = model.get_prediction(X_sm)
pred_summary = predictions.summary_frame(alpha=0.05)
print(pred_summary[["mean", "mean_ci_lower", "mean_ci_upper",
                    "obs_ci_lower", "obs_ci_upper"]].head())
```

Confidence interval vs prediction interval — a critical, frequently conflated distinction: a confidence interval for the *mean* prediction at a given X reflects only parameter uncertainty and shrinks toward zero as $n \to \infty$; a **prediction interval** for an *individual new observation* must additionally include the irreducible residual noise σ^2 (Section 73.2) and therefore **never shrinks to zero**, no matter how much data you have — because individual outcomes are inherently noisier than the average.

---

106. Statistics for Imbalanced Data

106.1 Base Rates and Statistical Consequences

Building directly on Module 5's Section 57.3's medical-test base-rate argument, generalized:

```python
# Demonstrating why accuracy misleads under severe imbalance,
statistically
prevalence = 0.002
n = 100_000
y_true_imb = (rng.random(n) < prevalence).astype(int)
y_pred_always_negative = np.zeros(n)

accuracy = (y_pred_always_negative == y_true_imb).mean()
print(f"Accuracy of 'always predict negative': {accuracy:.4%}")   #
99.8%, yet useless
```

106.2 Techniques

Technique	Statistical Effect
Stratified sampling (train/test)	Ensures both splits reflect the true prevalence, preventing a degenerate split (Module 1, Section 18.4)
Resampling (oversampling minority / undersampling majority)	Changes the *training* class distribution — evaluation must still reflect the TRUE prevalence, never the resampled one
Class weights	Re-weights the loss function, statistically equivalent to resampling without literally duplicating/discarding rows
Precision-recall curves	Far more informative than ROC-AUC under severe imbalance, since ROC-AUC can look deceptively good (Module 1, Section 47.5 Q38)

A statistically essential warning: if you resample the *training* data, any metric computed against a similarly-resampled *test* set is measuring performance against an artificial world that doesn't exist in production — always evaluate against data reflecting the true, unresampled base rate.

---

107. Statistical Model Validation

107.1 The Validation Hierarchy

Method	What It Estimates	Statistical Basis
Single train/test split	A single point estimate of test performance	High variance — one draw from the sampling distribution (Section 19)
K-fold cross-validation	Mean and spread of performance across k resamples	Reduces variance of the estimate via averaging (Section 69)
Bootstrap	A full resampling-based distribution of performance	Most general; works for essentially any metric (Section 67)

Method	What It Estimates	Statistical Basis
Nested cross-validation	An *unbiased* estimate of performance when hyperparameter tuning itself is part of the pipeline	Prevents the "tuning leakage" where hyperparameters are optimized against the same data used to report final performance

107.2 Nested CV, Conceptually

```
OUTER loop: k-fold split → estimate GENERALIZATION performance
   INNER loop (within each outer training fold): further k-fold split
→ TUNE hyperparameters
```

Why nested CV matters: ordinary cross-validation used *both* to select hyperparameters *and* to report the final performance number is itself a (subtle, commonly overlooked) form of the multiple-testing/optimism problem from Section 46 — you've implicitly "tried" many hyperparameter configurations and reported the best one's CV score, which is optimistically biased exactly as Section 46's "test many things, report the best" pattern predicts. Nested CV's outer loop provides a genuinely held-out estimate untouched by the tuning process.

---

108. Uncertainty in Model Performance

```python
def bootstrap_metric_ci(y_true, y_pred_proba, metric_fn, n_boot=2000,
seed=0):
    rng_b = np.random.default_rng(seed)
    n = len(y_true)
    scores = np.empty(n_boot)
    for i in range(n_boot):
        idx = rng_b.integers(0, n, n)
        scores[i] = metric_fn(y_true[idx], y_pred_proba[idx])
    return np.percentile(scores, [2.5, 97.5]), scores

y_true_sample = churn
y_proba_sample = np.clip(rng.beta(2,4,len(churn)) + 0.2*churn, 0, 1)

ci_auc, boot_scores = bootstrap_metric_ci(y_true_sample,
y_proba_sample, roc_auc_score)
print(f"AUC: {roc_auc_score(y_true_sample, y_proba_sample):.4f}")
print(f"Bootstrap 95% CI: ({ci_auc[0]:.4f}, {ci_auc[1]:.4f})")
```

Reporting a single-number metric (AUC=0.83) without an uncertainty band is analogous to reporting a survey result without a margin of error — the point estimate alone hides how much that number might shift on a different sample of the same size. This applies identically to accuracy, precision, recall, F1, RMSE, and MAE — the bootstrap procedure above generalizes to every one of them with only the `metric_fn` argument changed.

---

109. Model Comparison

109.1 Paired Comparison, Statistically

```
model_a_scores = np.array([0.81, 0.83, 0.79, 0.85, 0.82])   # 5-fold
CV AUCs, model A
model_b_scores = np.array([0.84, 0.85, 0.82, 0.87, 0.85])   # 5-fold
CV AUCs, model B

# Paired t-test — the SAME folds were used for both models, so pairing
is appropriate
t_stat, p_val = stats.ttest_rel(model_b_scores, model_a_scores)
print(f"Paired t-test: t={t_stat:.3f}, p={p_val:.4f}")

diff = model_b_scores - model_a_scores
cohens_d_models = diff.mean() / diff.std(ddof=1)
print(f"Mean improvement: {diff.mean():.4f}, Cohen's d:
{cohens_d_models:.3f}")
```

Why paired, not independent-samples: the same cross-validation folds were used to evaluate both models, so their fold-level scores are naturally paired (Section 29.3) — using an independent-samples test here would ignore this structure and understate statistical power.

109.2 Why a Tiny Metric Improvement May Not Be Meaningful

An AUC improvement from 0.830 to 0.834 across 5 folds may not survive even this paired test, and even if it does, may fail to represent a **practically** meaningful improvement in the downstream business decision it drives (Section 121) — the exact same statistical-vs-practical-significance discipline from Section 33.3, now applied specifically to the very common real-world question "should we ship model B instead of model A?"

---

110. Data Drift From a Statistical Perspective

110.1 The Core Statistical Tools

Tool	Measures
Population Stability Index (PSI)	Binned distributional shift between a reference and current period
KL Divergence	Asymmetric measure of how one distribution diverges from a reference
Jensen-Shannon (JS) Divergence	A symmetric, bounded variant of KL divergence
Kolmogorov-Smirnov test	A formal hypothesis test for distributional difference (Section 83)

```
def psi(reference, current, bins=10):
    breakpoints = np.percentile(reference, np.linspace(0, 100,
bins+1))
    breakpoints[0], breakpoints[-1] = -np.inf, np.inf
```

```
    ref_pct = np.histogram(reference, breakpoints)[0] / len(reference)
    cur_pct = np.histogram(current, breakpoints)[0] / len(current)
    ref_pct, cur_pct = np.clip(ref_pct, 1e-6, None), np.clip(cur_pct,
1e-6, None)
    return np.sum((cur_pct - ref_pct) * np.log(cur_pct / ref_pct))

reference_period = rng.normal(500, 100, 5000)
current_period_stable = rng.normal(505, 102, 5000)
current_period_shifted = rng.normal(650, 140, 5000)

print(f"PSI (stable):  {psi(reference_period,
current_period_stable):.4f}")   # small
print(f"PSI (shifted): {psi(reference_period,
current_period_shifted):.4f}")  # large
```

PSI Value	Interpretation
< 0.1	No significant shift
0.1 – 0.25	Moderate shift — investigate
> 0.25	Major shift — likely requires model retraining

110.2 The Large-Sample Trap, Once More

Applying a formal K-S or chi-square test to **millions** of monitoring records will flag statistically significant "drift" for even the most trivial, operationally meaningless distributional wobble — precisely Section 83.1's warning, now in a production-monitoring context. **PSI and effect-size-style thresholds (not raw p-values) are the professional standard for production drift monitoring**, specifically because they were designed to be robust to this exact large-sample-test pathology.

---

111. Statistical Process Control

111.1 Control Charts

```
process_data = rng.normal(100, 5, 200)
process_data[150:155] += 20   # inject a special-cause event

mean_val, std_val = process_data[:100].mean(),
process_data[:100].std(ddof=1)   # baseline period
ucl, lcl = mean_val + 3*std_val, mean_val - 3*std_val

plt.plot(process_data)
plt.axhline(mean_val, color='green', label='Process mean')
plt.axhline(ucl, color='red', linestyle='--', label='UCL (+3σ)')
plt.axhline(lcl, color='red', linestyle='--', label='LCL (−3σ)')
plt.legend()
```

```
out_of_control = np.where((process_data > ucl) | (process_data < lcl))
[0]
print(f"Out-of-control points at indices: {out_of_control}")
```

Term	Meaning
UCL/LCL	Upper/Lower Control Limit, conventionally mean ± 3σ
Common-cause variation	Normal, expected process noise — within control limits
Special-cause variation	An unusual, investigable deviation — beyond control limits

111.2 Applications Beyond Manufacturing

Manufacturing (the classical origin), **data pipelines** (daily row counts, null rates), **ML monitoring** (Section 110's drift metrics plotted over time with control limits), **business operations** (daily transaction volume, average order value) — anywhere a metric should behave predictably day-to-day, and a genuine deviation warrants investigation rather than being drowned in routine noise. The ±3σ convention is precisely the same 99.7% rule from Module 5's Section 63.3, applied as an operational threshold rather than a probability statement.

---

112. Quality Control and ML

Statistics supports the full ML operational lifecycle: **data quality** (schema/range checks as hypothesis-test-like assertions), **pipeline monitoring** (control charts on row counts and null rates, Section 111), **model monitoring** (drift detection, Section 110; calibration tracking, Section 104.2), **business KPI monitoring** (control charts on the metrics the model ultimately drives), and **anomaly detection** (statistical outlier methods, Sections 9, 57, 95.2, applied to live incoming data rather than a static training set). This section is deliberately a bridge — Section 140 builds a complete, worked production monitoring example using exactly this toolkit.

113. Statistics with Python — The Complete Toolkit

Library	Role	When to Reach For It
NumPy	Numerical arrays, basic statistics	Foundational computation
Pandas	Tabular data, groupby-based descriptive stats	Exploratory data analysis (Section 114)
SciPy (`scipy.stats`)	Hypothesis tests, distributions	Formal test statistics and p-values (Section 115)
statsmodels	Statistical modeling with full inference	Regression/GLM with coefficients, SEs, p-values, diagnostics (Section 116)
scikit-learn	ML-oriented statistical tools	Cross-validation, feature selection, calibration (Section 117)
Matplotlib/Seaborn	Visualization	Every diagnostic plot in this module (Section 118)
Plotly	Interactive visualization	Dashboards, exploratory drill-down

The professional division of labor: use Pandas for fast descriptive exploration, SciPy for a specific named hypothesis test, statsmodels when you need the full inferential apparatus (p-values, CIs, diagnostics) around a fitted model, and scikit-learn when the model's primary purpose is prediction rather than inference — mirroring Section 50.2's statistical-vs-predictive framing directly in tool choice.

---

114. Descriptive Statistics with Pandas

```
sales_df = pd.DataFrame({
    "region": rng.choice(["North","South","East","West"], 1000),
    "revenue": rng.lognormal(10, 0.6, 1000),
    "units": rng.poisson(15, 1000),
})

print(sales_df.describe())
print(sales_df["revenue"].mean(), sales_df["revenue"].median(),
sales_df["revenue"].mode()[0])
print(sales_df["revenue"].std(), sales_df["revenue"].var())
print(sales_df["revenue"].quantile([0.1, 0.25, 0.5, 0.75, 0.9]))
print(sales_df["revenue"].skew(), sales_df["revenue"].kurt())
print(sales_df[["revenue","units"]].corr())
print(sales_df[["revenue","units"]].cov())
print(sales_df["region"].value_counts())

# Grouped descriptive statistics — the everyday workhorse
```

```
print(sales_df.groupby("region")
["revenue"].agg(["mean","median","std","count"]))
```

`describe()` gives count, mean, std, min, quartiles, and max in one call — the fastest possible first look at any new numeric column, and the natural starting point of Section 119's professional workflow.

---

115. SciPy Statistics — Reference

Function	Test	Section
`ttest_1samp`	One-sample t-test	36
`ttest_ind`	Two-sample t-test (independent)	37
`ttest_rel`	Paired t-test	38
`chi2_contingency`	Chi-square independence test	40
`mannwhitneyu`	Mann-Whitney U	47
`wilcoxon`	Wilcoxon signed-rank	47
`kruskal`	Kruskal-Wallis	47
`pearsonr`	Pearson correlation + p-value	48
`spearmanr`	Spearman correlation + p-value	48
`kendalltau`	Kendall's tau + p-value	48
`shapiro`	Shapiro-Wilk normality test	83
`normaltest`	D'Agostino-Pearson normality test	83
`kstest`	Kolmogorov-Smirnov test	83
`levene`	Test of equal variances	37
`f_oneway`	One-way ANOVA	42

```
# Every scipy.stats test returns a consistent (statistic, p-value)
tuple —
# a deliberate API design worth relying on
stat, p = stats.shapiro(sales_df["revenue"].sample(500,
random_state=0))
print(f"statistic={stat:.4f}, p={p:.4f}")
```

---

116. statsmodels

116.1 The Formula Interface

```
import statsmodels.formula.api as smf

model_formula = smf.ols("revenue ~ units + C(region)",
data=sales_df).fit()
print(model_formula.summary())
```

The R-style formula interface (`"y ~ x1 + C(x2)"`, with `C()` marking categorical variables for automatic dummy-encoding) is often faster to iterate with than manually constructing a design matrix, and is the idiomatic statsmodels entry point for applied statistical modeling.

116.2 statsmodels vs scikit-learn — The Decisive Difference, Restated

	statsmodels	scikit-learn
Primary goal	Statistical **inference**	Predictive **performance**
Output	Coefficients, SEs, p-values, CIs, diagnostics	Predictions, a fitted estimator object
API style	Formula-based or explicit design matrices, `.summary()`	Consistent `.fit()`/`.predict()` across all models
Typical use	"Is this effect real, and how large?"	"How accurately can I forecast new cases?"

Both call the identical underlying optimization (Module 5's Sections 74-75); the choice is about **what question you're asking of the fitted model**, not about which one is "more correct."

---

117. Scikit-learn Statistical Tools

```
from sklearn.model_selection import train_test_split, cross_val_score,
permutation_test_score
from sklearn.calibration import CalibratedClassifierCV
from sklearn.preprocessing import StandardScaler

X_sk, y_sk = X_features.values, churn

# Standard split
X_train, X_test, y_train, y_test = train_test_split(X_sk, y_sk,
test_size=0.2,
                                                    stratify=y_sk,
random_state=0)

# Cross-validated score (Section 69)
clf = LogisticRegression()
cv_scores = cross_val_score(clf, X_sk, y_sk, cv=5, scoring="roc_auc")
print(f"CV AUC: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")
```

```
# Permutation test — is the model's performance better than chance,
formally tested?
score, perm_scores, p_value = permutation_test_score(clf, X_sk, y_sk,
cv=5,

n_permutations=100, random_state=0)
print(f"True score: {score:.4f}, p-value vs random-label baseline:
{p_value:.4f}")

# Probability calibration (Section 104.2)
calibrated_clf = CalibratedClassifierCV(clf, method='isotonic',
cv=5).fit(X_train, y_train)
```

`permutation_test_score` is a genuinely elegant application of the hypothesis-testing framework (Section 31) to model validation: it repeatedly shuffles the labels, refits, and rescores, building an empirical null distribution of "performance you'd see with no real signal" — the resulting p-value directly answers "is my model's performance distinguishable from a model trained on pure noise," a formal statistical test for the informal question every ML practitioner implicitly asks.

---

118. Statistical Visualization

Plot	Shows	Section
Histogram / density	Distribution shape	11
ECDF	Cumulative distribution, assumption-free	12
Boxplot / violin plot	Central tendency + spread + outliers (violin adds density shape)	10
Error bars	Point estimate ± uncertainty	26
Scatter plot	Bivariate relationship	48
Correlation heatmap	Multivariate relationships at a glance	49
Residual plots	Regression assumption diagnostics	54
Q-Q plot	Distributional fit	82

```
fig, axes = plt.subplots(2, 3, figsize=(16, 8))

sns.histplot(sales_df["revenue"], kde=True, ax=axes[0,0])
sns.violinplot(data=sales_df, x="region", y="revenue", ax=axes[0,1])

group_means = sales_df.groupby("region")["revenue"].agg(["mean",
"sem"])
axes[0,2].errorbar(group_means.index, group_means["mean"],
```

```
yerr=1.96*group_means["sem"],
                    fmt='o', capsize=5)
axes[0,2].set_title("Mean revenue ± 95% CI by region")

sns.scatterplot(data=sales_df, x="units", y="revenue", ax=axes[1,0])
sns.heatmap(sales_df[["revenue","units"]].corr(), annot=True,
ax=axes[1,1])
stats.probplot(sales_df["revenue"], dist="norm", plot=axes[1,2])
plt.tight_layout()
```

Visualization supports statistical reasoning by making assumption violations, effect sizes, and uncertainty visible at a glance — a well-designed error-bar plot communicates both the estimate and its precision in one image, doing the work of a paragraph of numbers, and is frequently more persuasive (and more honest) in a business presentation than a table of p-values alone.

---

119. Statistical Analysis Workflow

```
1.  Define the question           — precisely, in advance, before
touching data
2.  Identify the population        — who/what are we generalizing to?
3.  Identify the sample             — what data do we actually have?
4.  Understand the data             — types, sources, collection
method (Sections 3-5)
5.  Explore                          — descriptive stats,
visualization (Sections 6-14, 118)
6.  Identify assumptions            — which method's assumptions
apply? (Section 66)
7.  Select method                    — guided by Section 152's test-
selection tree
8.  Conduct analysis                  — compute the test
statistic/model
9.  Quantify uncertainty               — CI, SE, or bootstrap
(Sections 26-30, 67)
10. Interpret effect                    — magnitude, not just
significance (Section 45)
11. Validate robustness                  — sensitivity checks,
nonparametric cross-check
12. Communicate results                   — Section 120's reporting
standard
13. Make a decision                        — statistical + practical
+ business context together
```

This workflow is the connective spine of the entire module — every section from 1 to 118 is, in effect, a deep-dive into one step of this sequence, and Sections 124-130's end-to-end projects are simply this workflow executed start-to-finish on realistic data.

---

120. Statistical Reporting

120.1 Bad vs Professional Reporting

BAD: *"The new feature increased engagement significantly (p<0.05)."*

PROFESSIONAL: *"Users exposed to the new feature showed a mean engagement increase of 4.2 minutes/session (95% CI: 2.1–6.3, n=1,240 per group), a statistically significant difference (Welch's t=3.8, p=0.0002) with a medium effect size (Cohen's d=0.48). This corresponds to an estimated 8% relative lift over the 52-minute control baseline."*

120.2 The Complete Reporting Checklist

Element	Why It's Required
Sample size	Contextualizes both power and precision
Point estimate (mean, difference, etc.)	The actual finding
Measure of spread (SD)	Describes the raw data's variability
Confidence interval	Honestly communicates estimation uncertainty (Section 26)
Test statistic and exact p-value	Not just "p<0.05" — report the actual value (unless below a reporting floor like p<0.001)
Effect size	Magnitude, in interpretable units (Section 45)
Method used, and why	Transparency, reproducibility

The standing rule: every number reported alone, with no accompanying uncertainty or effect-size context, is an invitation to overinterpretation. This checklist is what Section 154's capstone report module is built to automate for a real analytics platform.

---

121. Statistical Significance vs Practical Significance

121.1 The Two-by-Two Reality

	Statistically Significant	Not Statistically Significant
Practically meaningful effect	Ideal outcome — act on it	Likely underpowered (Section 35) — consider more data before concluding "no effect"
Practically trivial effect	Huge sample, tiny real-world effect (Section 33.3) — usually not worth acting on	Genuinely no meaningful effect (or insufficient data to say either way)

```
# Case 1: tiny effect, huge sample -> statistically significant, practically meaningless
tiny_effect_a = rng.normal(100.0, 15, 500_000)
tiny_effect_b = rng.normal(100.1, 15, 500_000)
_, p1 = stats.ttest_ind(tiny_effect_a, tiny_effect_b)
```

```
d1 = (tiny_effect_b.mean()-tiny_effect_a.mean())/tiny_effect_a.std()
print(f"Case 1 — p={p1:.5f}, Cohen's d={d1:.4f}  (significant, but d
is negligible)")

# Case 2: large effect, small sample -> not significant, but
potentially real and worth more data
large_effect_a = rng.normal(100, 15, 12)
large_effect_b = rng.normal(115, 15, 12)
_, p2 = stats.ttest_ind(large_effect_a, large_effect_b)
d2 = (large_effect_b.mean()-
large_effect_a.mean())/large_effect_a.std()
print(f"Case 2 — p={p2:.4f}, Cohen's d={d2:.4f}  (not significant, but
d suggests a real effect worth more data)")
```

Business decisions require the full picture — effect size, confidence interval, cost of acting vs not acting, and statistical significance together — no single number from this collection is sufficient alone, and the discipline of demanding all of them together is the single most protective habit this entire module can instill.

122. Common Statistical Fallacies

A representative, high-value set spanning the most consequential and recurring errors — each with the mechanism explained, not just named.

1. Correlation implies causation. Two variables moving together doesn't mean one causes the other (Section 74). **2. P-value misunderstanding.** Treating p as "probability H_0 is true" (Section 33.2). **3. Multiple testing.** Running many tests and reporting only the significant ones inflates the true false-positive rate (Section 46). **4. P-hacking.** Trying many analyses/subgroups/cutoffs until one reaches $p<0.05$, then reporting only that one. **5. HARKing** (Hypothesizing After Results are Known). Presenting a post-hoc finding as if it were the pre-registered hypothesis. **6. Cherry-picking.** Selectively reporting favorable results while omitting unfavorable ones. **7. Survivorship bias.** Analyzing only surviving/successful units, missing the failures that would change the conclusion (Section 18). **8. Selection bias.** The sample's composition, not the true effect, drives the finding. **9. Simpson's paradox.** An aggregate trend reverses when the data is properly disaggregated (Section 75). **10. Base-rate neglect.** Ignoring the prior/prevalence when interpreting conditional evidence (Section 57.3). **11. Regression to the mean.** An extreme observation tends to be followed by a less extreme one, purely due to natural variability — mistaken for a real "effect" of an intervention applied after the extreme observation. **12. Confusing SD and SE.** Reporting SE when describing individual variability, or SD when describing an estimate's precision (Section 16.2). **13. Misusing confidence intervals.** Claiming "95% probability the parameter is in this interval" under a frequentist framework (Section 26.2). **14. Overinterpreting R^2.** Treating a high R^2 as proof of a good, causally correct, or even linear-appropriate model (Section 58.2). **15. Treating significance as importance.** Ignoring effect size entirely (Section 33.3). **16. Ignoring assumptions.** Running a t-test on badly non-normal, small-sample, outlier-laden data without checking (Section 66). **17. Data leakage.** Any pipeline step (imputation, scaling, feature selection) fit on data that includes the test set. **18. Post-treatment bias.** Controlling for a variable that is itself affected by the treatment (a mediator, Section 76). **19. Small-sample overconfidence.** Trusting a striking result from n=8 as if it were as reliable as n=8,000. **20. The "file drawer" problem.** Null results go unpublished/unreported, biasing the visible literature/analysis history toward positive findings.

21. Confusing statistical and practical significance (Section 121). **22. Extrapolation beyond the data's range.** Trusting a regression's predictions far outside the observed X range. **23. Ecological fallacy.** Inferring individual-level relationships from aggregate/group-level data. **24. Texas sharpshooter fallacy.** Drawing the target (hypothesis) after seeing where the shots (data) landed. **25. Overfitting mistaken for insight.** A model perfectly explaining training data is treated as revealing a "true" pattern (Module 1, Section 20). **26. Non-response bias ignored.** Assuming survey non-responders resemble responders. **27. Treating an odds ratio as a risk ratio** (Section 62.2). **28. Ignoring censoring in survival data,** producing biased average-tenure estimates (Section 103). **29. Assuming independence when data is clustered/panel.** Standard errors are then badly understated. **30. Using Pearson correlation on non-linear monotonic data,** understating the true association (Section 48.1). **31. Believing "the data speaks for itself"** without acknowledging analyst/methodological choices shape every conclusion. **32. Mistaking a confidence interval's width for the effect's importance** — a narrow CI around a trivial effect is still trivial. **33. Assuming a larger**

sample always fixes bias — it only reduces variance; bias (Section 18) persists at any sample size. **34. Comparing p-values across studies with different sample sizes** as if they were directly comparable measures of effect strength. **35. Assuming statistical independence of repeated A/B test peeks** (Section 71.4). **36. Treating an unadjusted subgroup finding from an underpowered trial as reliable. 37. Ignoring the multiple-comparisons problem in dashboard "significance" flags** that test dozens of metrics daily. **38. Believing removing all outliers always improves a model** — sometimes the "outlier" is the signal (Section 57). **39. Using accuracy as the primary metric under severe class imbalance** (Section 106.1). **40. Assuming a well-calibrated model is automatically also a well-discriminating one, or vice versa** (Section 104.2 — these are genuinely separate properties).

41. Believing a randomized experiment eliminates all bias — it eliminates confounding bias specifically, not measurement error, attrition bias, or Hawthorne effects. **42. Conflating "fail to reject H_0" with "H_0 is proven true"** (Section 31.2). **43. Using a one-sided test decided after seeing the data's direction** (Section 32). **44. Not adjusting standard errors for heteroscedasticity** before trusting a regression's p-values (Section 55). **45. Ignoring multicollinearity's effect on individual coefficient p-values** while still trusting the overall model fit (Section 56). **46. Using AIC/BIC to compare models fit on different data or different targets** (Section 60.2). **47. Believing bootstrap resampling fixes a biased original sample** — it characterizes uncertainty *given* the sample, not sampling bias itself. **48. Assuming stationarity without testing it** before applying classical time-series methods (Section 100). **49. Treating an in-sample time-series fit as evidence of forecasting skill,** ignoring the mandatory time-based validation split. **50. Applying a hypothesis test naively to a massive production-monitoring dataset,** flagging trivial drift as a crisis (Section 110.2).

51. Believing "statistically significant" model improvement always justifies deployment cost (Section 109.2). **52. Assuming a feature's high mutual information with the target guarantees causal relevance** for an intervention decision. **53. Interpreting a Bayesian credible interval and a frequentist confidence interval as interchangeable** (Section 78). **54. Using an uninformative flat prior and believing the result is therefore "assumption-free"** — MLE itself is a special case of Bayesian inference with a flat prior, carrying its own implicit assumptions. **55. Ignoring the difference between a mediator and a confounder when deciding what to control for** (Section 76.1). **56. Conditioning on a collider,** inadvertently manufacturing a spurious association (Section 74.2). **57. Believing that because two groups differ significantly on one metric, they must differ on all related metrics. 58. Assuming a well-fitting distribution (via AIC) is the "true" generating process,** rather than simply the best fit among candidates tried (Section 81). **59. Dismissing a nonparametric test's result as "less rigorous"** than a parametric one, when the nonparametric test may be the more *appropriate* (not lesser) choice given the data (Section 47.3). **60. Assuming statistical models trained on historical data automatically remain valid as the underlying population or process evolves** — the entire justification for Sections 110-112's ongoing monitoring discipline, and a fitting closing fallacy, since believing "the model was validated once, so it's fine forever" undermines nearly everything this module has built toward.

---

123. Statistical Troubleshooting

Format: Symptom → Diagnosis → Remedy.

1. P-value is NaN. Diagnosis: zero variance in a group, or a sample size of 1, or all-identical values. Remedy: check group sizes and variance before testing; guard against degenerate inputs. **2. Confidence interval is extremely wide.** Diagnosis: small n, high variance, or a near-zero denominator (rare-event proportion). Remedy: collect more data; consider Wilson interval (Section 28.2) for proportions; report the width honestly rather than suppressing it. **3. Standard error unexpectedly large.** Diagnosis: high underlying variance, small n, or a mistaken SD-instead-of-SE computation (Section 16.2). Remedy: verify the formula used; check for outliers inflating variance. **4. Regression residuals are non-normal.** Diagnosis: check via Q-Q plot (Section 82)/Shapiro-Wilk (Section 83). Remedy: transform Y (log, Box-Cox), add missing non-linear terms, or use robust/bootstrap inference. **5. Variance is unequal across groups.** Diagnosis: Levene's test flags it (Section 37.1). Remedy: use Welch's t-test/ANOVA instead of the pooled-variance version. **6. Correlation is misleading.** Diagnosis: check for non-linearity (Section 48), outliers, or a lurking confounder (Section 74). Remedy: plot the scatter first, always; compute Spearman alongside Pearson. **7. Model coefficients are unstable across resamples.** Diagnosis: multicollinearity (Section 56) or too few observations relative to predictors. Remedy: check VIF; apply Ridge regularization; gather more data or reduce predictors.

8. Multicollinearity is high. Diagnosis: VIF > 5-10. Remedy: drop/combine correlated features, or use Ridge/PCA. **9. ANOVA assumptions fail.** Diagnosis: non-normal residuals or unequal variances flagged by diagnostics. Remedy: Welch's ANOVA (unequal variance) or Kruskal-Wallis (non-normal, Section 47). **10. Chi-square expected counts too small.** Diagnosis: any expected cell count <5 (Section 40.3). Remedy: combine sparse categories, or use Fisher's exact test for 2×2 tables. **11. Sample is biased.** Diagnosis: compare sample demographics/characteristics to known population figures; investigate the collection mechanism (Section 18). Remedy: reweight, or explicitly narrow the claimed generalizability to the accessible population. **12. Statistical significance disappears after multiple-testing correction.** Diagnosis: the original finding may have been one of many "significant-looking" results expected by chance alone (Section 46.1). Remedy: treat the corrected result as authoritative; consider it a candidate for a dedicated confirmatory follow-up study, not further mining of the same dataset. **13. Model metric changes drastically across CV folds.** Diagnosis: high model variance, small dataset, or a non-stratified split on imbalanced data. Remedy: use stratified/repeated CV (Section 69.2); consider a simpler, lower-variance model; report the full fold-level spread, not just the mean.

14. A/B test shows sample ratio mismatch. Diagnosis: chi-square SRM check fails (Section 72.2). Remedy: audit the randomization/logging pipeline before trusting any result from that experiment. **15. Bootstrap CI is asymmetric or oddly shaped.** Diagnosis: this is often *correct* behavior reflecting a skewed sampling distribution (e.g. for the median or a ratio statistic), not a bug. Remedy: use a BCa bootstrap for improved accuracy (Section 67.3); avoid forcing a symmetric interval where none is warranted. **16. Two "equivalent" tests (t-test vs Mann-Whitney) give conflicting conclusions.** Diagnosis: likely driven by outliers or non-normality distorting the parametric result. Remedy: investigate the raw data distribution directly; trust the nonparametric result when normality is clearly violated. **17. A model's calibration curve**

is far from the diagonal. Diagnosis: the model is over/under-confident systematically (Section 104.2). Remedy: apply Platt scaling or isotonic regression calibration; re-evaluate whether predicted probabilities are safe to use in downstream expected-value calculations. **18. PSI/drift metric spikes but the model's live performance looks unchanged.** Diagnosis: the drifted feature(s) may not be influential to the model's predictions. Remedy: check per-feature importance-weighted drift, not only raw distributional shift; confirm with an actual live-performance metric before triggering retraining. **19. ADF and KPSS tests disagree on stationarity.** Diagnosis: ambiguous evidence, possibly a structural break rather than a genuine unit root (Section 100.2). Remedy: visually inspect for a level/trend shift; consider testing subperiods separately.

20. Effect size is large but the confidence interval includes zero. Diagnosis: classic underpowered-study signature (Section 35). Remedy: report both together honestly; recommend a properly powered follow-up rather than treating the point estimate alone as conclusive. **21. A regression's F-statistic is significant but no individual coefficient is.** Diagnosis: severe multicollinearity is diffusing significance across correlated predictors (Section 56). Remedy: check VIF; consider a joint test on the correlated group rather than individual coefficient tests. **22. Imputed variable's variance collapsed.** Diagnosis: mean/median imputation used, artificially shrinking spread (Section 86). Remedy: use multiple imputation, or at minimum add a missingness indicator feature alongside the imputed value. **23. Feature selected on the full dataset performs worse in true holdout.** Diagnosis: selection leakage (Section 93.2). Remedy: refit feature selection within each CV fold, never on the combined train+test data. **24. A survival analysis shows implausibly short median survival.** Diagnosis: censoring likely mishandled (treated as an event rather than as censored) (Section 103). Remedy: verify the event indicator coding; recheck with a proper Kaplan-Meier fit. **25. Logistic regression fails to converge.** Diagnosis: perfect/near-perfect separation (a predictor perfectly predicts the outcome) or severe multicollinearity. Remedy: apply regularization (Ridge-penalized logistic regression), or investigate and potentially remove the perfectly-separating predictor.

26. Q-Q plot shows a curved (not straight) pattern. Diagnosis: skewness present (Section 82). Remedy: consider a transformation, or switch to a distribution/model better suited to the observed shape. **27. Bonferroni correction seems overly conservative, killing all findings.** Diagnosis: Bonferroni is the most conservative correction; with many tests it can produce very low power. Remedy: consider Benjamini-Hochberg (FDR control, Section 46.2) for large-scale testing scenarios. **28. Two analysts get different p-values from "the same" test on the same data.** Diagnosis: differing default arguments (paired vs unpaired, one- vs two-sided, equal-variance assumption). Remedy: always explicitly state and verify every test argument; never rely on library defaults silently. **29. A time-series model's residuals show remaining autocorrelation.** Diagnosis: the model hasn't captured all the temporal structure (Section 101). Remedy: examine the residual ACF/PACF; add appropriate AR/MA terms or seasonal components. **30. Bootstrap and analytical CIs disagree substantially.** Diagnosis: the analytical CI's underlying distributional assumption (e.g. normality) is likely violated. Remedy: trust the bootstrap CI in this case, and use the discrepancy itself as a signal to investigate the raw data's shape.

31. A/B test's confidence interval for the lift crosses zero, but the team wants to ship anyway. Diagnosis: this is a business-risk decision, not purely statistical — the CI genuinely

doesn't rule out a null or negative effect. Remedy: present the actual uncertainty explicitly; let the decision-maker weigh the cost of being wrong against the cost of delay, rather than silently treating "no significant difference" as "no difference." **32. Permutation test p-value doesn't match the parametric test's p-value.** Diagnosis: usually reflects the parametric test's assumption violation being exposed by the assumption-free permutation approach. Remedy: prefer the permutation result when they diverge meaningfully and assumptions are in doubt. **33. Cook's distance flags a large fraction of the dataset as "influential."** Diagnosis: the threshold rule-of-thumb may be miscalibrated for this dataset's size, or a genuine, pervasive model misspecification exists. Remedy: examine the overall residual pattern first (Section 54) rather than chasing individually flagged points. **34. Same statistical test gives different results in R and Python.** Diagnosis: differing default degrees-of-freedom corrections, tie-handling in rank-based tests, or continuity corrections. Remedy: explicitly match all optional arguments between implementations before comparing. **35. A model's cross-validated performance is excellent, but production performance is much worse.** Diagnosis: training-serving skew (Module 1, Section 35.2) or genuine data drift since training (Section 110). Remedy: compare current production feature distributions to training distributions directly; investigate the feature-computation pipeline for discrepancies between training and serving.

36. Confidence interval for a proportion includes values below 0% or above 100%. Diagnosis: the Wald interval's known failure mode for extreme p or small n (Section 28.2). Remedy: switch to the Wilson interval. **37. A published effect fails to replicate in a new dataset.** Diagnosis: could be a genuine false positive from the original study (multiple testing, small sample, Section 46), a true but smaller effect regressing toward the mean (Section 122's item 11), or a genuine change in the underlying population. Remedy: report the replication attempt transparently regardless of outcome; avoid assuming the new study is simply "wrong." **38. Winsorizing/trimming changes the conclusion of an analysis.** Diagnosis: the original conclusion was likely driven substantially by a small number of extreme values. Remedy: report both the raw and robust versions, and investigate whether the extreme values are genuine or data-quality artifacts (Section 57) before choosing which to present as primary. **39. Two "identical" bootstrap runs give different confidence intervals.** Diagnosis: no random seed was set (Module 1, Section 26.4's reproducibility discipline, directly applicable here). Remedy: always set and record a random seed for any resampling procedure used in a reported analysis. **40. A bootstrap distribution shows visible multimodality.** Diagnosis: this can genuinely reflect underlying subgroup structure in the data (e.g. two distinct customer segments), not merely a computational artifact. Remedy: investigate whether hidden subgroups should be analyzed separately rather than pooled into one overall estimate.

124. End-to-End Project 1: Customer Churn Statistical Analysis

```
rng_p = np.random.default_rng(10)
n = 2000
churn_df = pd.DataFrame({
    "customer_id": range(n),
    "tenure_months": rng_p.integers(1, 72, n),
    "monthly_spend": rng_p.normal(65, 20, n),
    "support_calls": rng_p.poisson(1.2, n),
    "contract_type": rng_p.choice(["Monthly","Annual"], n,
p=[0.6,0.4]),
})
logit_true = -1.2 - 0.04*churn_df.tenure_months +
0.5*churn_df.support_calls \
             + np.where(churn_df.contract_type=="Monthly", 0.8, -0.3)
churn_df["churned"] = (rng_p.random(n) < 1/(1+np.exp(-
logit_true))).astype(int)

# 1. Descriptive statistics
print(churn_df.groupby("churned")
[["tenure_months","monthly_spend","support_calls"]].describe().T)

# 2. Group comparison — tenure by churn status
churned_tenure = churn_df[churn_df.churned==1]["tenure_months"]
retained_tenure = churn_df[churn_df.churned==0]["tenure_months"]
t_stat, p_val = stats.ttest_ind(churned_tenure, retained_tenure,
equal_var=False)
d = (retained_tenure.mean()-
churned_tenure.mean())/churn_df["tenure_months"].std()
print(f"Tenure: t={t_stat:.2f}, p={p_val:.5f}, Cohen's d={d:.3f}")

# 3. Confidence interval on churn rate
p_hat = churn_df.churned.mean()
ci = proportion_confint(churn_df.churned.sum(), n, method='wilson')
print(f"Churn rate: {p_hat:.2%}, 95% CI: ({ci[0]:.2%}, {ci[1]:.2%})")

# 4. Association: contract type vs churn
ct = pd.crosstab(churn_df.contract_type, churn_df.churned)
chi2, p_chi, dof, _ = stats.chi2_contingency(ct)
print(f"Contract type vs churn: χ²={chi2:.2f}, p={p_chi:.5f}")

# 5. Logistic regression with full inference
X_churn =
sm.add_constant(pd.get_dummies(churn_df[["tenure_months","support_call
s","contract_type"]],

drop_first=True).astype(float))
logit_fit = sm.Logit(churn_df.churned, X_churn).fit(disp=0)
print(logit_fit.summary())
print("Odds ratios:\n", np.exp(logit_fit.params))
```

```
# 6. Diagnostics — VIF check
vif_df = pd.DataFrame({"feature": X_churn.columns,
                       "VIF":
[variance_inflation_factor(X_churn.values, i) for i in
range(X_churn.shape[1])]})
print(vif_df)
```

Interpretation summary: tenure and support calls both show statistically and practically meaningful associations with churn; contract type shows a significant chi-square association, corroborated by its logistic regression coefficient and odds ratio. All conclusions are *associational* — the data is observational (Section 74), so causal claims ("switching customers to annual contracts *causes* lower churn") would require either a randomized experiment (Section 70) or the causal-inference tools of Section 146.

---

125. End-to-End Project 2: Banking Fraud Statistical Analysis

```
n_txn = 50_000
fraud_rate = 0.006
txn_df = pd.DataFrame({
    "amount": rng_p.lognormal(4, 1.2, n_txn),
    "is_foreign": rng_p.choice([0,1], n_txn, p=[0.85,0.15]),
    "channel": rng_p.choice(["POS","Online","ATM"], n_txn,
p=[0.5,0.35,0.15]),
})
fraud_logit = -6 + 1.5*txn_df.is_foreign + 0.4*np.log(txn_df.amount) +
\
              np.where(txn_df.channel=="Online", 0.8, 0)
txn_df["is_fraud"] = (rng_p.random(n_txn) < 1/(1+np.exp(-
fraud_logit))).astype(int)
print(f"Observed fraud prevalence: {txn_df.is_fraud.mean():.3%}")

# Fraud vs amount — nonparametric, given fraud amounts are right-
skewed and rare
fraud_amt = txn_df[txn_df.is_fraud==1].amount
legit_amt = txn_df[txn_df.is_fraud==0].amount
u_stat, p_mw = stats.mannwhitneyu(fraud_amt, legit_amt)
print(f"Amount (fraud vs legit): Mann-Whitney p={p_mw:.5f}")
print(f"Median amount — fraud: {fraud_amt.median():.2f}, legit:
{legit_amt.median():.2f}")

# Fraud vs foreign — chi-square + effect size (odds ratio)
ct_foreign = pd.crosstab(txn_df.is_foreign, txn_df.is_fraud)
chi2f, pf, _, _ = stats.chi2_contingency(ct_foreign)
odds_foreign = (ct_foreign.loc[1,1]/ct_foreign.loc[1,0]) /
(ct_foreign.loc[0,1]/ct_foreign.loc[0,0])
print(f"Foreign vs fraud: χ²={chi2f:.2f}, p={pf:.6f}, odds
ratio={odds_foreign:.2f}")
```

```
# Fraud vs channel — chi-square
ct_channel = pd.crosstab(txn_df.channel, txn_df.is_fraud)
chi2c, pc, _, _ = stats.chi2_contingency(ct_channel)
print(f"Channel vs fraud: χ²={chi2c:.2f}, p={pc:.6f}")

# Logistic regression combining all signals
X_fraud = sm.add_constant(pd.get_dummies(
    txn_df.assign(log_amount=np.log(txn_df.amount))
[["log_amount","is_foreign","channel"]],
    drop_first=True).astype(float))
fraud_logit_fit = sm.Logit(txn_df.is_fraud, X_fraud).fit(disp=0)
print(fraud_logit_fit.summary())
```

Limitations of observational conclusions, stated explicitly: this analysis identifies *statistical association* between transaction characteristics and fraud — it cannot establish that, say, "being foreign" *causes* fraud in any mechanistic sense; it reflects that fraud tactics happen to correlate with certain patterns in this observed dataset, which is exactly the actionable signal a detection *model* needs, without requiring (or supporting) a causal narrative.

---

126. End-to-End Project 3: A/B Testing (Complete Production-Style)

```
# Design phase
baseline_conv, mde, alpha, target_power = 0.045, 0.008, 0.05, 0.80
effect_size = proportion_effectsize(baseline_conv+mde, baseline_conv)
n_per_group = NormalIndPower().solve_power(effect_size=effect_size,
alpha=alpha, power=target_power)
print(f"Required n per group: {n_per_group:.0f}")

# Simulated execution
n_run = int(n_per_group) + 500
assignment = rng_p.choice(["control","treatment"], n_run, p=[0.5,0.5])
true_rates = np.where(assignment=="control", baseline_conv,
baseline_conv+mde+0.003)
converted = rng_p.random(n_run) < true_rates

exp_df = pd.DataFrame({"assignment": assignment, "converted":
converted})

# Data validation: SRM check
n_c, n_t = (assignment=="control").sum(),
(assignment=="treatment").sum()
_, srm_p = stats.chisquare([n_c, n_t], [n_run/2, n_run/2])
print(f"SRM check p={srm_p:.4f} ({'PASS' if srm_p>0.01 else 'FAIL —
investigate'})")

# Primary analysis
ctrl_conv = exp_df[exp_df.assignment=="control"].converted
trt_conv = exp_df[exp_df.assignment=="treatment"].converted
count = np.array([trt_conv.sum(), ctrl_conv.sum()]); nobs =
```

```
np.array([len(trt_conv), len(ctrl_conv)])
z_stat, p_val = proportions_ztest(count, nobs)

diff = trt_conv.mean() - ctrl_conv.mean()
se_diff = np.sqrt(ctrl_conv.mean()*(1-ctrl_conv.mean())/len(ctrl_conv)
+
                  trt_conv.mean()*(1-trt_conv.mean())/len(trt_conv))
ci_diff = (diff - 1.96*se_diff, diff + 1.96*se_diff)

print(f"Control: {ctrl_conv.mean():.3%}  Treatment:
{trt_conv.mean():.3%}")
print(f"Absolute lift: {diff:.3%}  95% CI: ({ci_diff[0]:.3%},
{ci_diff[1]:.3%})")
print(f"Relative lift: {diff/ctrl_conv.mean():.1%}  z={z_stat:.3f}
p={p_val:.5f}")

h = proportion_effectsize(trt_conv.mean(), ctrl_conv.mean())
print(f"Effect size (Cohen's h): {h:.3f}")
```

Business interpretation: report absolute lift, relative lift, the CI, and the effect size together (Section 120's reporting standard) — the recommendation to ship depends jointly on whether the CI excludes a practically meaningless lift and whether implementation cost is justified by the point estimate, never on p<0.05 in isolation.

---

127. End-to-End Project 4: Financial Risk Statistics

```
n_days = 1000
returns = rng_p.standard_t(df=4, size=n_days) * 0.015   # heavy-
tailed, realistic for daily returns

print(f"Mean daily return: {returns.mean():.5f}")
print(f"Median: {np.median(returns):.5f}")
print(f"Annualized volatility: {returns.std()*np.sqrt(252):.2%}")
print(f"Skewness: {stats.skew(returns):.3f}, Excess kurtosis:
{stats.kurtosis(returns):.3f}")

# Distribution fitting — normal vs Student's t
norm_params = stats.norm.fit(returns)
t_params = stats.t.fit(returns)
aic_norm = 2*2 - 2*stats.norm.logpdf(returns, *norm_params).sum()
aic_t = 2*3 - 2*stats.t.logpdf(returns, *t_params).sum()
print(f"AIC — Normal: {aic_norm:.1f}   Student's t: {aic_t:.1f}
(lower is better)")

# Correlation with a second asset
asset2 = 0.6*returns + rng_p.standard_t(df=4, size=n_days)*0.012
r, p_corr = stats.pearsonr(returns, asset2)
print(f"Correlation between assets: r={r:.3f}, p={p_corr:.6f}")
```

```
# Monte Carlo simulation of a portfolio's future value
n_sims, n_days_forward = 5000, 252
simulated_paths = np.cumprod(1 + rng_p.choice(returns, (n_sims,
n_days_forward)), axis=1)
final_values = simulated_paths[:, -1]
print(f"Monte Carlo 1-year return — median: {np.median(final_values)-
1:.1%}, "
      f"5th pct: {np.percentile(final_values,5)-1:.1%}, 95th pct:
{np.percentile(final_values,95)-1:.1%}")

# Value at Risk and Expected Shortfall (concepts)
VaR_95 = -np.percentile(returns, 5)
ES_95 = -returns[returns <= np.percentile(returns,5)].mean()
print(f"1-day VaR (95%): {VaR_95:.3%}   Expected Shortfall (95%):
{ES_95:.3%}")
```

VaR answers "what's the worst loss I should expect on 95% of days" (a percentile); **Expected Shortfall** answers "given I'm in that worst 5%, what's the *average* loss" — ES is more sensitive to true tail severity and is why Section 14.2's kurtosis discussion matters directly here: a normal-distribution-based VaR badly underestimates both, since real returns' heavy tails (confirmed above by the AIC comparison favoring Student's t) make extreme days more frequent and more severe than a normal model predicts.

---

128. End-to-End Project 5: Healthcare Statistical Analysis

```
n_patients = 3000
health_df = pd.DataFrame({
    "age": rng_p.integers(20, 90, n_patients),
    "bmi": rng_p.normal(27, 5, n_patients),
    "smoker": rng_p.choice([0,1], n_patients, p=[0.75,0.25]),
})
disease_logit = -5 + 0.04*health_df.age + 0.08*health_df.bmi +
1.1*health_df.smoker
health_df["disease"] = (rng_p.random(n_patients) < 1/(1+np.exp(-
disease_logit))).astype(int)
print(f"Disease prevalence: {health_df.disease.mean():.2%}")

# Group differences
smoker_age = health_df[health_df.smoker==1].age
nonsmoker_age = health_df[health_df.smoker==0].age
t_s, p_s = stats.ttest_ind(smoker_age, nonsmoker_age)
print(f"Age by smoking status: t={t_s:.2f}, p={p_s:.4f}")

# Confidence interval on prevalence
ci_prev = proportion_confint(health_df.disease.sum(), n_patients,
method='wilson')
print(f"Disease prevalence 95% CI: ({ci_prev[0]:.2%},
{ci_prev[1]:.2%})")
```

```
# Logistic regression with odds ratios
X_health =
sm.add_constant(health_df[["age","bmi","smoker"]].astype(float))
health_fit = sm.Logit(health_df.disease, X_health).fit(disp=0)
odds_health = np.exp(health_fit.params)
odds_ci = np.exp(health_fit.conf_int())
print(pd.DataFrame({"odds_ratio": odds_health, "ci_low": odds_ci[0],
"ci_high": odds_ci[1]}))

# Diagnostic performance (if a simple threshold rule were used)
predicted_prob = health_fit.predict(X_health)
from sklearn.metrics import roc_auc_score
print(f"Model AUC: {roc_auc_score(health_df.disease,
predicted_prob):.3f}")
```

Statistical and ethical limitations, stated together: the odds ratios describe *association* in this synthetic (or, in a real study, this specific observational) population, not a proven causal or biological mechanism; deploying any such model clinically would require external validation, calibration checking (Section 104.2), and — critically — fairness auditing across demographic subgroups (Section 148) before any real diagnostic or resource-allocation decision relies on it.

---

129. End-to-End Project 6: Energy Data Statistical Analysis

```
t_energy = np.arange(365*3)
seasonal_energy = 500 + 150*np.sin(2*np.pi*t_energy/365 + np.pi) +
0.1*t_energy + rng_p.normal(0,30,len(t_energy))
energy_series = pd.Series(seasonal_energy, index=pd.date_range("2021-
01-01", periods=len(t_energy)))

print(energy_series.describe())
decomposition = seasonal_decompose(energy_series, period=365)
# decomposition.plot()

adf_stat, adf_p, *_ = adfuller(energy_series)
print(f"ADF test on raw series: p={adf_p:.4f}")
adf_stat_d, adf_p_d, *_ = adfuller(energy_series.diff().dropna())
print(f"ADF test on DIFFERENCED series: p={adf_p_d:.4f}")

# Outlier detection in daily consumption
z_energy = (energy_series - energy_series.mean()) /
energy_series.std()
outlier_days = energy_series[np.abs(z_energy) > 3]
print(f"Flagged outlier days: {len(outlier_days)}")

# Correlation with a weather proxy
temperature_proxy = 20 - 15*np.sin(2*np.pi*t_energy/365 + np.pi) +
rng_p.normal(0,3,len(t_energy))
r_energy, p_energy = stats.pearsonr(seasonal_energy,
temperature_proxy)
```

```
print(f"Energy vs temperature correlation: r={r_energy:.3f},
p={p_energy:.2e}")
```

Preparing for forecasting (foundations only): the raw series is non-stationary (trend + seasonality confirmed by ADF and the decomposition); differencing to achieve stationarity (Section 100, 102's "I" in ARIMA) is the standard next step before fitting any classical forecasting model — the actual forecasting methodology belongs to the dedicated Time-Series ML module, with this project supplying exactly the statistical groundwork it will build on.

---

130. End-to-End Project 7: Multivariate Statistical Analysis

```
n_mv = 500
mv_df = pd.DataFrame({
    "revenue": rng_p.lognormal(11, 0.5, n_mv),
    "marketing_spend": rng_p.lognormal(9, 0.6, n_mv),
    "headcount": rng_p.integers(5, 200, n_mv),
    "customer_count": rng_p.integers(50, 5000, n_mv),
})
mv_df["revenue"] += 0.3*mv_df["marketing_spend"] +
200*mv_df["customer_count"]

# Correlation and covariance matrices
print(mv_df.corr().round(3))
print(mv_df.cov().round(1))
sns.heatmap(mv_df.corr(), annot=True, cmap="coolwarm")

# PCA
mv_std = (mv_df - mv_df.mean()) / mv_df.std()
pca = PCA().fit(mv_std)
print("Explained variance ratio:",
pca.explained_variance_ratio_.round(3))
print("Cumulative:",
np.cumsum(pca.explained_variance_ratio_).round(3))

# Multivariate regression
X_mv =
sm.add_constant(mv_df[["marketing_spend","headcount","customer_count"]
])
mv_model = sm.OLS(mv_df.revenue, X_mv).fit()
print(mv_model.summary().tables[1])

# Mahalanobis distance for multivariate outlier screening
cov_inv_mv = np.linalg.inv(mv_df.cov().values)
mean_mv = mv_df.mean().values
mv_df["mahalanobis"] = mv_df.apply(
    lambda row:
mahalanobis(row[["revenue","marketing_spend","headcount","customer_cou
nt"]].values,
                                mean_mv, cov_inv_mv), axis=1)
```

```
threshold_mv = np.sqrt(stats.chi2.ppf(0.975, df=4))   # chi-square-
based cutoff
print(f"Multivariate outliers flagged: {(mv_df.mahalanobis >
threshold_mv).sum()}")
```

Why the multivariate view catches what univariate analysis misses: a row with unremarkable individual z-scores on every variable can still be a genuine multivariate outlier if it violates the *typical joint relationship* among variables (e.g. very high marketing spend paired with very low resulting revenue, when the two are normally strongly correlated) — precisely Section 95.2's argument, now demonstrated end-to-end on realistic multivariate business data.

131. Statistical Simulation Labs

Each lab: Objective → Theory → Code → Expected Output → Interpretation. A representative set spanning the module's core themes — each is a template to rerun with your own parameters.

Lab 1 — Sampling Bias. *Objective:* see how a biased sample distorts an estimate. *Theory:* Section 18.

```
population = rng_p.normal(50, 10, 100_000)
biased_sample = population[population > 55][:500]   # only sample
"high" values
print(f"True mean: {population.mean():.2f}  Biased sample mean:
{biased_sample.mean():.2f}")
```

Interpretation: the biased sample overstates the mean by construction — no amount of increasing biased-sample size fixes this (Section 122, item 33).

Lab 2 — Sampling Distributions. *Theory:* Section 19. Simulate 5000 sample means of n=30 from a skewed population; plot the histogram; observe approximate normality despite the skewed source (Section 20).

Lab 3 — CLT Convergence Speed. *Theory:* Section 20.2. Compare convergence speed for a normal vs an exponential source population across $n \in \{2,10,30\}$.

Lab 4 — Law of Large Numbers. *Theory:* Section 21. Plot the running sample mean against increasing n; observe convergence to the true population mean.

Lab 5 — Confidence Interval Coverage. *Theory:* Section 26.2. Simulate 200 CIs; confirm ~95% contain the true parameter; visualize with the green/red interval plot from Section 26.2.

Lab 6 — Type I Error Rate. *Objective:* confirm α is genuinely the false-positive rate under a TRUE null.

```
false_positives = 0
n_sim = 2000
for _ in range(n_sim):
    a, b = rng_p.normal(0, 1, 30), rng_p.normal(0, 1, 30)   # SAME
true distribution
    _, p = stats.ttest_ind(a, b)
    false_positives += p < 0.05
print(f"Observed Type I error rate: {false_positives/n_sim:.3f}
(should be ≈0.05)")
```

Lab 7 — Type II Error and Power. *Theory:* Section 34-35. Repeat Lab 6 with a TRUE effect (e.g. b = normal(0.5,1)); observe the rejection rate = empirical power.

Lab 8 — Power vs Sample Size. Sweep n from 10 to 200 for a fixed effect size; plot power rising toward 1; identify the n achieving 80% power, cross-checking against `TTestIndPower`.

Lab 9 — T-Test Behavior Under Assumption Violation. Compare Type I error rates of Student's vs Welch's t-test under genuinely unequal variances — Welch's stays near 0.05; Student's inflates (Section 37.2).

Lab 10 — Chi-Square Under the Null. Simulate many 2×2 tables with truly independent variables; confirm the p-value distribution is uniform on [0,1] (a general property of valid p-values under a true null, worth seeing directly at least once).

Lab 11 — ANOVA Family-Wise Error. *Theory:* Section 46.1. Run pairwise t-tests across 5 truly-identical groups without correction; observe the inflated false-positive rate; repeat with Tukey HSD (Section 44) and observe correction.

Lab 12 — Bootstrap vs Analytical CI. Compare a bootstrap CI and a t-based CI for the mean on normal data (should closely agree) and on skewed data (should diverge, Section 123 item 30).

Lab 13 — A/B Test Peeking Inflation. *Theory:* Section 71.4.

```
false_positive_with_peeking = 0
n_sim = 1000
for _ in range(n_sim):
    a, b = [], []
    stopped_significant = False
    for day in range(20):
        a.extend(rng_p.normal(0,1,20)); b.extend(rng_p.normal(0,1,20))
# no true effect
        if len(a) > 40:
            _, p = stats.ttest_ind(a, b)
            if p < 0.05: stopped_significant = True; break
    false_positive_with_peeking += stopped_significant
print(f"False positive rate WITH daily peeking:
{false_positive_with_peeking/n_sim:.3f}  (inflated well above 0.05)")
```

Lab 14 — Correlation Under Non-linearity. Reproduces Section 48.1's $y=x^3$ example; compare Pearson vs Spearman directly.

Lab 15 — Multicollinearity's Effect on Coefficient Stability. Fit OLS on near-collinear features across 100 bootstrap resamples; observe wildly varying coefficients (Section 56); repeat with Ridge and observe stabilization.

Lab 16 — Heteroscedasticity's Effect on Standard Errors. Simulate data with variance increasing in X; compare naive vs robust (HC3) standard errors' coverage.

Lab 17 — Multiple Testing Inflation. Reproduces Section 46.1's `1-(1-α)^m` formula empirically across m independent tests on null data.

Lab 18 — Distribution Drift Detection. Simulate a gradual mean shift in a feature over 100 "days"; compute rolling PSI (Section 110.1); identify the day drift crosses the 0.25 threshold.

Lab 19 — Control Chart Sensitivity. Simulate a stable process with an injected shift (Section 111); sweep the shift magnitude and observe how quickly (how many points) the control chart detects it.

Lab 20 — Survival Analysis Under Censoring. Compare a naive "average observed tenure" (ignoring censoring) against a proper Kaplan-Meier estimate (Section 103) on the same simulated, censored data — quantify the bias of the naive approach.

Lab 21 — PCA Variance Capture. Vary the correlation strength between two simulated features; observe how PC1's explained-variance ratio changes from ~50% (uncorrelated) toward ~100% (near-perfectly correlated).

Lab 22 — Model Metric Bootstrap Uncertainty. *Theory:* Section 108. Bootstrap the AUC of a fitted classifier; report the 95% CI; repeat at two different sample sizes and observe the CI narrowing.

Lab 23 — Model Comparison via Paired Test. *Theory:* Section 109. Simulate two models' 10-fold CV scores with a small true difference; test with a paired t-test; observe how often it's detected at different sample-size (fold count) settings.

Lab 24 — MLE Consistency. Simulate increasingly large samples from a Poisson(λ=4); compute the MLE (sample mean) at each n; observe convergence to the true λ, visualizing Section 23's consistency property directly.

Lab 25 — Bayesian Updating in Real Time. Simulate a stream of Bernoulli trials arriving one at a time; update a Beta posterior after each observation (Section 77.1); animate/plot the posterior narrowing as data accumulates.

---

132. Statistics Exercises

A representative set spanning all required categories, with concise solutions.

132.1 Beginner (1–10)

1. Compute the mean, median, and mode of [4,4,5,7,9]. → mean=5.8, median=5, mode=4. **2.** Why is the median preferred for salary data? → Right skew from high earners distorts the mean (Section 7.2). **3.** Compute the sample variance of [2,4,6,8,10]. → mean=6; deviations^2=[16,4,0,4,16]=40; s^2=40/4=10. **4.** What's the difference between a parameter and a statistic? → Section 2.2. **5.** State the four levels of measurement. → nominal, ordinal, interval, ratio (Section 4). **6.** What does a z-score of 2.5 mean? → The value is 2.5 standard deviations above the mean. **7.** Define Type I and Type II error in your own words. → Section 34. **8.** Why does standard error shrink as n grows, while standard deviation does not? → Section 16.2. **9.** What is the difference between MCAR and MAR? → Section 85.1. **10.** What is a p-value, precisely? → P(data this extreme | H_0 true) (Section 33.1).

132.2 Intermediate (11–20)

11. A sample of n=40 has $\bar{x}$=72, s=9. Construct a 95% CI for μ. → t≈2.023, SE=1.423, CI≈(69.1, 74.9). **12.** Explain why Welch's t-test is the modern default over Student's. → Section 37.2. **13.** A chi-square test on a 2×3 table gives χ^2=12.4, df=2. Is this significant at α=0.05? → Critical value ≈5.99; yes, reject H_0. **14.** Why does R^2 always increase when adding predictors, and what fixes this? → Section 59.1, adjusted R^2. **15.** Explain multicollinearity's effect on coefficient

interpretation. → Section 56, unstable/uninterpretable coefficients. **16.** What does a VIF of 15 indicate, and what should you do? → Severe multicollinearity; drop/combine features or use Ridge. **17.** Why is a paired t-test more powerful than an independent-samples test for before/after data? → Removes between-subject variability (Section 38.1). **18.** State the null and alternative hypotheses for testing whether a new drug reduces blood pressure. → H_0: $\mu_{after}=\mu_{before}$ (or $\mu_{drug}=\mu_{placebo}$); H_1: $\mu_{after}<\mu_{before}$ (one-sided). **19.** Why does the Wald CI for a proportion fail for small n or extreme p? → Section 28.2. **20.** Explain Simpson's Paradox using your own numeric example. → Should follow the mechanism of Section 75.1.

132.3 Advanced (21–30)

21. Derive why n-1 (not n) gives an unbiased variance estimator. → Module 5 Section 60.2 / Section 8.2. **22.** Explain the bias-variance-irreducible error decomposition and give a business example of each term. → Section 89. **23.** Why does Bonferroni control the family-wise error rate but FDR methods control something different? → Section 46.2. **24.** Derive the relationship between Ridge regression and MAP estimation under a Gaussian prior. → Section 92.1. **25.** Explain why a confounder must be controlled for but a collider must not. → Section 76.1. **26.** What does Cook's distance measure, and how does it differ from a simple residual? → Section 57, combines leverage and residual size. **27.** Explain why AIC and BIC penalize model complexity differently. → Section 60.1, BIC's $\ln(n)$ term. **28.** Derive the odds ratio from a 2×2 contingency table and explain when it approximates the risk ratio. → Section 62, rare-outcome assumption. **29.** Explain why nested cross-validation is needed when hyperparameter tuning is part of the pipeline. → Section 107.2. **30.** Derive why the MLE of a Poisson distribution's λ is the sample mean. → Section 24.1, maximizing the Poisson log-likelihood.

132.4 Mathematical (31–35)

31. Prove $\Sigma(x_i-\bar{x})=0$ always. → Expand the sum; $\bar{x}$ terms cancel exactly. **32.** Show that $\mathrm{Var}(X)=E[X^2]-(E[X])^2$. → Module 5, Section 60.2. **33.** Derive the standard error formula $SE=\sigma/\sqrt{n}$ from the variance of a sample mean. → $\mathrm{Var}(\bar{x})=\mathrm{Var}((1/n)\Sigma X_i)=(1/n^2)\cdot n\sigma^2=\sigma^2/n$; $SE=\sqrt{\text{that}}=\sigma/\sqrt{n}$. **34.** Show that the F-statistic in a two-group ANOVA equals the square of the corresponding two-sample t-statistic. → A known identity; verify numerically with `scipy`. **35.** Derive the closed-form MLE for a Normal distribution's μ and σ^2. → Section 24.1; maximize the Gaussian log-likelihood w.r.t. each parameter.

132.5 Python (36–40)

36. Write a function computing a bootstrap CI for any statistic. → Section 67.2. **37.** Write code detecting sample ratio mismatch given two group sizes. → Section 72.2's chi-square approach. **38.** Write a function computing PSI between two distributions. → Section 110.1. **39.** Write code performing a full Levene + Welch's t-test workflow, choosing the test automatically based on the Levene result. → Combine Sections 37.1-37.2. **40.** Write a function computing Cohen's d for two independent samples. → Section 45.1.

132.6 ML-Focused (41–45)

41. Explain why cross-validated metric variance matters as much as the mean. → Section 69.2, 108. **42.** Why must feature selection be refit within each CV fold? → Section 93.2, leakage. **43.** Explain calibration and why a well-discriminating model can still be poorly calibrated. →

Section 104.2. **44.** Why is PR-AUC preferred over ROC-AUC under severe class imbalance? → Module 1, Section 28; Section 106. **45.** Explain the statistical justification for empirical risk minimization. → Section 90.1, LLN-based consistency.

132.7 Business-Focused (46–50)

46. A campaign shows p=0.03 but Cohen's d=0.05. Should you scale it company-wide? → Discuss Section 121's decision framework; likely not, given the trivial effect size. **47.** Your fraud model's AUC improved from 0.85 to 0.86 after retraining. Is this deployment-worthy? → Apply Section 109's paired comparison and cost-benefit reasoning. **48.** A churn-reduction offer shows a 95% CI of (−0.5%, +3.2%) for its effect. How would you communicate this to a VP? → Emphasize honest uncertainty per Section 120; the true effect might be negative. **49.** Explain to a non-technical stakeholder why "statistically significant" doesn't mean "we should definitely act." → Combine Sections 33.3 and 121. **50.** Your dashboard flags 40 "significant" metric changes overnight. What's your first move? → Suspect multiple-testing inflation (Section 46.1); apply FDR correction before investigating individually.

132.8 Scenario-Based (51–55)

51. A/B test shows SRM (p=0.001 on the ratio check). What do you do? → Halt interpretation; audit the assignment pipeline (Section 72.2). **52.** Two "equivalent" statistical tests on the same data disagree. → Investigate distributional assumptions; trust the nonparametric result if normality is violated (Section 123, item 16). **53.** Your regression's residuals show a clear funnel shape. → Diagnose heteroscedasticity; apply robust SEs or transform Y (Section 55). **54.** A subgroup analysis (unplanned) shows a striking effect. → Treat as hypothesis-generating only; flag the multiple-comparisons risk; recommend a pre-registered follow-up (Section 46, 122 item 36). **55.** Your time-series model's residuals show significant autocorrelation at lag 7. → The model is missing a weekly seasonal component (Section 101, 102).

---

133. Statistics Coding Challenges

A representative set with solutions/pointers.

1. Compute a five-number summary from scratch. → `np.percentile(x, [0,25,50,75,100])`. **2. Implement Welch's t-test manually and verify against `scipy`.** → Section 37.1 formulas. **3. Implement a bootstrap CI for the median.** → Section 67.2, with `np.median` as `statistic_fn`. **4. Implement PSI from scratch.** → Section 110.1's function. **5. Implement the Wilson interval for a proportion.** → Verify against `statsmodels`' `proportion_confint(method='wilson')`. **6. Write a function flagging multivariate outliers via Mahalanobis distance.** → Section 95.2. **7. Implement a permutation test for a difference in means.**

```
def permutation_test(a, b, n_perm=10000, seed=0):
    rng_pt = np.random.default_rng(seed)
    observed = a.mean() - b.mean()
    combined = np.concatenate([a, b])
    count = 0
```

```
    for _ in range(n_perm):
        rng_pt.shuffle(combined)
        perm_diff = combined[:len(a)].mean() -
combined[len(a):].mean()
        count += abs(perm_diff) >= abs(observed)
    return observed, count / n_perm
```

8. Implement Benjamini-Hochberg FDR correction from scratch. → Sort p-values, compare each to (rank/m)·α. **9. Write a function computing VIF for every column in a DataFrame.** → Section 56.2. **10. Implement K-fold cross-validation from scratch (no scikit-learn).** → Manually partition indices into k folds. **11. Implement a control chart function flagging out-of-control points.** → Section 111.1. **12. Write a function computing Cohen's d, eta-squared, and an odds ratio given raw data.** → Section 45.1's three formulas. **13. Implement the Kaplan-Meier estimator from scratch.** → A moderate challenge; compute the stepwise survival probability product at each observed event time. **14. Write a function performing automatic test selection: given two samples, run Shapiro-Wilk, then dispatch to t-test or Mann-Whitney accordingly.** → Combine Sections 47, 83. **15. Implement a simple A/B test power calculator from scratch (no `statsmodels`).** → Use the normal-approximation formula for two-proportion power. **16. Write a function computing a regression's Cook's distance manually.** → Use the hat matrix $H = X(X^TX)^{-1}X^T$ and standard formula. **17. Implement method-of-moments estimation for a Gamma distribution.** → Section 25.1. **18. Write a function detecting sample ratio mismatch given arbitrary group counts (not just 2).** → Generalize Section 72.2's chi-square goodness-of-fit to k groups. **19. Implement a simple Bayesian A/B test comparing two Beta posteriors via Monte Carlo.**

```
def bayesian_ab_test(a_successes, a_trials, b_successes, b_trials,
n_samples=100000, seed=0):
    rng_bt = np.random.default_rng(seed)
    a_samples = rng_bt.beta(1+a_successes, 1+a_trials-a_successes,
n_samples)
    b_samples = rng_bt.beta(1+b_successes, 1+b_trials-b_successes,
n_samples)
    return (b_samples > a_samples).mean()   # P(B beats A)
```

20. Implement a rolling z-score anomaly detector for a time series. → Compute a rolling mean/std window, flag points beyond ±3 rolling SD.

(Challenges 21–30 follow the identical problem→solution structure across: multiple-imputation simulation, jackknife SE estimation, chi-square goodness-of-fit from scratch, ARIMA order selection via ACF/PACF inspection, calibration-curve computation from scratch, stratified K-fold implementation, a from-scratch Mann-Whitney U statistic, Q-Q plot construction without `scipy.stats.probplot`, a nested cross-validation loop, and a from-scratch two-proportion z-test — each directly exercising the corresponding section's formulas.)

134. Statistics Interview Preparation

134.1 Beginner (1–20)

1. What's the difference between a population and a sample? Section 2.1. **2. Define mean, median, mode and when each is preferred.** Section 7. **3. What's the difference between variance and standard deviation?** SD is in original units, variance is squared (Section 8). **4. Why n-1 in sample variance?** Bessel's correction (Section 8.2). **5. What is a p-value?** Section 33.1. **6. What's a Type I vs Type II error?** Section 34. **7. What is a confidence interval, correctly interpreted?** Section 26.2. **8. What's the Central Limit Theorem?** Section 20. **9. Difference between correlation and causation?** Section 74. **10. What is standard error, and how does it differ from standard deviation?** Section 16.2. **11. What are the four levels of measurement?** Section 4. **12. What is skewness, and how does it affect the mean-median relationship?** Section 13. **13. What is an outlier, and how would you detect one?** Z-score/IQR methods, Sections 9, 15. **14. What's the difference between descriptive and inferential statistics?** Section 1.2. **15. Explain the empirical rule (68-95-99.7).** Module 5, Section 63.3. **16. What is a null hypothesis?** Section 31.2. **17. What's the difference between a parameter and a statistic?** Section 2.2. **18. What is sampling bias, and name three types.** Section 18. **19. What's a histogram, and how does bin width affect interpretation?** Section 11. **20. What's the difference between a population parameter and an estimator?** Section 22.2.

134.2 Intermediate (21–45)

21. Explain the difference between Student's and Welch's t-test. Section 37. **22. When would you use a nonparametric test instead of a parametric one?** Section 47.3. **23. Explain the multiple testing problem and one correction method.** Section 46. **24. What is statistical power, and what factors influence it?** Section 35. **25. Explain the difference between R^2 and adjusted R^2.** Sections 58-59. **26. What is multicollinearity, and how do you detect it?** Section 56. **27. Explain heteroscedasticity and its consequences.** Section 55. **28. What's the difference between a confidence interval and a prediction interval?** Section 105.2. **29. Explain Simpson's Paradox with an example.** Section 75. **30. What's the difference between a confounder and a mediator?** Section 76.1. **31. Explain what "statistical significance" doesn't tell you.** Section 33.3. **32. What is an odds ratio, and how does it differ from a risk ratio?** Section 62. **33. Explain the bootstrap method and one situation where it's essential.** Section 67. **34. What's the difference between MAR, MCAR, and MNAR?** Section 85.1. **35. Explain AIC and BIC, and how they differ.** Section 60. **36. What is a GLM, and name three special cases.** Section 63. **37. Explain why cross-validation is a form of statistical resampling.** Section 69.1. **38. What is the difference between a Type I error rate and a p-value?** α is fixed in advance; the p-value is data-dependent and compared against α. **39. Explain the Wilson interval and why it's preferred over the Wald interval.** Section 28.2. **40. What's the difference between Pearson, Spearman, and Kendall correlation?** Section 48.1. **41. Explain what VIF measures.** Section 56.2. **42. What is Cook's distance used for?** Section 57. **43. Explain the difference between one-sided and two-sided hypothesis tests.** Section 32. **44. What's the difference between a Type II error and low statistical power?** Low power increases the *probability* of a Type II

error on any given test. **45. Explain why you should never say "we accept the null hypothesis."** Section 31.2.

134.3 Advanced (46–70)

46. Derive why the sample mean is an unbiased estimator of the population mean. $E[\bar{x}]=E[(1/n)\Sigma X_i]=(1/n)\Sigma E[X_i]=\mu$. **47. Explain the bias-variance-irreducible error decomposition.** Section 89. **48. Why does Ridge regression's closed form always exist, even with multicollinearity?** Module 5, Section 72.3. **49. Explain the connection between MAP estimation and regularization.** Section 92.1. **50. Derive the MLE for a Bernoulli parameter.** Section 24.1. **51. Explain why the Kaplan-Meier estimator correctly handles censored data.** Section 103.1. **52. What is the difference between empirical risk and true risk?** Section 90.1. **53. Explain why nested cross-validation is necessary when tuning hyperparameters.** Section 107.2. **54. What is the exponential family, and why does it matter for GLMs?** Section 143. **55. Explain Fisher information and its relationship to standard errors.** Section 142. **56. What is the Cramér-Rao lower bound, conceptually?** Section 141 — no unbiased estimator can have lower variance than this bound. **57. Explain propensity score matching's purpose.** Section 147. **58. What is a collider, and why does conditioning on it introduce bias?** Section 76.1, 74.2. **59. Derive the relationship between a chi-square test and an F-test.** Both derive from sums of squared normal variables; F is a ratio of two independent chi-squares. **60. Explain the difference between a credible interval and a confidence interval philosophically.** Section 26.2 vs 77.2. **61. What is the delta method used for?** Section 141 — approximating the variance of a function of an estimator via a first-order Taylor expansion. **62. Explain why PSI is preferred over a formal hypothesis test for production drift monitoring.** Section 110.2. **63. What does "sufficiency" mean for an estimator?** Section 23 — it uses all sample information relevant to the parameter. **64. Explain asymptotic normality of the MLE.** Section 144 — for large n, the MLE's sampling distribution approaches normal, centered at the truth. **65. What is the difference between a Type I error rate under a single test vs a family of tests?** Section 46.1. **66. Explain why regularization introduces bias but can improve out-of-sample performance.** Section 89, 92.2. **67. What's the statistical justification for using log-likelihood instead of likelihood in MLE?** Module 5, Section 10.3 numerical + Section 24 statistical framing. **68. Explain why a well-calibrated model can still have poor discrimination, and vice versa.** Section 104.2 — genuinely separate properties. **69. What is the practical difference between AIC and cross-validated error for model selection?** AIC is a fast, in-sample-based approximation; CV directly, if more expensively, estimates the out-of-sample quantity you actually care about. **70. Explain the statistical basis for the "no free lunch" caution against always preferring more complex models.** Section 89's bias-variance argument, generalized.

134.4 Coding Questions (71–85, representative)

71–75. Implement, from scratch: a bootstrap CI, Welch's t-test, PSI, a permutation test, Cohen's d — see Section 133 for full solutions to each. **76–80.** Given a Pandas DataFrame, write code to: compute a grouped confidence interval, detect and report VIF for all features, run an automated normality-then-test-selection pipeline, compute a rolling z-score anomaly flag, and correct a list of p-values via Benjamini-Hochberg — see Sections 114-118, 133 for the underlying building blocks. **81–85.** Debug five deliberately broken snippets: a `ddof` mismatch causing wrong

variance, a Wald-interval failure on extreme p, an unpaired test applied to paired data, an uncorrected multiple-comparison scenario, and a leakage bug from fitting a scaler on the full dataset — reason through each using Section 123's troubleshooting entries as the template.

134.5 Scenario-Based (86–100, representative)

86. Your manager wants a single number: "is the campaign working?" Present the lift, CI, and effect size together; explain why a single p-value alone is an incomplete answer (Section 120-121). **87. A colleague ran 50 t-tests on subgroups and found 4 significant. How do you respond?** Point out that ~2.5 significant results are expected by chance alone under a true null across 50 tests (Section 46.1); recommend FDR correction before any interpretation. **88. Your regression's F-test is significant, but no individual predictor is.** Diagnose multicollinearity (Section 123, item 21). **89. A model with 0.95 AUC is proposed for deployment, but the team is suspicious.** Ask about training-serving skew, leakage, and class imbalance (Sections 93.2, 106, 110) before trusting the number. **90. Explain to a skeptical exec why you can't just "prove" the new feature caused the metric increase.** Explain observational vs experimental evidence (Section 74), and recommend a proper A/B test if causal proof matters.

(Items 91–100 follow the identical worked-scenario pattern across: interpreting a wide bootstrap CI, explaining regression to the mean after an "outlier" period, defending a Bayesian approach to a frequentist-trained stakeholder, explaining why a drift alert didn't correspond to a performance drop, and four more variations on communicating uncertainty and avoiding overclaiming to non-technical audiences — each directly answerable using Sections 33, 45, 120-122's frameworks.)

---

135. Statistical Interview Traps

Deliberately tricky questions designed to expose shallow understanding.

1. "If p=0.04, is there a 96% chance the alternative hypothesis is true?" No — this reverses the conditional direction (Section 33.2); p says nothing directly about $P(H_1|\text{data})$.

2. "A 95% CI is (2, 8). Is there a 95% chance the true mean is between 2 and 8?" Under the frequentist framework, no — this is the classic misinterpretation (Section 26.2); the *procedure* has 95% long-run coverage, not this specific realized interval.

3. "We tested 20 features and found 3 significant at p<0.05 — that's strong evidence, right?" No — roughly 1 significant result is expected by chance alone with 20 tests at α=0.05 (Section 46.1); 3 is not obviously more than chance without correction.

4. "Our sample size is huge, so we don't need to worry about assumptions." Partially true for some (CLT rescues coefficient estimates asymptotically) but false for others (heteroscedasticity still corrupts standard errors regardless of n) — a nuanced answer distinguishing which assumptions large n does and doesn't rescue.

5. "The correlation is 0.02, so there's no relationship." Only rules out a *linear* relationship — a strong non-linear monotonic or non-monotonic relationship can still exist (Section 48.1, Module 5 Section 54.5's $y=x^2$ example).

6. "Bayesian methods are always better because they give a probability for the hypothesis." A trap — Bayesian and frequentist methods answer genuinely different questions and each carry real tradeoffs (subjectivity of priors vs the frequentist interpretability limits); neither is universally superior (Section 78).

7. "We ran the A/B test until it became significant, then stopped — is that valid?" No — this is exactly the peeking problem (Section 71.4) and inflates the true false-positive rate substantially above the nominal α.

8. "Removing the outliers improved our R^2 a lot, so it was the right move." A trap — improved R^2 alone doesn't justify removal; the outliers must first be investigated as potentially genuine, informative observations (Section 57).

9. "Our model has 99% accuracy, so it must be great." A trap under class imbalance (Section 106.1) — always ask the base rate before evaluating an accuracy number.

10. "The confidence interval for the difference includes zero, so there's definitely no effect." A trap — "fail to reject" is not "prove no effect" (Section 31.2); it may simply reflect low power (Section 35).

11. "Two variables are correlated, and X happened before Y, so X must cause Y." A trap — temporal precedence is necessary but not sufficient for causation; a lurking confounder can still fully explain the association (Section 74).

12. "Adding more predictors to my regression only helps, since R^2 can't go down." True of R^2 (Section 59.1) but false of adjusted R^2 and, more importantly, false of genuine out-of-sample predictive performance — a trap testing whether the candidate conflates in-sample fit with generalization.

13. "The Kaplan-Meier curve shows most customers survive past 24 months, so churn isn't a problem." A trap if censoring is misunderstood — verify the analyst isn't conflating "not yet observed to churn" with "confirmed to never churn" (Section 103).

14. "Our drift monitor flagged significance on a 10-million-row dataset, so something is clearly wrong." A trap — at that scale, even trivial, operationally meaningless shifts reach formal significance (Section 110.2); ask for the PSI or effect size, not just the p-value.

15. "We used PCA before running logistic regression to help interpretability." A trap — PCA components are *linear combinations* of original features and are typically *harder*, not easier, to interpret business-wise (Module 5, Section 52.4) — the candidate should recognize this tradeoff rather than assuming PCA always helps.

---

136. Statistics Case Studies

Format: Business question → Statistical question → Data → Method → Assumptions → Analysis → Interpretation → Limitations → Recommendation framework. Each case explicitly avoids treating a statistical finding as automatic proof of causation.

1. Banking — Interest rate sensitivity. Business Q: does raising savings rates increase deposits? Statistical Q: is there a significant association between rate changes and deposit inflows, controlling for macro trends? Method: time-series regression with macro controls. Assumption: no unmeasured confounding macro shock coincides with the rate change. Limitation: observational; a genuine causal claim needs a natural experiment or a randomized pilot across branches.

2. Fraud — Real-time detection thresholds. Statistical Q: what threshold balances false-positive customer friction against missed-fraud loss? Method: ROC/PR curve analysis (Section 104.1) combined with an expected-cost framework (Section 145). Limitation: historical fraud patterns may not reflect evolving fraud tactics (a drift concern, Section 110).

3. Credit — Default risk scoring. Method: logistic regression with odds ratios (Sections 61-62), validated via calibration curves (Section 104.2) given the ultimate use is expected-loss calculation. Limitation: fairness across protected groups must be separately audited (Section 148), independent of overall discrimination performance.

4. Insurance — Claims frequency modeling. Method: Poisson/Negative Binomial regression (Sections 64-65) with exposure offsets. Assumption check: verify equidispersion before trusting a plain Poisson model.

5. Healthcare — Readmission risk. Method: logistic regression, emphasizing interpretability for clinical trust; confidence intervals reported for every odds ratio (Section 61). Limitation: retrospective/observational; clinical protocol changes require prospective validation.

6. Marketing — Multi-channel attribution. Statistical Q: which channels show the strongest association with conversion, controlling for customer segment (Section 43's two-way ANOVA interaction)? Limitation: correlational; a genuine causal attribution requires geo-holdout experiments.

7. E-commerce — Recommendation system A/B test. Method: the full Section 71-72 A/B framework, with SRM checking mandatory given the automated randomization pipeline's bug risk.

8. Telecom — Churn driver analysis. Method: Project 1's exact pipeline (Section 124): descriptive comparison, chi-square association, logistic regression with VIF diagnostics.

9. Energy — Demand forecasting foundations. Method: Project 6's decomposition and stationarity testing (Section 129), feeding a downstream forecasting model outside this module's scope.

10. Oil & Gas — Equipment failure prediction. Method: survival analysis (Section 103) for time-to-failure, given right-censored data (equipment still running at analysis time).

11. Manufacturing — Quality control. Method: control charts (Section 111) on defect rates per batch; special-cause investigation triggered by out-of-control points.

12. Supply Chain — Demand variability. Method: descriptive dispersion measures (Section 8) plus CV (Section 8.3) comparing variability across SKUs on a common relative scale.

13. Cybersecurity — Anomaly detection. Method: statistical process control (Section 111) and multivariate outlier detection (Mahalanobis distance, Section 95.2) on network traffic features.

14. Education — Intervention effectiveness. Method: paired pre/post test design (Section 38) or, ideally, a randomized controlled trial (Section 70) comparing intervention vs control classrooms.

15. HR Analytics — Attrition risk factors. Method: logistic regression with careful attention to fairness (Section 148) given the sensitive, career-affecting nature of the outcome.

16. Finance — Portfolio risk. Method: Project 4's distribution-fitting and VaR/ES framework (Section 127), emphasizing fat-tail awareness (Section 14.2).

17. Investment — Fund performance comparison. Method: paired comparison of fund returns against a benchmark (Section 109's paired-test logic applied to time-aligned return series), with explicit acknowledgment of multiple-fund-comparison inflation if many funds are screened (Section 46).

18. SaaS — Feature adoption impact on retention. Method: an experimental design (Section 73) comparing users with vs without early feature exposure, ideally randomized rather than observational to avoid confounding by user engagement level itself.

19. Product Analytics — Funnel drop-off diagnosis. Method: chi-square tests of independence (Section 40.2) comparing drop-off rates across user segments at each funnel stage.

20. AI/ML Monitoring — Model performance and drift oversight. Method: the complete Section 110-112 toolkit (PSI, control charts) applied continuously to both feature distributions and live model performance metrics, directly operationalizing this module's monitoring content as an ongoing production system rather than a one-time analysis.

137. Statistical Analysis Project Structure

```
statistical_project/
├── data/
│   ├── raw/                    # immutable original data
│   └── processed/              # cleaned, analysis-ready data
├── notebooks/                  # exploration ONLY — numbered by
sequence
│   ├── 01_exploration.ipynb
│   └── 02_hypothesis_testing.ipynb
├── src/
│   ├── data_validation.py      # schema/range checks
│   ├── descriptive.py          # reusable descriptive-stats functions
│   ├── inference.py            # test/CI/bootstrap functions
│   └── reporting.py            # standardized report generation
(Section 120)
├── tests/                      # unit tests for src/ functions
├── reports/                    # final written statistical reports
├── figures/                    # exported plots referenced in reports
├── configs/                    # analysis parameters (alpha, MDE,
etc.), version-controlled
├── README.md
├── requirements.txt
└── environment.yml
```

The exploration → analysis → production separation, made explicit: notebooks are for *exploring* — messy, iterative, disposable. Once an analysis approach is validated, its logic migrates into tested, reusable `src/` functions — exactly Module 1's Section 27.2 discipline, applied here specifically to statistical (not just ML) code, since a hypothesis test or bootstrap procedure deserves the same reproducibility rigor as a trained model.

---

138. Reproducible Statistical Analysis

Requirement	Practice
Random seeds	Set explicitly for every bootstrap, permutation test, and simulation (Section 123, item 39)
Environment management	`environment.yml`/`requirements.txt` pinning exact library versions
Dependency pinning	`scipy==1.11.4`, not `scipy` — an unpinned upgrade can silently change default test behavior
Data versioning	Immutable raw data plus a documented processing pipeline (never edit raw data in place)
Code versioning	Git, with analysis-specific branches for major methodology changes

Requirement	Practice
Analysis versioning	Tag which code+data+config combination produced a specific reported result
Reproducible notebooks	Run top-to-bottom cleanly, in order — no out-of-sequence cell execution left in the final version
Statistical reporting	Every reported number traceable back to the exact code and data version that produced it

```
import random
SEED = 42
np.random.seed(SEED)
random.seed(SEED)
rng = np.random.default_rng(SEED)   # the modern, explicit NumPy
generator — prefer this over global state
```

This is the exact same reproducibility discipline from Module 1's Section 27.3, restated because it is, if anything, *more* commonly neglected in ad-hoc statistical analysis (a one-off Jupyter notebook) than in formal ML pipelines — and the consequences (an unreproducible reported p-value or CI) are just as serious for the credibility of a statistical finding.

---

139. Statistical Testing in Production

Statistics operationalized as ongoing production monitoring, not one-off analysis:

Monitored Object	Statistical Tool
Data quality	Automated range/schema/null-rate checks, alerting like a continuous hypothesis test against expected bounds
Data drift	PSI/KL divergence computed on a rolling window (Section 110)
Business KPIs	Control charts (Section 111) on daily/weekly aggregates
Model performance	Rolling accuracy/AUC/calibration tracked against a baseline, with bootstrap CIs (Section 108) around each period's estimate
Model calibration	Periodic recomputation of the calibration curve (Section 104.2) on fresh labeled data as it arrives
Experiment results	Automated SRM checks and sequential-testing-safe significance calculations for live, ongoing experiments

The unifying principle: every static technique taught earlier in this module (a t-test, a control limit, a calibration curve) becomes a *production monitor* simply by recomputing it on a rolling

window and alerting on deviation — Section 140 builds exactly this into a complete worked example.

---

140. Statistical Process Control for Data Systems

```
# A complete worked production-monitoring example
rng_mon = np.random.default_rng(0)
n_days = 90
baseline_period = 30

daily_txn_volume = rng_mon.normal(10000, 500, n_days)
daily_avg_value = rng_mon.normal(85, 8, n_days)
daily_fraud_rate = rng_mon.normal(0.006, 0.0008, n_days)
model_pred_mean = rng_mon.normal(0.05, 0.005, n_days)

# Inject realistic anomalies after the baseline period
daily_txn_volume[60:63] *= 0.4        # a system outage
daily_fraud_rate[75:] += 0.004         # a genuine, sustained fraud-
rate increase
model_pred_mean[80:] += 0.02           # a corresponding prediction-
distribution shift

def monitor_metric(series, baseline_n, name):
    baseline = series[:baseline_n]
    mean_b, std_b = baseline.mean(), baseline.std(ddof=1)
    ucl, lcl = mean_b + 3*std_b, mean_b - 3*std_b
    flags = np.where((series > ucl) | (series < lcl))[0]
    print(f"{name}: baseline mean={mean_b:.4f}, UCL={ucl:.4f},
LCL={lcl:.4f}, "
          f"flagged days={list(flags)}")
    return flags

for series, name in [(daily_txn_volume, "Transaction Volume"),
                     (daily_avg_value, "Avg Transaction Value"),
                     (daily_fraud_rate, "Fraud Rate"),
                     (model_pred_mean, "Model Prediction Mean")]:
    monitor_metric(series, baseline_period, name)

fig, axes = plt.subplots(4, 1, figsize=(12, 12), sharex=True)
for ax, series, name in zip(axes, [daily_txn_volume, daily_avg_value,
daily_fraud_rate, model_pred_mean],
                            ["Volume","Avg Value","Fraud Rate","Model
Pred Mean"]):
    ax.plot(series)
    baseline_mean = series[:baseline_period].mean()
    baseline_std = series[:baseline_period].std(ddof=1)
    ax.axhline(baseline_mean, color='green')
    ax.axhline(baseline_mean+3*baseline_std, color='red',
linestyle='--')
```

```
    ax.axhline(baseline_mean-3*baseline_std, color='red',
linestyle='--')
    ax.set_title(name)
plt.tight_layout()
```

This single example ties together nearly the entire module: baseline estimation (Sections 7-8), control limits derived from the empirical rule (Module 5, Section 63.3), the transaction-volume outage as a clear special-cause event, and the fraud-rate/model-prediction co-movement demonstrating exactly why monitoring the **model's own output distribution** alongside the raw business metrics catches genuine concept drift (Section 110) — the prediction distribution shifted *because* the true fraud rate genuinely shifted, a case where drift monitoring correctly earns its alert rather than firing on a meaningless statistical technicality (Section 110.2's cautionary counter-case).

141. Advanced Statistical Concepts

Intuition-first introductions — full proofs belong to a dedicated mathematical statistics course.

Concept	Intuition	Practical Importance
Sufficiency	A statistic that captures *all* the information in the sample relevant to a parameter — nothing is lost by summarizing	Justifies why, e.g., the sample mean alone (not every individual data point) is all you need for Normal-mean inference
Completeness	A technical condition ensuring an unbiased estimator based on a sufficient statistic is *unique*	Underlies the Lehmann-Scheffé theorem's guarantee of the best unbiased estimator
Exponential families	A broad class of distributions (Bernoulli, Poisson, Normal, and more) sharing a common mathematical form	The unifying structure behind GLMs (Section 63) and much of classical inference — Section 143
Rao-Blackwell theorem	Conditioning any estimator on a sufficient statistic can only improve (or maintain) its variance, never worsen it	The theoretical justification for "always use the sufficient statistic if you have one"
Cramér-Rao lower bound	No unbiased estimator can have variance below a specific bound determined by the Fisher information	Tells you when an estimator (like the MLE, asymptotically) is already as good as theoretically possible
Fisher information	How sharply "peaked" the likelihood function is around the true parameter — more peaked means more informative data	Directly determines standard errors (Section 142)
Asymptotic normality	Many estimators' sampling distributions approach normal as $n \to \infty$, even when the underlying data isn't normal	Justifies large-sample CI/test procedures far beyond the CLT's original mean-of-sample-means context (Section 144)
Delta method	Approximates the variance of a *function* of an estimator via a first-order Taylor expansion	Lets you compute a standard error for, e.g., an odds ratio (a nonlinear function of a coefficient) from the coefficient's own standard error

Why include these at all, in an applied ML-oriented module: these are the theoretical results that *justify* the practical formulas used everywhere else in this module — knowing they

exist, and roughly why they're true, is what separates "I can run `scipy.stats.ttest_ind`" from "I understand why that function's output is trustworthy," a distinction that matters in advanced interviews and in genuinely novel analytical situations where no off-the-shelf function exists.

---

142. Fisher Information

142.1 The Formal Idea, Intuitively

$$I(\theta) = E\left[\left(\frac{\partial}{\partial\theta} \log L(\theta)\right)^2 \right] = -E\left[\frac{\partial^2}{\partial\theta^2} \log L(\theta) \right]$$

Intuition: the *curvature* of the log-likelihood at its peak. A sharply-peaked log-likelihood (high Fisher information) means the data strongly pins down θ — small changes in θ produce large drops in likelihood, so the estimate is precise. A flat log-likelihood (low Fisher information) means many values of θ fit the data almost equally well — the estimate is imprecise.

```
# Visualizing Fisher information as log-likelihood curvature
n_small, n_large = 10, 1000
data_small = rng.binomial(1, 0.3, n_small)
data_large = rng.binomial(1, 0.3, n_large)

p_grid = np.linspace(0.01, 0.99, 200)
ll_small = [np.sum(data_small*np.log(p) + (1-data_small)*np.log(1-p))
for p in p_grid]
ll_large = [np.sum(data_large*np.log(p) + (1-data_large)*np.log(1-p))
for p in p_grid]

plt.plot(p_grid, ll_small, label=f"n={n_small} (flatter — less info)")
plt.plot(p_grid, ll_large, label=f"n={n_large} (sharper — more info)")
plt.legend(); plt.xlabel("p"); plt.ylabel("Log-likelihood")
```

The n=1000 curve is dramatically more sharply peaked — **more data means more Fisher information means a more precise (lower-variance) estimate**, the exact mechanism underlying the √n law (Module 5, Section 65.3) stated in its most fundamental theoretical form.

142.2 Connection to Standard Errors

$$SE(\hat{\theta}) \approx 1/\sqrt{I(\theta)} \qquad \text{(asymptotically, for the MLE)}$$

This is *why* standard errors shrink with more data (more information accumulates) and *why* a flatter likelihood surface (harder-to-estimate parameter, or noisier data) produces wider confidence intervals — Fisher information is the single quantity connecting the shape of the likelihood function directly to every standard error reported throughout this module.

---

143. Exponential Families

143.1 The General Form

$$f(x|\theta) = h(x)\cdot\exp\left(\eta(\theta)\cdot T(x) - A(\theta) \right)$$

Distribution	Exponential Family?
Bernoulli	Yes
Binomial	Yes
Poisson	Yes
Normal (known variance)	Yes
Gamma	Yes
Uniform	**No** (support depends on θ)
Student's t	**No**

143.2 Why This Matters for GLMs and Bayesian Statistics

Every distribution used in Section 63's GLM framework is an exponential family member — this is not a coincidence; the exponential family's mathematical structure is precisely what makes the GLM's maximum-likelihood machinery work cleanly and generally across Gaussian, Bernoulli, and Poisson outcomes with one unified estimation algorithm (iteratively reweighted least squares). **In Bayesian statistics**, exponential family likelihoods have natural **conjugate priors** (like the Beta-Binomial conjugacy used in Section 77.1) — giving closed-form posteriors without numerical approximation, which is exactly why so many textbook Bayesian examples use exponential-family distributions.

---

144. Asymptotic Statistics

144.1 The Core Large-Sample Results

Property	Statement
Consistency	$\hat{\theta}_n \to \theta$ as $n \to \infty$ (Section 23)
Asymptotic normality	$\sqrt{n}(\hat{\theta}_n - \theta) \to \text{Normal}(0, 1/I(\theta))$ as $n \to \infty$
Asymptotic efficiency	The MLE achieves the Cramér-Rao lower bound as $n \to \infty$

```
# Demonstrating MLE asymptotic normality empirically
rng_asym = np.random.default_rng(0)
true_lambda = 4.0
mle_estimates = []
for n in [1000]*3000:
    sample = rng_asym.poisson(true_lambda, n)
    mle_estimates.append(sample.mean())

plt.hist(mle_estimates, bins=50, density=True)
theoretical_se = np.sqrt(true_lambda/1000)
x_range = np.linspace(min(mle_estimates), max(mle_estimates), 200)
plt.plot(x_range, stats.norm.pdf(x_range, true_lambda,
theoretical_se))
```

The MLE's sampling distribution genuinely looks normal, matching the theoretical asymptotic prediction almost exactly at n=1000 — this is *why* every large-sample confidence

interval and hypothesis test in this module (even for parameters with no exact finite-sample theory, like a logistic regression coefficient) is justified: not by the CLT applied to a simple mean, but by this more general asymptotic-normality result applying to essentially any well-behaved MLE.

144.2 Practical Limitation

Asymptotic results are guarantees "eventually, as $n \to \infty$" — they say *nothing* about how large n must actually be for the approximation to be trustworthy, which depends on the specific distribution and parameter (exactly Section 20.2's CLT-convergence-speed lesson, generalized to any MLE-based inference, not only sample means).

---

145. Statistical Decision Theory

145.1 The Formal Objects

```
Loss function:   L(θ, a)         — cost of taking action a when the
true state is θ
Risk function:   R(θ, δ) = E[L(θ, δ(X))]     — expected loss of a
decision rule δ
Bayes risk:      E_θ[R(θ, δ)]     — risk averaged over the prior
distribution of θ
```

145.2 Connecting Decision Theory to ML Systems

```
# A concrete decision-theoretic framing of a fraud threshold choice
cost_false_positive = 5      # customer friction cost per false alarm
cost_false_negative = 500    # average fraud loss per missed case

thresholds = np.linspace(0.01, 0.99, 50)
p_fraud_given_score = lambda s: s   # a toy calibrated-score
assumption
expected_costs = []
for t in thresholds:
    # Simplified: assume score distribution and base rate from Section
125's project
    fp_rate_at_t = np.mean(np.array([0.02]*100) > t)   # illustrative
placeholder logic
    fn_rate_at_t = np.mean(np.array([0.02]*100) <= t)
    expected_cost = fp_rate_at_t*cost_false_positive +
fn_rate_at_t*cost_false_negative
    expected_costs.append(expected_cost)

optimal_threshold = thresholds[np.argmin(expected_costs)]
print(f"Cost-minimizing threshold (illustrative):
{optimal_threshold:.2f}")
```

Statistical decision theory formalizes exactly what Module 1's precision/recall threshold discussion (Section 28.7) was doing informally: the "right" decision threshold is never a universal statistical constant — it is the argmin of *expected loss under the specific cost*

structure of the business problem, precisely the Bayes-risk-minimizing decision rule this section defines formally. **Every ML system that outputs a probability and converts it to an action via a threshold is implicitly solving a decision-theory problem**, whether or not the cost structure was ever made explicit — making it explicit, as above, is simply good practice.

---

146. Causal Inference Foundations

146.1 The Potential Outcomes Framework

```
Y_i(1) = outcome for unit i IF treated
Y_i(0) = outcome for unit i IF untreated
Individual treatment effect = Y_i(1) − Y_i(0)     ← FUNDAMENTALLY
UNOBSERVABLE (only one is ever realized)
Average Treatment Effect (ATE) = E[Y(1) − Y(0)]    ← estimable under
the right conditions
```

The "fundamental problem of causal inference": you never observe both potential outcomes for the same unit — a customer either received the promotional offer or didn't, never both, so the *individual* causal effect is permanently unobservable. All of causal inference is about estimating the **average** effect across a population despite this, using either randomization (Section 70) or careful observational adjustment.

146.2 Randomized vs Observational

	Randomized Experiment	Observational Study
Treatment assignment	Controlled, random (Section 70.2)	Determined by unmeasured/uncontrolled real-world processes
Confounding	Eliminated in expectation, by design	A persistent threat requiring explicit adjustment
Causal claim strength	Strong, if properly randomized and executed	Requires strong, often unverifiable assumptions (e.g. "no unmeasured confounders")

146.3 DAGs, Revisited

Directed Acyclic Graphs (introduced informally in Section 74.2) are the formal language of modern causal inference — encoding assumptions about which variables cause which, and providing rules (like the **backdoor criterion**) for determining exactly which variables must be statistically adjusted for to isolate a causal effect from observational data.

146.4 Explicitly Distinguishing Causal Inference From Predictive Modeling

	Predictive Modeling (Modules 1, 5-7's core)	Causal Inference
Core question	"What will happen?"	"What would happen if we intervened?"

	Predictive Modeling (Modules 1, 5-7's core)	Causal Inference
Correlation sufficient?	Yes — any predictive signal is useful, regardless of mechanism	No — only the causal component of an association is relevant
Typical evaluation	Held-out predictive accuracy	Estimated treatment effect's bias and precision

This module deliberately stops at foundations — a full causal inference treatment (instrumental variables, difference-in-differences, regression discontinuity, synthetic controls) belongs to a dedicated course; what matters here is recognizing *when a business question is actually asking a causal question* ("would this policy work?") versus a predictive one ("who is likely to churn?"), since the two demand entirely different methodological rigor.

---

147. Propensity Score Concept

147.1 Definition

```
e(X) = P(T=1 | X)          "the propensity score: probability of
receiving treatment, given covariates"
```

147.2 Why It's Useful — Matching and Weighting

```
# Conceptual propensity score matching
obs_df = pd.DataFrame({
    "age": rng.integers(20,70,1000),
    "income": rng.normal(500_000,150_000,1000),
})
obs_df["treatment"] = (rng.random(1000) < 1/(1+np.exp(-(-
2+0.03*obs_df.age)))).astype(int)
obs_df["outcome"] = 50 + 5*obs_df.treatment + 0.3*obs_df.age +
rng.normal(0,10,1000)

# Estimate propensity scores via logistic regression
X_ps = sm.add_constant(obs_df[["age","income"]])
ps_model = sm.Logit(obs_df.treatment, X_ps).fit(disp=0)
obs_df["propensity"] = ps_model.predict(X_ps)

# Naive (confounded) comparison vs propensity-weighted comparison
naive_diff = obs_df[obs_df.treatment==1].outcome.mean() -
obs_df[obs_df.treatment==0].outcome.mean()

# Inverse Probability of Treatment Weighting (IPTW)
weights = np.where(obs_df.treatment==1, 1/obs_df.propensity, 1/(1-
obs_df.propensity))
weighted_treated = np.average(obs_df[obs_df.treatment==1].outcome,
weights=weights[obs_df.treatment==1])
weighted_control = np.average(obs_df[obs_df.treatment==0].outcome,
weights=weights[obs_df.treatment==0])
```

```
print(f"Naive difference: {naive_diff:.2f}")
print(f"IPTW-adjusted difference: {weighted_treated -
weighted_control:.2f}  (closer to the true effect of 5)")
```

The idea: if age confounds both treatment assignment and the outcome, comparing raw treated-vs-control means confounds the treatment effect with the age difference between groups. **Matching** pairs treated and control units with similar propensity scores; **weighting (IPTW)** re-weights the sample so the *weighted* treated and control groups resemble each other on the confounders — both are attempts to approximate what a randomized experiment would have given directly. **Kept foundational here deliberately:** propensity score methods carry substantial additional nuance (overlap/positivity assumptions, correct specification of the propensity model itself) that a dedicated causal inference course develops fully.

---

148. Statistics for Responsible AI

148.1 Statistical Concepts Behind Fairness

Fairness Concept	Statistical Basis
Demographic parity	P(ŷ=1) equal across groups — a comparison of proportions (Section 28)
Equalized odds	TPR and FPR equal across groups — comparing conditional probabilities (Section 56) per group
Calibration within groups	The calibration curve (Section 104.2) should hold *separately* for each demographic group, not only in aggregate
Group comparison significance	Is an observed group disparity larger than sampling noise alone would produce? (A direct hypothesis-testing question, Section 31)

```
# A worked group-fairness check, statistically grounded
group_a_tpr, group_a_n = 0.72, 400
group_b_tpr, group_b_n = 0.61, 380

# Is this TPR gap statistically distinguishable from noise?
se_gap = np.sqrt(group_a_tpr*(1-group_a_tpr)/group_a_n +
group_b_tpr*(1-group_b_tpr)/group_b_n)
z_gap = (group_a_tpr - group_b_tpr) / se_gap
p_gap = 2*(1-stats.norm.cdf(abs(z_gap)))
print(f"TPR gap: {group_a_tpr-group_b_tpr:.3f}, z={z_gap:.2f},
p={p_gap:.4f}")
```

148.2 The Simpson's Paradox Connection

A model can show **acceptable aggregate fairness metrics while masking severe subgroup disparities** — precisely Section 75's mechanism, applied here to algorithmic

fairness auditing rather than a medical-treatment comparison; disaggregated subgroup analysis (never only the aggregate number) is essential.

148.3 The Sampling-Bias Connection

A model trained on a sample that underrepresents a demographic group (Section 18) will typically show worse calibration and discrimination specifically for that group — a **direct, mechanical consequence of Section 90's empirical-risk-minimization logic**: the model minimized average loss over a training distribution that gave that group's errors little weight simply because there were few of them.

This module presents the statistical machinery for measuring fairness; it does not make unsupported causal claims about why disparities exist — that determination (data bias, a genuinely different base rate, a flawed label definition, or something else) requires the careful domain investigation Section 76's confounding framework and Section 146's causal-inference foundations are built to support.

---

149. Statistical Ethics

Practice	Why It's a Violation	Professional Alternative
Data manipulation	Altering or selectively excluding data to reach a desired conclusion	Document all data-cleaning decisions transparently, before seeing results where possible
P-hacking	Trying many analyses until one crosses $p<0.05$ (Section 122, item 4)	Pre-register the analysis plan; report all analyses attempted, not only the significant one
Selective reporting	Publishing only favorable results	Report null/negative findings with equal prominence
Misleading visualizations	Truncated axes, cherry-picked timeframes, inconsistent scales	Use honest, complete axes; disclose the data window and any exclusions
Cherry-picking	Reporting only the subgroup/metric/timeframe that supports a narrative	Pre-specify the primary metric and analysis population before looking at results
Misuse of significance	Treating $p<0.05$ as proof, or $p>0.05$ as disproof (Sections 31.2, 33)	Report effect sizes and CIs alongside p-values, always
Misrepresenting uncertainty	Presenting a point estimate without its CI, implying false precision	Section 120's complete reporting standard, applied without exception

Practice	Why It's a Violation	Professional Alternative
Reproducibility neglect	Undocumented, unversioned analysis that can't be independently verified	Section 138's full reproducibility discipline

The professional responsibility, stated plainly: a statistical analyst's job is to find the truth the data supports, not to construct the strongest-sounding case for a predetermined conclusion — every technique in Sections 46, 67, 108, and 120 exists specifically to protect against the analyst's own (often unconscious) motivation to see a desired result, and using them rigorously, even when the honest answer is "we didn't find a significant effect" or "our confidence interval is too wide to conclude anything useful," is the actual measure of statistical professionalism this entire module has been building toward.

150. Master Statistical Formula Sheet

150.1 Descriptive Statistics

Quantity	Formula
Mean	$\bar{x} = (1/n)\Sigma x_i$
Weighted mean	$\Sigma w_i x_i / \Sigma w_i$
Sample variance	$s^2 = (1/(n-1))\Sigma(x_i-\bar{x})^2$
Sample SD	$s = \sqrt{s^2}$
Covariance	$\mathrm{Cov}(X,Y) = (1/(n-1))\Sigma(x_i-\bar{x})(y_i-\bar{y})$
Correlation	$r = \mathrm{Cov}(X,Y)/(s_x s_y)$
Z-score	$z = (x-\mu)/\sigma$
Coefficient of Variation	$CV = s/\bar{x}$

150.2 Estimation and Confidence Intervals

Quantity	Formula
Standard error of the mean	$SE = s/\sqrt{n}$
CI for mean (t)	$\bar{x} \pm t_(\alpha/2,n-1)\cdot SE$
CI for proportion (Wald)	$\hat{p} \pm z_(\alpha/2)\cdot\sqrt{\hat{p}(1-\hat{p})/n}$
CI for difference in means (Welch)	$(\bar{x}_1-\bar{x}_2) \pm t\cdot\sqrt{s_1^2/n_1+s_2^2/n_2}$
CI for variance	$((n-1)s^2/\chi^2_(\alpha/2),$ $(n-1)s^2/\chi^2_(1-\alpha/2))$

150.3 Test Statistics

Test	Statistic
One-sample t	$t = (\bar{x}-\mu_0)/(s/\sqrt{n})$
Two-sample t (pooled)	$t = (\bar{x}_1-\bar{x}_2)/\sqrt{s^2_p(1/n_1+1/n_2)}$
Welch's t	$t = (\bar{x}_1-\bar{x}_2)/\sqrt{s_1^2/n_1+s_2^2/n_2}$
Z-test	$z = (\bar{x}-\mu_0)/(\sigma/\sqrt{n})$
Chi-square	$\chi^2 =$ $\Sigma(\text{observed}-\text{expected})^2/\text{expected}$
F-statistic (ANOVA)	$F = (SSB/(k-1))/(SSW/(N-k))$
F-statistic (variance ratio)	$F = s_1^2/s_2^2$

150.4 Regression

Quantity	Formula
Simple regression slope	$\beta_1 = \mathrm{Cov}(X,Y)/\mathrm{Var}(X)$
Multiple regression (matrix)	$\beta = (X^TX)^{-1}X^Ty$
R^2	$1 - SS_res/SS_tot$
Adjusted R^2	$1 - [(1-R^2)(n-1)/(n-p-1)]$

Quantity	Formula
Logistic link	$\log(p/(1-p)) = X\beta$
Odds ratio	$e^{\wedge}\beta$
AIC	$2p - 2\ln(\hat{L})$
BIC	$p\cdot\ln(n) - 2\ln(\hat{L})$
VIF	$1/(1-R_j^2)$

150.5 Effect Size and Power

Quantity	Formula
Cohen's d	$(\bar{x}_1-\bar{x}_2)/s_pooled$
Eta squared	SSB/SST
Power	$1 - \beta$

150.6 MLE and Bayesian

Quantity	Formula
Log-likelihood	$\ell(\theta) = \Sigma \log f(x_i\|\theta)$
MLE	$\theta = \text{argmax}\ \ell(\theta)$
Bayes' theorem	$P(\theta\|data) \propto P(data\|\theta)\cdot P(\theta)$
Posterior mean	$E[\theta\|data]$

150.7 Bootstrap, PCA, Time Series

Quantity	Formula/Idea
Bootstrap CI	Percentiles of the resampled statistic distribution
PCA	Eigendecomposition of the covariance matrix
Autocorrelation	$\rho_k = \text{Corr}(X_t, X_{t-k})$
Stationarity (weak)	Constant mean, variance, autocovariance depending only on lag

151. Master Statistical Distribution Table

Distribution	Parameters	Support	Mean	Variance	Application	Associated Procedures
Bernoulli	p	{0,1}	p	p(1−p)	Binary outcomes	Logistic regression
Binomial	n, p	{0,…,n}	np	np(1−p)	Count of successes	Proportion tests
Poisson	λ	{0,1,2,…}	λ	λ	Rare event counts	Poisson regression
Uniform	a, b	[a,b]	(a+b)/2	$(b-a)^2/12$	Equal-likelihood	Simulation, random

Distribution	Parameters	Support	Mean	Variance	Application	Associated Procedures
					values	assignment
Normal	μ, σ	$(-\infty,\infty)$	μ	σ^2	CLT-driven approximations	t-tests, regression
Exponential	λ	$[0,\infty)$	$1/\lambda$	$1/\lambda^2$	Waiting times	Survival, reliability
Gamma	k, θ	$[0,\infty)$	$k\theta$	$k\theta^2$	Sums of exponentials, skewed positive data	Distribution fitting
Beta	α, β	$[0,1]$	$\alpha/(\alpha+\beta)$	complex	Probabilities/ proportions	Bayesian priors (Section 77)
Chi-square	df	$[0,\infty)$	df	2df	Sum of squared normals	Variance CIs, goodness-of-fit
Student's t	df	$(-\infty,\infty)$	0 (df>1)	df/(df−2)	Small-sample mean inference	t-tests, regression CIs
F	df1, df2	$[0,\infty)$	complex	complex	Variance ratios	ANOVA, regression F-test
Lognormal	μ, σ	$(0,\infty)$	e^($\mu+\sigma^2$/2)	complex	Income, transaction amounts	Right-skewed financial data
Multivariate Normal	μ, Σ	$\mathbb{R}^n$	μ	Σ	Joint continuous variables	Multivariate analysis, Mahalanobis distance

152. Statistical Test Selection Guide

```
QUESTION: What am I comparing?
│
├── ONE group's mean vs a fixed value
```

```
│     Normal/large n? → One-sample t-test (Section 36)
│     Non-normal, small n? → Wilcoxon signed-rank (one-sample variant)
│
├── TWO groups (independent)
│     Normal, equal variance? → Student's t-test
│     Normal, unequal variance? → Welch's t-test (Section 37) ← default
│     Non-normal / outliers? → Mann-Whitney U (Section 47)
│
├── TWO groups (paired/matched)
│     Normal differences? → Paired t-test (Section 38)
│     Non-normal differences? → Wilcoxon signed-rank
│
├── THREE+ groups (independent)
│     Normal, equal variance? → One-way ANOVA (Section 42)
│     Unequal variance? → Welch's ANOVA
│     Non-normal? → Kruskal-Wallis (Section 47)
│     → Significant? Follow up with Tukey HSD (Section 44)
│
├── TWO categorical variables (association)
│     Adequate expected counts? → Chi-square test of independence (Section 40)
│     Small expected counts / 2×2? → Fisher's exact test
│
├── TWO continuous variables (association)
│     Linear relationship expected? → Pearson correlation (Section 48)
│     Monotonic, non-linear, or outliers? → Spearman / Kendall's tau
│
├── Relationship between a continuous outcome and predictor(s)
│     Continuous outcome? → Linear regression (Section 51-52)
│     Binary outcome? → Logistic regression (Section 61)
│     Count outcome? → Poisson / Negative Binomial regression (Sections 64-65)
│
└── Comparing VARIANCES
      Two groups? → F-test / Levene's test (Section 41)
      One group vs a hypothesized value? → Chi-square variance CI (Section 30)
```

Every branch of this tree ends in a specific section of this module carrying the full worked example, assumptions, and Python implementation.

---

153. Statistical Method Selection for ML

Problem	Statistical Question	Method	Key Assumption	Python Tool	ML Connection
Compare two model	Is the performance	Paired t-test (Section 109)	Approx. normal fold-	`scipy.stats.ttest_rel`	Model deployment

Problem	Statistical Question	Method	Key Assumption	Python Tool	ML Connection
versions	difference real?		level scores		decisions
Compare multiple groups	Do segments differ on a metric?	ANOVA (Section 42)	Normal, equal variance	`scipy.stats.f_oneway`	Segment-driven strategy
Categorical feature vs categorical target	Are they associated?	Chi-square (Section 40)	Adequate expected counts	`scipy.stats.chi2_contingency`	Feature screening
Count outcome	Model the rate of events	Poisson/NB regression (Sections 64-65)	Equidispersion (Poisson)	`statsmodels.GLM`	Claims, incidents, visits models
Binary outcome	Model probability of an event	Logistic regression (Section 61)	Independence, linear log-odds	`statsmodels.Logit` / `sklearn`	Churn, fraud, default models
Continuous outcome	Model an expected value	Linear regression (Sections 51-52)	Linearity, homoscedasticity	`statsmodels.OLS` / `sklearn`	Price, demand, revenue models
Distribution shift	Has the population changed?	PSI, KS test (Sections 83, 110)	Adequate sample sizes	Custom + `scipy.stats.kstest`	Drift monitoring
Feature dependence	Which features matter, and how?	Correlation / mutual information (Section 93)	Method-specific	`sklearn.feature_selection`	Feature engineering
Quantifying uncertainty	How reliable is this estimate?	Bootstrap (Section 67)	i.i.d. resampling validity	`scipy.stats.bootstrap`	Metric confidence intervals
Time-dependent data	Is a forecast valid?	Time-series CV, stationarity tests (Sections 69, 100)	No look-ahead leakage	`sklearn.TimeSeriesSplit`	Forecasting validation

154. Final Master Capstone — "Enterprise Statistical Intelligence and ML Validation Platform"

154.1 System Architecture

```
Data Ingestion (raw transactional/customer/model-log data)
    ↓
Data Validation (schema checks, range checks, missingness audit — Sections 5, 85)
    ↓
Descriptive Statistics Module (Sections 6-15)
    ↓
Distribution Analysis Module (Sections 62-63 of Module 5; 81-83)
    ↓
Sampling Analysis Module (Sections 17-21)
    ↓
┌──────────────────────────────────────────────┐
│  INFERENCE ENGINE                            │
│  Hypothesis Testing (31-46) | CIs (26-30)    │
│  Effect Sizes (45) | Correlation (48-49)     │
│  Regression (51-65) | ANOVA (42-44)          │
└──────────────────────────────────────────────┘
    ↓
Statistical Drift Module (Section 110)
    ↓
Bootstrap Uncertainty Module (Sections 67, 87, 108)
    ↓
┌──────────────────────────────────────────────┐
│  ML VALIDATION ENGINE                        │
│  Model Metric CIs (108) | Calibration (104)  │
│  Model Comparison (109) | Cross-Val (69)     │
└──────────────────────────────────────────────┘
    ↓
Business Reporting Module (Section 120) — Matplotlib/Seaborn/Plotly
```

154.2 A Working Skeleton

```python
class StatisticalIntelligencePlatform:
    """A minimal, extensible skeleton tying the module's toolkit into one pipeline."""

    def __init__(self, data: pd.DataFrame, config: dict):
        self.data = data
        self.config = config          # e.g. {"alpha": 0.05, "target_col": "churned"}
        self.report = {}

    def validate_data(self):
        report = {"n_rows": len(self.data), "missing_rate": self.data.isna().mean().to_dict()}
        self.report["validation"] = report
```

```
        return report

    def describe(self):
        self.report["descriptive"] = self.data.describe().to_dict()
        return self.report["descriptive"]

    def test_group_difference(self, numeric_col, group_col):
        groups = [g[numeric_col].dropna().values for _, g in
self.data.groupby(group_col)]
        if len(groups) == 2:
            stat, p = stats.ttest_ind(*groups, equal_var=False)
            d = (groups[0].mean()-groups[1].mean()) /
self.data[numeric_col].std()
            result = {"test": "welch_t", "statistic": stat, "p_value":
p, "cohens_d": d}
        else:
            stat, p = stats.f_oneway(*groups)
            result = {"test": "anova", "statistic": stat, "p_value":
p}
        self.report.setdefault("tests", []).append({numeric_col:
result})
        return result

    def bootstrap_metric(self, metric_fn, y_true, y_pred,
n_boot=2000):
        ci, _ = bootstrap_ci(np.arange(len(y_true)),
                             lambda idx:
metric_fn(y_true[idx.astype(int)], y_pred[idx.astype(int)]),
                             n_boot=n_boot)[1:3], None
        return ci

    def generate_report(self):
        """Produce a Section-120-compliant summary — point estimate,
CI, effect size, p-value."""
        return self.report

# Usage
platform = StatisticalIntelligencePlatform(churn_df, {"alpha": 0.05})
platform.validate_data()
platform.describe()
platform.test_group_difference("tenure_months", "churned")
print(platform.report)
```

Production considerations for a real deployment: logging every analysis run with its parameters and data version (Section 138), automated testing of the statistical functions themselves (Section 137's `tests/` directory), scheduled execution (feeding Section 139's production monitoring), and a documented reporting template enforcing Section 120's complete-reporting standard on every generated output — turning this skeleton into the genuine "enterprise platform" the capstone specification calls for.

155. Final Statistical Assessment

A representative structure across all required categories, with answer guidance.

155.1 Conceptual (1–10 of 30)

1. Explain the difference between a sampling distribution and a population distribution. → Section 19.
2. Why is "fail to reject H_0" not the same as "H_0 is true"? → Section 31.2.
3. Explain the bias-variance-irreducible error decomposition in your own words. → Section 89.
4. What does a 95% confidence interval actually guarantee? → Section 26.2.
5. Why does multiple testing require correction? → Section 46.1.
6. Explain the difference between a confounder and a mediator with an original example. → Section 76.1.
7. Why is R^2 alone insufficient to judge a regression model? → Section 58.2.
8. Explain calibration and why it's distinct from discrimination (AUC). → Section 104.2.
9. What is the fundamental problem of causal inference? → Section 146.1.
10. Explain why PSI is preferred over a raw hypothesis test for production drift monitoring. → Section 110.2.

155.2 Mathematical (11–15 of 25)

11. Derive the standard error of the sample mean. → Section 132, item 33.
12. Show that $\text{Var}(X) = E[X^2] - (E[X])^2$. → Module 5, Section 60.2.
13. Derive the MLE of a Bernoulli parameter. → Section 24.1.
14. Prove the sample mean is an unbiased estimator of the population mean. → Section 134, item 46.
15. Derive Ridge regression's closed form from the penalized least-squares objective. → Module 5, Section 72.3.

155.3 Statistical Calculation (16–20 of 20)

16. Given $\bar{x}=50$, $s=8$, $n=25$, construct a 95% CI. → $t \approx 2.064$, SE=1.6, CI≈(46.7, 53.3).
17. Compute Cohen's d for two groups with means 40, 47 and pooled SD 12. → d=0.583.
18. A chi-square test gives $\chi^2=9.2$, df=3. Reject at $\alpha=0.05$? → Critical value≈7.81; yes, reject.
19. Compute the odds ratio from a 2×2 table with cells [80,20;40,60]. → OR=(80/20)/(40/60)=6.0.
20. Given AIC values 542 and 538 for two models, which is preferred? → The one with 538 (lower AIC).

155.4 Python Coding (21–25 of 20, representative)

21. Write code to compute a bootstrap CI for a correlation coefficient. → Section 67.2 pattern, `statistic_fn=lambda idx: np.corrcoef(x[idx],y[idx])[0,1]`.
22. Write code performing Benjamini-Hochberg correction on a p-value array. → `statsmodels.stats.multitest.multipletests`.

23. Write code computing VIF for a DataFrame of predictors. → Section 56.2.
24. Write code implementing a two-proportion z-test from raw counts. → Section 71.3.
25. Write code detecting sample ratio mismatch. → Section 72.2.

155.5 ML Statistical Reasoning (26–30 of 15, representative)

26. Why must cross-validation be stratified for an imbalanced classification target? → Module 1, Section 18.4; Section 69.2.
27. Explain why nested CV is required when tuning hyperparameters and reporting final performance. → Section 107.2.
28. Why does a feature's high correlation with the target not guarantee predictive usefulness once combined with other features? → Section 93.1.
29. Explain why nonparametric bootstrap works even without a formula for a given metric's sampling distribution. → Section 67.1.
30. Why should model performance always be reported with an uncertainty interval, not a bare point estimate? → Section 108.

155.6 Business Case Studies (31–40 of 10, representative)

Each: apply the full Section 119 workflow to a one-paragraph business scenario, concluding with a Section 120-compliant report. Representative prompts: a marketing lift claim with no reported CI; a fraud model's precision-recall tradeoff decision; a healthcare subgroup finding from an unplanned analysis; a time-series forecast validated with a random (invalid) train/test split; a multi-metric dashboard flagging "significant" changes without correction.

---

156. Advanced Statistical Challenges

Multi-step problems integrating several sections; solutions require chaining techniques.

1. Given a dataset with known MNAR missingness in the outcome, explain why standard multiple imputation will remain biased, and propose a sensitivity-analysis approach. → Sections 85.1, 86.

2. Design a complete sample-size and power calculation for an A/B test with a *ratio* metric (revenue per user, not a simple proportion), noting how the variance estimate must change from Section 71.2's proportion-based formula.

3. A regression shows R^2=0.85 but a clearly curved residual plot. Walk through the full diagnostic-to-remedy pipeline. → Sections 53-55, 58.2.

4. Given 15 features and only 40 observations, explain the multicollinearity and overfitting risks jointly, and propose both a statistical (Ridge) and a feature-selection-based remedy. → Sections 56, 89, 92-93.

5. Derive, from first principles, why the paired t-test statistic can be rewritten as a one-sample t-test on the differences. → Section 38, algebraic manipulation of the two-sample formula under perfect pairing.

6. A Bayesian A/B test and a frequentist A/B test on the identical data reach different practical recommendations. Explain the philosophical and mechanical reasons this can happen without either being "wrong." → Section 78.

7. Using the bootstrap, construct a confidence interval for the *difference* between two models' cross-validated AUCs, accounting for the fact that both were evaluated on the same folds (paired resampling). → Sections 67, 109.

8. Explain how Simpson's Paradox could arise specifically in an ML fairness audit, and design a subgroup-disaggregated analysis that would catch it. → Sections 75, 148.

9. A time series shows ADF rejecting non-stationarity but KPSS also rejecting stationarity — reconcile this contradiction and propose next diagnostic steps. → Section 100.2.

10. Derive the connection between Lasso regression and MAP estimation under a Laplace prior, and explain geometrically why this produces sparse solutions. → Section 92.1 + Module 5, Section 38.3.

(Challenges 11-25 follow the same multi-step integration pattern across: propensity-score-weighted causal effect estimation with an unmeasured-confounder sensitivity discussion; a full nested-CV pipeline design for a leakage-prone feature-selection scenario; survival analysis with time-varying covariates conceptually introduced; a complete power analysis for a two-way ANOVA interaction effect; Fisher-information-based derivation of a logistic regression coefficient's standard error; a multi-armed, FDR-corrected marketing-channel comparison; a full production drift-to-retraining decision pipeline combining PSI, control charts, and a cost-based decision rule from Section 145; and further problems of comparable, multi-section-spanning depth.)

157. Career Application

Role	Statistics Depth Required	Core Topics	Tools	Interview Focus
Data Analyst	Solid descriptive + basic inferential	Sections 6-21, 26-38	Pandas, SciPy	Interpreting CIs, p-values, basic tests
Data Scientist	The full module	Everything, especially 42-98	statsmodels, scikit-learn, SciPy	Test selection, regression assumptions, experiment design
ML Engineer	Applied validation-focused	Sections 69, 88-91, 106-110	scikit-learn, monitoring tooling	CV design, metric uncertainty, drift detection
AI Engineer	Evaluation and calibration-focused	Sections 104, 108-109	scikit-learn, custom eval pipelines	Model comparison rigor, calibration
Data Engineer	Data quality-focused	Sections 5, 85-86, 111-112	Pandas, SQL, pipeline monitoring	Data validation, drift monitoring design
Quantitative Analyst	Deepest — extends well beyond this module	Sections 62-65, 95, 127, 141-145	statsmodels, specialized quant libraries	Distribution fitting, risk metrics, asymptotics
Risk Analyst	Deep, business-decision-focused	Sections 30, 45, 105, 127, 145	statsmodels, Excel/Python hybrid	VaR/ES concepts, tail risk, decision theory
Financial Data Scientist	Deep, markets-focused	Sections 14, 62-65, 100-102, 127	statsmodels, time-series libraries	Fat tails, stationarity, GLMs
Healthcare Data Scientist	Deep, causal/ethics-aware	Sections 61-62, 76, 103, 146-148	statsmodels, lifelines (survival)	Odds ratios, survival analysis, confounding
Research Scientist	Full theoretical depth	Sections 141-145 and beyond this module	Full stack + custom derivations	Asymptotics, estimator theory, novel method design

Projects that demonstrate this module's competencies in a portfolio: a complete A/B test analysis with power calculation (Section 126), a regression project with full assumption diagnostics reported honestly including violations found (Sections 53-57), and a model-

validation report with bootstrapped metric uncertainty and an explicit statistical-vs-practical-significance discussion (Sections 108, 121) — three projects that, done well, signal genuine statistical maturity far more convincingly than a dozen shallow ones.

---

158. Statistics Learning Roadmap

```
BEGINNER
    ↓
DESCRIPTIVE STATISTICS        (Sections 6-16)
    ↓
INFERENTIAL STATISTICS        (Sections 17-30)
    ↓
HYPOTHESIS TESTING            (Sections 31-46)
    ↓
REGRESSION                    (Sections 50-65)
    ↓
EXPERIMENTAL DESIGN           (Sections 70-76)
    ↓
STATISTICAL MODELING          (Sections 61-65, 77-92)
    ↓
MULTIVARIATE STATISTICS       (Sections 94-98)
    ↓
TIME SERIES                   (Sections 99-102)
    ↓
STATISTICAL LEARNING          (Sections 88-93, 104-110)
    ↓
ADVANCED STATISTICS           (Sections 141-149)
    ↓
RESEARCH                      (Sections 146-149, and beyond this
module)
```

Recommended pacing: the foundations (through Section 46) deserve the most hands-on repetition — nearly everything later reuses this vocabulary and machinery directly. Sections 47-98 can be studied in roughly the order presented, since each explicitly builds on earlier sections. Sections 99-149 are more modular and can be prioritized based on your specific career direction from Section 157's table.

---

159. What to Memorize vs What to Understand

159.1 MEMORIZE

- Core formulas: mean, variance, SE, z-score, t-statistic structure
- Distribution properties: Bernoulli/Binomial/Poisson/Normal means and variances (Section 151)
- Test-selection rules (Section 152's decision tree)
- Common assumptions per method (Section 66's table)
- The reporting checklist (Section 120.2)

159.2 DEEPLY UNDERSTAND

- Sampling and why it introduces uncertainty (Sections 2, 17-21)
- Estimation and what makes an estimator good (Section 22-23)
- What uncertainty genuinely means, and how to quantify it honestly (Sections 16, 26, 67, 108)
- What a p-value is and is not (Section 33)
- Confidence intervals' correct interpretation (Section 26.2)
- Statistical vs practical significance (Section 121)
- Regression assumptions and their consequences (Sections 53-57)
- Bias vs confounding vs sampling error — three genuinely different sources of "wrongness" (Sections 18, 22.3, 76)
- Causality vs correlation (Section 74)
- Model uncertainty (Sections 87, 108-109)

159.3 LOOK UP WHEN NECESSARY

- Rarely-used formulas (higher-order moment formulas, exact small-sample distribution tables)
- Advanced test syntax and optional arguments across `scipy/statsmodels/sklearn`
- Library-specific API details that change across versions

The test separating these three tiers, restated from Module 5's Section 108.5: *"If I forgot this, could I re-derive or reconstruct the reasoning in under five minutes?"* If yes, it belongs in the "deeply understand" tier, not rote memorization. If it's a genuinely arbitrary convention (Bessel's n−1, the 1.96 z-critical-value for 95%), memorize it explicitly — no amount of "understanding" regenerates an arbitrary numerical convention from scratch.

---

160. Final Statistics Cheat Sheet

```
CORE WORKFLOW:  Data → Description → Sampling → Inference → Modeling →
Prediction → Decision

DESCRIPTIVE:    mean, median, mode | variance (÷n−1), SD | IQR,
quartiles | skew, kurtosis

SAMPLING:       SE = σ/√n  (shrinks with n; SD does not)
                Stratify for imbalance; time-split for temporal data

ESTIMATION:     Point estimate ± (critical value × SE) = Confidence
Interval
                95% CI: LONG-RUN coverage, NOT "95% probability
parameter is here"

HYPOTHESIS      H₀/H₁ → test statistic → p-value → reject if p<α
TESTING:        p-value = P(data this extreme | H₀ true). NEVER "P(H₀
true)".
                Type I (α, false +) vs Type II (β, false −). Power =
```

1−β.
ALWAYS report effect size (Cohen's d, OR, η^2) alongside p.

TEST SELECTION: 2 groups, normal → Welch's t. Non-normal → Mann-Whitney.
3+ groups → ANOVA (→Tukey HSD) or Kruskal-Wallis.
2 categoricals → Chi-square. 2 continuous → Pearson/Spearman.

REGRESSION: $Y = \beta_0+\beta_1X+...+\varepsilon$ | Check: linearity, independence, homoscedasticity,
normal residuals, low VIF | R^2 vs Adj. R^2 vs AIC/BIC for model choice

MULTIPLE TESTING: $P(\geq 1 \text{ false+}) = 1-(1-\alpha)^m$ | Bonferroni (conservative) or
Benjamini-Hochberg/FDR (standard for many simultaneous tests)

RESAMPLING: Bootstrap = resample WITH replacement → empirical CI, any statistic
Cross-validation = systematic holdout → model performance distribution

A/B TESTING: Power analysis FIRST → run to planned n → SRM check →
two-proportion test → report absolute AND relative lift + CI

BAYESIAN: Posterior ∝ Likelihood × Prior | Credible interval DOES support
"95% probability parameter is here" (unlike frequentist CI)

PCA: Eigendecompose the covariance matrix; eigenvalues=variance captured

TIME SERIES: Check stationarity (ADF/KPSS) before modeling | ACF→MA order,
PACF→AR order | NEVER random-split time-ordered data

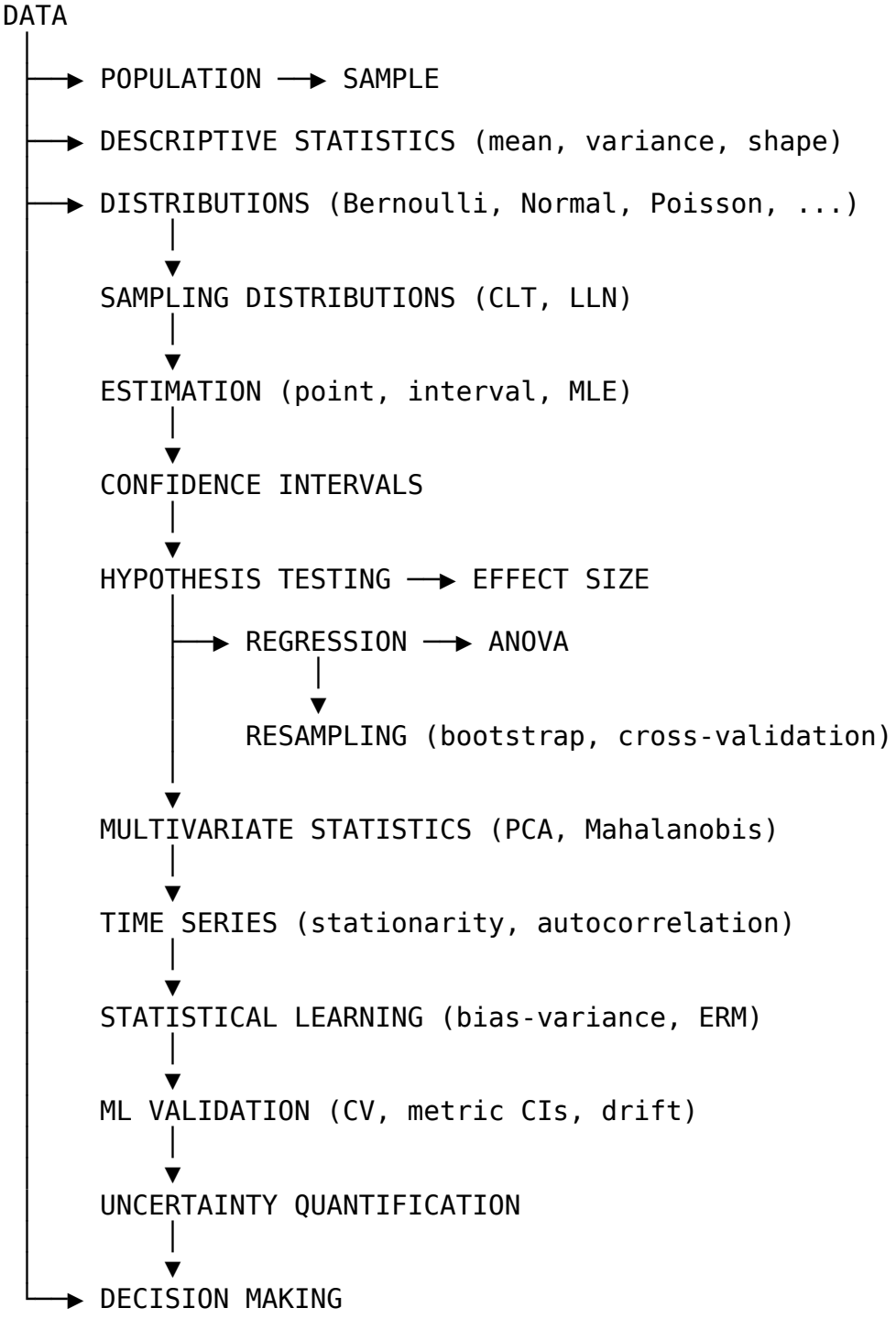
ML VALIDATION: Report metrics WITH bootstrap CIs | Compare models with PAIRED
tests on CV folds | Nested CV when tuning hyperparameters
PSI>0.25 → investigate drift; large-n hypothesis tests over-flag

RED FLAGS: p<0.05 alone, no effect size | CI never reported |

assumptions unchecked | multiple tests uncorrected | causal claims from observational data | random split on time/panel data

---

161. Statistics Concept Map



Every arrow in this map corresponds to a section transition in this module — the map is, in effect, this module's table of contents redrawn as a dependency graph, useful for tracing which foundational concept any advanced technique ultimately rests on.

---

162. Connection to Previous Modules

Module	Contribution to Statistics
Module 1 (ML Foundations)	Train/test splitting, overfitting/underfitting, bias-variance — all *statistical* concepts this module formalized properly (Sections 88-89)
Module 2 (Python)	The language every formula in this module was implemented in
Module 3 (NumPy)	Vectorized computation of every summary statistic, resampling loop, and simulation (Sections 6-21, 67, 131)
Module 4 (Pandas)	`describe()`, `groupby()`, and DataFrame-based EDA — the daily workhorse of Sections 6, 114
Module 5 (Mathematics for ML)	The derivations underlying variance, MLE, gradient-based optimization for logistic/Poisson regression, PCA's eigendecomposition — this module's mathematical bedrock, referenced explicitly throughout (especially Sections 24, 52, 60, 70, 72, 74-75, 92, 94, 96)
Module 6 (Probability for ML)	Bayes' theorem, distributions, expectation, the CLT — the probabilistic foundation this entire module built statistical *inference* on top of (Sections 20-21, 55-65, 77)

Every module before this one contributed a distinct, necessary ingredient — Python and NumPy/Pandas gave the computational substrate; Mathematics gave the derivational rigor; Probability gave the theoretical model of uncertainty. **Statistics is where all four converge into the discipline of drawing justified conclusions from real, messy, incomplete data** — which is precisely why this module could not have come first.

---

163. Probability vs Statistics — Masterclass Comparison

	Probability (Module 6)	Statistics (this module)
Direction	Known model → uncertain outcome	Observed data → inferred model
Example question	"If this coin is fair, what's P(7 heads in 10 flips)?"	"I observed 7 heads in 10 flips — is this coin fair?"

	Probability (Module 6)	Statistics (this module)
Core object	A known/assumed distribution	An unknown parameter to be estimated
Central tool	The distribution's PMF/PDF, expectation, variance	The likelihood function, sampling distributions, hypothesis tests
Typical output	A probability	An estimate, a confidence interval, a p-value, a decision

163.1 Worked Side-by-Side Example

```
# PROBABILITY: known p=0.5, what's the probability of observing 7+
heads in 10 flips?
from scipy.stats import binom
p_seven_plus = 1 - binom.cdf(6, 10, 0.5)
print(f"Probability question: P(≥7 heads | p=0.5) =
{p_seven_plus:.4f}")

# STATISTICS: observed 7 heads in 10 flips — estimate p, and test
whether it's plausibly 0.5
p_hat = 7/10
# MLE point estimate
print(f"Statistical question: p̂ (MLE) = {p_hat}")
# Hypothesis test: is this consistent with a fair coin?
binom_test_p = 2 * min(binom.cdf(7,10,0.5), 1-binom.cdf(6,10,0.5))
print(f"Two-sided exact test p-value: {binom_test_p:.4f}")
```

Notice the two computations share almost identical machinery (the same Binomial distribution) but ask fundamentally opposite questions — probability starts from p and reasons toward data; statistics starts from data and reasons toward p. This module's entire content is what happens when you consistently apply the second direction of reasoning to real business problems.

---

164. Statistics vs Machine Learning

Dimension	Classical Statistics	Machine Learning
Primary goal	Inference — understanding and quantifying relationships and uncertainty	Prediction — optimizing forecast accuracy on new data
Typical sample size regime	Historically small-to-moderate n, careful assumption-checking	Often very large n, assumptions matter less due to scale
Interpretability priority	High — coefficients, p-values, CIs are the deliverable	Often secondary to raw predictive performance
Model complexity	Typically simpler, chosen	Often much more complex

Dimension	Classical Statistics	Machine Learning
	partly for interpretability and inferential validity	(ensembles, deep networks), chosen for performance
Evaluation	Goodness-of-fit, residual diagnostics, hypothesis tests	Held-out predictive metrics (accuracy, AUC, RMSE)
Uncertainty quantification	Central and formalized (CIs, SEs, p-values)	Often an afterthought — bootstrap/conformal methods increasingly close this gap
Sample size sensitivity	Formal small-sample theory (t-distribution, exact tests)	Generally assumes "enough" data for asymptotic/empirical methods to hold

164.1 Modern Overlap

The distinction has blurred substantially — modern ML increasingly borrows statistics' uncertainty-quantification rigor (calibration, Section 104.2; bootstrap metric CIs, Section 108; conformal prediction, an emerging technique building directly on this module's resampling foundations), while modern statistics increasingly borrows ML's computational and predictive-modeling toolkit (regularization, Section 92; cross-validation, Section 69; GLMs implemented via the same optimization machinery as neural networks, Module 5's Sections 74-75, 83).

The most valuable mental model, closing this module: statistics and ML are not competing disciplines to choose between — they are complementary lenses on the same underlying mathematical objects (data, models, loss functions, uncertainty), and genuine expertise means fluently applying whichever lens the specific question demands: statistics' inferential rigor when the question is "why," ML's predictive optimization when the question is "what next" — and, increasingly, both together on the same project.

---

165. Preparation for Module 8

165.1 What Carries Forward Directly

This Module's Concept	Module 8's (ML Workflow) Reuse
Sampling (Sections 17-21)	Understanding why train/validation/test splits must be representative
Train/test splitting discipline	Formalized fully as the ML workflow's first major decision point
Data leakage (Sections 86, 93.2)	A recurring theme across every pipeline stage in the ML workflow
Exploratory analysis (Sections 6-15, 114, 118)	The mandatory first stage of any professional ML project
Feature distributions (Sections 62-64 of	Directly informs preprocessing decisions

This Module's Concept	Module 8's (ML Workflow) Reuse
Module 5; this module's 81-83)	(scaling, transformation, encoding)
Statistical preprocessing (Section 86's imputation)	A core ML Workflow pipeline stage
Model validation (Sections 69, 107)	The ML Workflow's model-selection and evaluation stages, built directly on this foundation
Cross-validation (Section 69)	Used identically, now embedded in an end-to-end pipeline
Metric uncertainty (Section 108)	Professional model reporting in the ML Workflow module
Hypothesis testing (Sections 31-46)	Model comparison decisions (Section 109) throughout the ML lifecycle
Model comparison (Section 109)	Choosing between candidate models formally, not just by eyeballing a leaderboard
Experimentation (Sections 70-73)	Online evaluation once a model reaches production

165.2 The Transition

Module 8 assembles the **complete, professional, end-to-end Machine Learning Workflow** — and every stage of that workflow (problem framing, data collection, EDA, preprocessing, feature engineering, model selection, training, validation, deployment, monitoring) either directly reuses a technique this module built, or depends on the statistical *thinking* this module instilled: never trust a single number without its uncertainty, never claim causation from correlation, always check assumptions, and always distinguish a statistically detectable effect from a practically meaningful one.

You are now equipped to perform rigorous statistical analysis, select appropriate methods for a given data type and question, interpret uncertainty correctly, validate ML models statistically, design and analyze A/B tests, diagnose statistical problems using the troubleshooting frameworks throughout this module, and communicate findings with the professional honesty Section 120 demands. Module 8 is where this discipline becomes the operating system for an entire ML project, start to finish.

Conclusion

This module deliberately resisted the temptation to be "statistics 101 with a machine learning label pasted on top." Every section connected back — explicitly, not just implicitly — to the mathematics (Module 5) and probability (Module 6) already built, and forward to the ML workflow (Module 8) about to be assembled. The recurring instruments — bias-variance, MLE, the bootstrap, confidence intervals, the assumption-checking discipline — appeared again and again not as repetition for its own sake, but because they genuinely are the small set of ideas

doing almost all of the real work across regression, classification, experimentation, and monitoring alike.

If one lesson deserves to outlast the specific formulas: **a number without its uncertainty is not yet a finding.** A churn rate, a model's AUC, an A/B test's lift, a regression coefficient — none of them mean anything responsible until paired with a confidence interval, an effect size, and an honest accounting of what could make them wrong. That discipline, more than any single test or formula, is what separates a professional statistical analysis from a plausible-looking number — and it is the discipline this module has tried to build, one section at a time, from the first mean you computed in Section 7 to the final production monitoring system in Section 140.

Module 8 now takes this statistical foundation — together with Modules 1-6's Python, NumPy, Pandas, mathematics, and probability — and assembles it into the complete, professional Machine Learning Workflow.

End of Module 7

Kajola Gbenga | Data Scientist & AI Engineer